Modern DB Specifics — Snowflake / BigQuery / Postgres 16 / MySQL 8 / SQL Server 2022

SQL Interview Prep — 2026 Edition · Learn & Excel

This file covers the modern DB content the lander explicitly promises — Postgres 15-16 specifics, Snowflake, BigQuery, MySQL 8.x, and SQL Server 2022. 160 questions across 5 sections. Each follows the same format: tags · question · 30-second answer · step-by-step thought process · edge cases · what NOT to say · common follow-up.

Sections

Section 1 — Postgres 15-16 Specifics

This section is for backend engineers and data engineers interviewing at product companies and data-role companies where Postgres is the primary OLTP store — Razorpay, Swiggy, Cred, Meesho, Zerodha, Flipkart, Zomato, PhonePe, plus analytics-leaning fintech and D2C teams. The 40 questions here assume you already know the SQL fundamentals from Files 01-04 and want to demonstrate that you can actually reason about modern Postgres internals: the MERGE statement that became generally available in Postgres 15, jsonb path expressions, logical replication improvements in Postgres 16, partitioning, generated columns, advisory locks, the bloat-vacuum cycle, and the index types unique to Postgres.

Version assumptions: every question is answered for Postgres 15 or 16. Where a feature has been around longer (jsonb GIN indexes, BRIN indexes, partial indexes) the tag explicitly notes the minimum version. Where a feature is new in 15 (MERGE GA, SQL/JSON constructors) or 16 (ANY_VALUE, logical replication on standby, bidirectional logical replication) the tag will say so. If you are on Postgres 13 or 14 in production but interviewing for a shop that runs 16, study these as forward-looking material — the planner internals and locking model have not fundamentally changed.

Indian-context tables used throughout: `orders`, `payments`, `transactions`, `users`, `merchants`, `rides`, `inventory`. Treat them as the same logical schema across questions so you can chain examples mentally.

Q1 · MERGE in Postgres 15 — Syntax and ON CONFLICT Comparison

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Walmart Labs · Postgres version: 15

The question:

> Postgres 15 finally added the MERGE statement. Walk me through the syntax and tell me when you'd reach for it instead of INSERT ... ON CONFLICT.

The 30-second answer: MERGE is a single statement that drives an INSERT / UPDATE / DELETE decision from a join between a target table and a source. ON CONFLICT only triggers on a unique-constraint violation during INSERT, so MERGE wins when your decision tree is richer than "insert, otherwise update."

Step-by-step thought process: 1. Frame the shape: `MERGE INTO target USING source ON join_condition WHEN MATCHED THEN ... WHEN NOT MATCHED THEN ... WHEN NOT MATCHED BY SOURCE THEN ...`. 2. Each WHEN clause can carry an additional AND predicate, so a single MERGE can branch into several UPDATE variants and a DELETE. 3. ON CONFLICT is shorter and faster for the simple upsert case (one INSERT, fall back to UPDATE on a unique conflict), so do not throw away the idiom you already know. 4. Reach for MERGE when (a) the source is itself a query or CTE, (b) you need DELETE as one of the branches, or (c) you need different UPDATE actions on different conditions. 5. Note that MERGE in Postgres 15 does not have the same anti-deadlock semantics as ON CONFLICT — see Q2.

Edge cases to mention: - WHEN NOT MATCHED BY SOURCE was added in Postgres 17; in 15-16 you only get WHEN MATCHED and WHEN NOT MATCHED. - A row that matches multiple source rows on the join condition raises a cardinality violation. Source must be unique on the join keys, or wrap it in a DISTINCT ON. - MERGE in 15-16 cannot use RETURNING — that landed in Postgres 17.

What NOT to say: - "MERGE is just syntactic sugar for ON CONFLICT" — it is not. The semantics differ; see the next question.

Common follow-up: *Why might MERGE deadlock where ON CONFLICT does not?*

```
MERGE INTO payments_summary t
USING (
  SELECT merchant_id, date_trunc('day', created_at) AS day, sum(amount) AS total
  FROM payments WHERE created_at >= current_date - 1 GROUP BY 1, 2
) s
ON t.merchant_id = s.merchant_id AND t.day = s.day
WHEN MATCHED AND t.total <> s.total THEN UPDATE SET total = s.total, updated_at = now()
WHEN MATCHED THEN DO NOTHING
WHEN NOT MATCHED THEN INSERT (merchant_id, day, total) VALUES (s.merchant_id, s.day, s.total);
```

Q2 · MERGE Concurrency — Why It Can Deadlock Where ON CONFLICT Cannot

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman Sachs India · Postgres version: 15

The question:

> If two sessions run the same MERGE statement concurrently, can they deadlock? Compare with INSERT ... ON CONFLICT.

The 30-second answer: Yes. MERGE in Postgres 15-16 is not atomic against concurrent inserts of the same key — between the "does the row exist?" check and the actual INSERT, another session can insert the same key and your MERGE raises a unique-violation error. ON CONFLICT, in contrast, is internally retried by the executor and is the documented safe upsert.

Step-by-step thought process: 1. MERGE evaluates the join, decides which WHEN branch to take, then executes the action. If the WHEN NOT MATCHED branch is taken but another session inserted the same key in the meantime, the INSERT will collide with the unique index and fail. 2. ON CONFLICT (INSERT first, fall back to UPDATE) lets the index serve as the arbiter — the duplicate row triggers the ON CONFLICT clause cleanly. 3. The practical fix for MERGE under concurrency: either wrap it in SERIALIZABLE isolation and retry on serialization failure, or take an explicit advisory lock on the natural key before the MERGE. 4. The other deadlock shape is intra-MERGE: if two MERGEs touch overlapping sets of rows in different orders, they can deadlock on row locks like any UPDATE-UPDATE pair. 5. For pure upserts, prefer ON CONFLICT. Reserve MERGE for multi-branch decisions where ON CONFLICT cannot express the logic.

Edge cases to mention: - ON CONFLICT DO NOTHING silently swallows the duplicate; ON CONFLICT (col) DO UPDATE acquires a row-level lock on the conflicting row. - Under READ COMMITTED, both MERGE and a plain INSERT can race; only the index check at commit time is bulletproof.

What NOT to say: - "Wrap MERGE in BEGIN ... COMMIT and it'll be safe." A transaction does not make the statement atomic against other sessions at READ COMMITTED.

Common follow-up: *Show the ON CONFLICT version of the previous query.*

Q3 · MERGE With Complex WHEN Branches — The Subscription Renewal Pattern

Tags: Difficulty: Medium · Asked at: Swiggy, Zerodha, Meesho · Postgres version: 15

The question:

> Write a MERGE that processes a subscription renewal batch. If the user is active and paying, extend the expiry. If they cancelled, mark inactive. If they're new, insert. Otherwise, do nothing.

The 30-second answer: Three WHEN branches, each with its own AND predicate on the join: WHEN MATCHED AND status='cancelled' THEN UPDATE inactive, WHEN MATCHED AND status='paid' THEN UPDATE expiry, WHEN NOT MATCHED THEN INSERT. The branches are evaluated top to bottom — first match wins.

Step-by-step thought process: 1. Order matters: put the more specific WHEN MATCHED predicates before the broader ones, because Postgres takes the first matching branch. 2. Use a CTE or VALUES list as the source so the same MERGE can ingest a parsed CSV batch or a JSON payload. 3. If a row matches no WHEN clause at all, it is silently skipped — this is the "otherwise do nothing" requirement. 4. Always set updated_at = now() in the UPDATE branches, otherwise downstream CDC and incremental ETL break.

Edge cases to mention: - A NULL on either side of the join condition behaves like an unmatched row — protect with COALESCE or IS NOT DISTINCT FROM. - If the same user_id appears twice in the source batch, the second occurrence raises a cardinality violation.

What NOT to say: - "Just chain three UPDATE statements." That defeats the point and runs three table scans.

Common follow-up: *How do you return the count of how many were inserted vs updated vs marked inactive?*

```
MERGE INTO subscriptions t
USING renewal_batch s ON t.user_id = s.user_id
WHEN MATCHED AND s.status = 'cancelled' THEN
  UPDATE SET active = false, cancelled_at = now()
WHEN MATCHED AND s.status = 'paid' THEN
  UPDATE SET expires_at = s.new_expiry, last_payment_at = now()
WHEN NOT MATCHED AND s.status = 'paid' THEN
  INSERT (user_id, plan, expires_at, active) VALUES (s.user_id, s.plan, s.new_expiry, true);
```

Q4 · MERGE Without RETURNING — How to Get the Affected Row Counts

Tags: Difficulty: Medium · Asked at: PhonePe, Cred, JP Morgan India · Postgres version: 15

The question:

> MERGE in Postgres 15-16 doesn't support RETURNING. How do you find out which rows were touched by which branch?

The 30-second answer: Either inspect the `command tag` returned by the driver (it gives you total rowcount but not per-branch breakdown), or split the work — pre-compute the partition into UPDATEs and INSERTs in a CTE, run them separately with RETURNING, and skip MERGE for this workload.

Step-by-step thought process: 1. Postgres 15-16 returns a single rowcount for MERGE — driver-side `cmd.RowsAffected()` in Go, `cursor.rowcount` in psycopg. 2. If you need per-branch numbers, the standard pattern is a writable CTE: `WITH updated AS (UPDATE ... RETURNING 1), inserted AS (INSERT ... RETURNING 1) SELECT (SELECT count(*) FROM updated), (SELECT count(*) FROM inserted)`. 3. For audit purposes, write to an audit table inside the WHEN branches — but MERGE WHEN clauses do not support sub-statements other than the INSERT / UPDATE / DELETE action, so you cannot inline an INSERT INTO audit_log there. 4. Upgrade path: Postgres 17 added RETURNING support to MERGE, including the merge_action() function that tells you which branch fired.

Edge cases to mention: - If your driver only reads the command tag, MERGE returns `MERGE n` not `UPDATE n` or `INSERT n` — code that branches on the command tag will break. - Triggers on the target table fire normally — you can use a row-level AFTER trigger to capture per-branch counts.

What NOT to say: - "Use OUTPUT clause." That is SQL Server, not Postgres.

Common follow-up: *Write the CTE-based upsert that does return per-branch counts in Postgres 15.*

Q5 · When MERGE Is the Wrong Tool — Pure Upsert Workloads

Tags: Difficulty: Easy · Asked at: Meesho, Zerodha, Walmart Labs · Postgres version: 15

The question:

> A teammate has switched all of your INSERT ... ON CONFLICT statements to MERGE because "it's the new standard." Push back.

The 30-second answer: For single-table single-key upserts, ON CONFLICT is shorter, faster, atomic against concurrent inserts, supports RETURNING, and has been the documented safe pattern for years. Use MERGE only when its multi-branch power is actually needed.

Step-by-step thought process: 1. Performance: ON CONFLICT goes through a tighter executor path tuned for the upsert case. 2. Concurrency: ON CONFLICT is internally race-safe; MERGE is not (see Q2). 3. Ergonomics: ON CONFLICT returns RETURNING in 15-16; MERGE does not. 4. Maintenance: ON CONFLICT is supported on every Postgres version your team is likely to run; MERGE is 15+, and a downgrade for any reason breaks the code. 5. Use the right tool: MERGE for richer decision trees, ON CONFLICT for upserts.

Edge cases to mention: - If you need INSERT ON CONFLICT DO UPDATE that references columns from the proposed INSERT row, use EXCLUDED — `SET amount = EXCLUDED.amount`.

What NOT to say: - "MERGE is faster." It is not, for the simple upsert.

Common follow-up: *Write the ON CONFLICT equivalent of the renewal MERGE in Q3.*

Q6 · jsonb vs json — Why Always jsonb for Operational Workloads

Tags: Difficulty: Easy · Asked at: Razorpay, Cred, Swiggy · Postgres version: 9.4+

The question:

> What is the difference between json and jsonb, and which one should I use to store webhook payloads?

The 30-second answer: json stores the raw text exactly as supplied, including whitespace and duplicate keys. jsonb parses to a binary tree, deduplicates keys, drops whitespace, sorts keys internally for efficient access, and supports GIN indexes and the rich path operators. Use jsonb for webhooks; only use json when you must preserve byte-exact original input.

Step-by-step thought process: 1. jsonb operations (->, ->>, @>, @?, @@) are all faster than the json equivalents because parsing is one-time at write. 2. jsonb supports GIN indexing with both jsonb_ops (default) and jsonb_path_ops (smaller, faster for @>). 3. json is the right call only if your downstream system is sensitive to the exact original string — e.g. you are computing an HMAC over the original webhook body for replay verification, in which case store it as bytea or text instead. 4. jsonb does not preserve key order or whitespace. If a downstream consumer cares, you have a bug upstream.

Edge cases to mention: - jsonb has a 1 GB hard limit per value, like all variable-length Postgres types. Practical limit is much lower because of TOAST overhead — keep individual jsonb values under a few MB. - jsonb NULL inside the JSON ("null") is different from SQL NULL. `jsonb '{"x": null}' -> 'x'` returns `jsonb 'null'`, not SQL NULL.

What NOT to say: - "json is faster because there's no parsing." It is faster to write, but every read re-parses.

Common follow-up: *Why might a GIN index on jsonb_path_ops be smaller and faster than the default jsonb_ops?*

Q7 · -> vs ->> vs #> vs #>> — Which Operator Returns What

Tags: Difficulty: Easy · Asked at: Swiggy, Meesho, Zerodha · Postgres version: 9.4+

The question:

> What is the difference between `payload -> 'user' -> 'id'` and `payload ->> 'id'` and `payload #>> '{user,id}'` ?

The 30-second answer: The single arrow (`->`) returns jsonb. The double arrow (`->>`) returns text. The hash variants (`#>` and `#>>`) take a path array and return jsonb or text respectively. Use `->>` when you want to compare or cast; use `->` when you want to keep chaining.

Step-by-step thought process: 1. `payload -> 'user'` returns the user sub-object as jsonb, ready to chain. 2. `payload -> 'user' ->> 'id'` returns the id as text. 3. `payload ->> 'user'` returns the entire user sub-object stringified as text — usually not what you want. 4. `payload #> '{user,id}'` is the path-array form of chained `->`. 5. To compare to a number, cast: `(payload -> 'user' ->> 'id')::bigint = 42`.

Edge cases to mention: - Indexing a string column for equality is straightforward; indexing the result of a numeric cast requires an expression index: `CREATE INDEX ON orders ((payload -> 'user' ->> 'id')::bigint)`. - For arrays, `->` takes an integer: `payload -> 'items' -> 0` for the first item.

What NOT to say: - "->> is just -> followed by ::text." It is not — `->` returns jsonb, casting that to text gives you a JSON-encoded string with surrounding quotes for string values.

Common follow-up: *Show how to extract all the order IDs from `payload->'items'` and sum their amounts.*

```
SELECT id,
       payload -> 'user' ->> 'email' AS email,
       (payload -> 'amount' ->> 'value')::numeric AS amount_rupees
FROM webhook_events
WHERE payload @> '{"event_type": "payment.captured"}'
ORDER BY received_at DESC LIMIT 100;
```

Q8 · @> @? @@ — The Containment and Path Operators

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Walmart Labs · Postgres version: 12+

The question:

> Walk me through `@>`, `@?`, and `@@`. When do you reach for each one?

The 30-second answer: `@>` tests if the left jsonb contains the right jsonb (subtree match). `@?` tests if a JSON path expression returns any result. `@@` evaluates a JSON path predicate and returns true / false. Containment is the workhorse for indexed filters; the path operators handle conditional structural queries.

Step-by-step thought process: 1. `payload @> '{"status": "captured"}'` is the indexable filter for "find rows whose JSON contains this key/value." Backed by GIN. 2. `payload @? '$.items[*] ? (@.qty > 5)'` returns true if any item has qty greater than 5. Useful for "any element matches" semantics. 3. `payload @@ '$.user.id == 42'` evaluates a boolean predicate. Equivalent to `@?` of the same path with a filter. 4. The `@?` and `@@` operators were added in Postgres 12, when SQL/JSON path arrived. Postgres 13-16 expanded the path language with more functions (jsonb_path_query, jsonb_path_exists, jsonb_path_match). 5. For indexing, `@>` lights up jsonb_ops or jsonb_path_ops GIN indexes; `@?` and `@@` need a GIN index with jsonb_path_ops or a specifically expression-indexed path.

Edge cases to mention: - `@>` is order-insensitive but exact on values — `'{"a": 1}' @> '{"a": 1.0}'` is true, but `'{"a": "1"}' @> '{"a": 1}'` is false. - Path expressions are strict by default — a missing key raises an error. Use lax mode (the default in jsonb_path_query) or wrap with `?` filters.

What NOT to say: - "@> is the same as ->." It is not — `@>` is containment, `->` is element extraction.

Common follow-up: *Write a query that finds all orders where any item has both qty > 5 and price < 100, indexed.*

Q9 · jsonb_path_query and the SQL/JSON Standard

Tags: Difficulty: Medium · Asked at: Cred, Goldman Sachs India, Meesho · Postgres version: 12+

The question:

> Show me how to extract all SKUs from a nested order payload using jsonb_path_query, and explain when this beats unnesting with `->`.

The 30-second answer: `jsonb_path_query(payload, '$.items[*].sku')` is a set-returning function that walks a JSONPath expression and returns one row per match. It beats the `->` chain when the structure is variable depth or when you need filter predicates inline.

Step-by-step thought process: 1. JSONPath is a separate language from SQL. The dollar sign refers to the document root; `[*]` is array wildcard; `?(@.predicate)` is a filter. 2. `jsonb_path_query` returns jsonb values; `jsonb_path_query_first` returns the first match; `jsonb_path_query_array` returns all matches packed into a jsonb array. 3. Use it inside a `CROSS JOIN LATERAL` to project per-match rows alongside the parent row. 4. For exists-only checks, use `@?` instead of jsonb_path_query inside an EXISTS — cleaner and indexable. 5. The path language supports arithmetic, type checks, and string functions, all of which run inside Postgres without round-tripping to the application.

Edge cases to mention: - Strict mode raises errors on missing keys or type mismatches; lax mode silently returns empty. The path functions default to lax — pass `'strict'` explicitly when you want to fail loudly. - jsonb_path_query is volatile-stable and may break parallel-safe planning in some cases; check EXPLAIN if a query that should parallelise refuses to.

What NOT to say: - "JSONPath is the same as XPath." It is not — different syntax, different semantics.

Common follow-up: *How would you index this query if you need it to scale to 100M payloads?*

```
SELECT o.id, sku.value AS sku
FROM orders o,
     jsonb_path_query(o.payload, '$.items[*].sku') AS sku(value)
WHERE o.created_at >= current_date - 7;
```

Q10 · jsonb_set and the Immutable-Update Pattern

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Zerodha · Postgres version: 9.5+

The question:

> Update a single nested key inside a jsonb column without losing the rest. What does jsonb_set do, and what is the gotcha around missing paths?

The 30-second answer: `jsonb_set(target, path_array, new_value, create_if_missing)` returns a new jsonb with one path overwritten. The gotcha: by default it does create the key if missing, but if any intermediate object in the path does not exist, it does nothing. Use `jsonb_set_lax` (Postgres 13+) for fine control over NULL handling.

Step-by-step thought process: 1. Path is a text array: `'{user,email}'`, not a string. 2. The fourth argument defaults to true (create missing leaf). Pass false to make the function a no-op when the leaf is missing — useful when you want "update if present." 3. To remove a key, use the `-` operator (`payload - 'key'`) or `#-` for nested removal. 4. To chain multiple updates: `jsonb_set(jsonb_set(payload, '{a}', '1'), '{b}', '2')`. For many updates, `payload || '{"a": 1, "b": 2}'::jsonb` (concatenation) is cleaner and merges shallowly. 5. Concatenation only merges at the top level — nested merges still need jsonb_set or a custom plpgsql merge function.

Edge cases to mention: - If the value being set is SQL NULL, default behaviour is to leave the row unchanged (Postgres 13+ via jsonb_set_lax with `'use_json_null'`). Old code may write SQL NULL into the whole column. - Writing to a jsonb column on every update bloats the TOAST chain — for hot keys consider a separate column.

What NOT to say: - "jsonb is immutable so I cannot update it." You replace the whole column value; the binary form just gets rewritten.

Common follow-up: *Write a query that bumps `payload->'counts'->'attempts'` by 1 for a given row.*

```
UPDATE webhook_events
SET payload = jsonb_set(
  payload,
  '{counts,attempts}',
  to_jsonb(COALESCE((payload -> 'counts' ->> 'attempts')::int, 0) + 1)
)
WHERE id = $1;
```

Q11 · GIN Indexes on jsonb — jsonb_ops vs jsonb_path_ops

Tags: Difficulty: Hard · Asked at: Cred, Razorpay, Walmart Labs · Postgres version: 9.4+

The question:

> You have a 200M-row events table and you want fast `@>` lookups. Compare jsonb_ops and jsonb_path_ops GIN indexes.

The 30-second answer: jsonb_ops (the default) indexes every key and every value separately, supports `@>`, `@?`, `@@`, `?`, `?&`, `?|`. jsonb_path_ops indexes only hashes of full key-value paths, supports only `@>`, is roughly half the size, and is noticeably faster for containment queries. Pick jsonb_path_ops when your only filter is `@>`.

Step-by-step thought process: 1. `CREATE INDEX ON events USING gin (payload)` builds a jsonb_ops index. 2. `CREATE INDEX ON events USING gin (payload jsonb_path_ops)` builds the smaller, faster, containment-only variant. 3. The size difference is significant — jsonb_path_ops can be 40-50% the size of jsonb_ops for typical payloads, which means more of it fits in shared_buffers. 4. If you also need key-exists queries (`payload ? 'fraud_flag'`), you need jsonb_ops. 5. Building either is slow on large tables — use CREATE INDEX CONCURRENTLY to avoid locking writes.

Edge cases to mention: - For very selective filters on a single nested key, an expression index on that path is faster than either GIN: `CREATE INDEX ON events ((payload ->> 'merchant_id'))` . - GIN indexes have a `fastupdate` parameter that defers list-to-tree merging; on write-heavy tables this gives bursts of slow VACUUM. Tune `gin_pending_list_limit` accordingly.

What NOT to say: - "GIN works the same as B-tree." Completely different — GIN is an inverted index, structured for many-to-one column-to-row mapping.

Common follow-up: *Why is partial-GIN (`WHERE event_type = 'payment'`) often the right move?*

Q12 · JSON_TABLE-Style Patterns Before Postgres 17

Tags: Difficulty: Medium · Asked at: Goldman Sachs India, JP Morgan India, Cred · Postgres version: 12+

The question:

> Standard SQL has JSON_TABLE for shredding JSON into relational rows. Postgres added it in 17. What do you do in Postgres 15-16 to get the same result?

The 30-second answer: Use `jsonb_to_recordset` (or `jsonb_to_record` for a single row), or a `CROSS JOIN LATERAL jsonb_path_query(...)` chain. Both let you project nested JSON arrays as named relational columns.

Step-by-step thought process: 1. `jsonb_to_recordset(payload -> 'items')` takes a jsonb array and returns SET OF RECORD. You must supply the column definition: `AS (sku text, qty int, price numeric)` . 2. For arbitrary depth or filtered extraction, use `jsonb_path_query` in a LATERAL join — it returns one row per match. 3. For a single nested object, `jsonb_to_record(payload -> 'user') AS u(id bigint, email text)` projects fields. 4. Wrap the extraction in a CTE so the rest of the query sees clean relational columns. 5. In Postgres 17, JSON_TABLE replaces all of this with a SQL-standard syntax. For 15-16, the LATERAL form is the idiomatic answer.

Edge cases to mention: - jsonb_to_recordset silently skips rows that do not match the column types in lax mode — pass `'strict'` to fail loudly during ETL. - The column definition list is mandatory and verbose; consider a view that wraps it.

What NOT to say: - "Just use JSON_TABLE." It does not exist in Postgres 15-16.

Common follow-up: *If the JSON array is very wide and you only need two fields, is jsonb_to_recordset still optimal?*

```
SELECT o.id, i.sku, i.qty
FROM orders o,
     jsonb_to_recordset(o.payload -> 'items') AS i(sku text, qty int)
WHERE o.merchant_id = $1 AND o.created_at >= current_date - 7;
```

Q13 · JSON Aggregates — jsonb_agg, jsonb_object_agg, and the GROUP BY Trick

Tags: Difficulty: Medium · Asked at: Swiggy, Meesho, Razorpay · Postgres version: 9.5+

The question:

> Collapse a one-to-many orders-items relationship into one row per order with a JSON array of items. Show two ways.

The 30-second answer: `jsonb_agg(row_to_jsonb(items))` groups items per order into a JSON array; `jsonb_object_agg(items.sku, items.qty)` groups them into a JSON object keyed by SKU. Both are aggregate functions and work like sum() inside a GROUP BY.

Step-by-step thought process: 1. `row_to_jsonb(t)` (or `to_jsonb(t)`) turns a row into a jsonb object — column names become keys. 2. Wrap that in `jsonb_agg(...)` and group by order_id. 3. To filter inside the aggregate, use the FILTER clause: `jsonb_agg(row_to_jsonb(i)) FILTER (WHERE i.status = 'active')`. 4. For ordered arrays, use `jsonb_agg(... ORDER BY i.line_no)` — useful for cart line items. 5. To produce a key-value object instead of an array, use `jsonb_object_agg(key_col, value_col)`. Duplicate keys keep the last value.

Edge cases to mention: - `jsonb_agg` of zero rows returns NULL, not `'[]'::jsonb`. Wrap with `COALESCE(jsonb_agg(...), '[]'::jsonb)`. - Large aggregates eat work_mem — for very fat result sets, materialise the inner query first.

What NOT to say: - "string_agg with commas then cast to jsonb." That breaks on quotes in the data.

Common follow-up: *How do you make the produced JSON pretty-printed?*

```
SELECT o.id,
       o.user_id,
       COALESCE(
         jsonb_agg(row_to_jsonb(i) ORDER BY i.line_no)
         FILTER (WHERE i.id IS NOT NULL),
         '[]'::jsonb
       ) AS items
FROM orders o
LEFT JOIN order_items i ON i.order_id = o.id
WHERE o.created_at >= current_date - 1
GROUP BY o.id, o.user_id;
```

Q14 · Logical Replication — What It Is and How It Differs From Streaming

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Zerodha · Postgres version: 10+

The question:

> Explain logical replication in Postgres. How is it different from streaming replication?

The 30-second answer: Streaming replication ships WAL segments byte-for-byte from primary to replica — the replica is an exact physical copy. Logical replication decodes WAL into row-level INSERT / UPDATE / DELETE events and replays them on the subscriber, which can have a different schema, different indexes, and even a different Postgres version.

Step-by-step thought process: 1. Streaming (a.k.a. physical) replication is all-or-nothing — you replicate the whole cluster. 2. Logical replication is per-publication: you choose which tables, optionally which columns, optionally with a row filter. 3. Publisher creates a PUBLICATION; subscriber creates a SUBSCRIPTION pointing at the publisher's slot. 4. Behind the scenes a logical replication slot accumulates WAL until the subscriber catches up. If the subscriber falls way behind, the publisher's pg_wal grows without bound — common production incident. 5. Use cases: zero-downtime major version upgrades, cross-region selective replication, ETL into a reporting cluster, sharding migrations.

Edge cases to mention: - Logical replication does not replicate DDL (CREATE TABLE, ALTER TABLE, etc.) in 15-16. You must run schema changes on both sides. - Sequences are not replicated by value — only the row data. After a failover, you must `setval` on the subscriber. - TRUNCATE is replicated (since Postgres 11), but only if both sides allow it.

What NOT to say: - "Logical replication is just streaming with filters." Internally completely different.

Common follow-up: *What happens if the subscriber goes offline for two days?*

Q15 · Postgres 16 — Logical Replication from a Standby

Tags: Difficulty: Hard · Asked at: Goldman Sachs India, JP Morgan India, Cred · Postgres version: 16

The question:

> What changed in Postgres 16 around logical replication, and why does it matter?

The 30-second answer: Postgres 16 added the ability to create logical replication slots and run logical replication from a physical standby. Before 16, logical replication ran only from the primary, which meant your subscriber load (read-heavy ETL traffic) added load to your primary. In 16 you can offload it to a hot standby.

Step-by-step thought process: 1. Enable `hot_standby_feedback = on` on the standby so it can hold the relevant snapshots. 2. Create the logical slot on the standby with `pg_create_logical_replication_slot`. 3. Point the subscriber at the standby instead of the primary. 4. The standby still consumes WAL from the primary; logical decoding now happens on the standby's CPU. 5. Caveats: if the standby falls behind the primary, logical replication latency grows. If you fail over the primary, the slot on the old standby is gone unless you set up failover slots (a separate feature, not in 16 core — comes from add-ons).

Edge cases to mention: - The standby must have the same major version as the primary for logical decoding to work. - `max_replication_slots` and `max_wal_senders` need bumping on the standby.

What NOT to say: - "Postgres 16 made replication bidirectional out of the box." Not true — it improved bidirectional logical replication (next question) but it is not a single toggle.

Common follow-up: *If your primary fails over, what happens to the logical slot on the standby?*

Q16 · Postgres 16 — Bidirectional Logical Replication

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Walmart Labs · Postgres version: 16

The question:

> Postgres 16 improved bidirectional logical replication. What's the setup and what conflicts do you have to worry about?

The 30-second answer: Postgres 16 added the `origin = none` option on CREATE SUBSCRIPTION (and improved replication origin handling) so you can set up two clusters each subscribing to each other

without infinite loop-back of the same change. You still own conflict resolution — two writes to the same row on both sides will collide.

Step-by-step thought process: 1. Create publications on both clusters for the same set of tables. 2. Create subscriptions on each side pointing at the other, with `origin = none` so a change replicated from cluster A back to cluster B does not bounce back to A. 3. Use replication origins to track which side originated each change. 4. Resolve conflicts in application logic — typical choices: last-write-wins (compare updated_at), per-region partitioning (each cluster owns a key range), or designate one cluster as authoritative for each row. 5. Sequences are dangerous in this setup — use UUIDs or non-overlapping sequence ranges (cluster A uses odd IDs, cluster B uses even).

Edge cases to mention: - DDL still does not replicate; schema must be kept in sync manually. - Trigger-driven secondary writes can amplify the change set — be deliberate about which triggers fire on replicated rows (use `ALTER TABLE ... ENABLE REPLICA TRIGGER ...`).

What NOT to say: - "It is multi-master." Postgres bidirectional logical replication is closer to async multi-leader; multi-master with proper conflict avoidance needs an extension.

Common follow-up: *How do you handle the case where the same primary-key INSERT happens on both clusters simultaneously?*

Q17 · Common Logical Replication Gotchas in Production

Tags: Difficulty: Hard · Asked at: Swiggy, Razorpay, Meesho · Postgres version: 10+

The question:

> List the top three things that go wrong with logical replication in production.

The 30-second answer: (1) Subscriber falls behind and the publisher's WAL fills the disk because the replication slot holds it; (2) DDL is not replicated, so schema changes silently desync the two sides; (3) tables without a primary key or REPLICA IDENTITY break UPDATE / DELETE replication.

Step-by-step thought process: 1. WAL retention: `pg_replication_slots` shows lag and retained WAL. Alert when retained WAL exceeds a threshold; consider `max_slot_wal_keep_size` (13+) to cap it and drop the slot if exceeded. 2. DDL drift: keep schema in version control, apply to both sides in the same migration job. Use pglogical or a custom DDL trigger if you must auto-replicate DDL. 3. REPLICA IDENTITY: by default Postgres uses the primary key to identify rows for UPDATE / DELETE on the subscriber. For tables without a PK, you must `ALTER TABLE ... REPLICA IDENTITY FULL` (slow — full-row comparison) or designate a unique index. 4. Other classics: long-running transactions on the publisher block WAL recycling; large initial copy of a big table can be slow — consider pg_dump + pg_restore + then start streaming. 5. Monitor `pg_stat_replication` and `pg_stat_subscription` continuously.

Edge cases to mention: - Toast columns: if a TOAST-ed column is not changed in an UPDATE, the new TOAST value is not in the WAL, and the subscriber must already have it — REPLICA IDENTITY FULL fixes this at a cost. - Sequences: not replicated by value. After failover you must align them or new INSERTs collide.

What NOT to say: - "Logical replication is fire-and-forget." It is fire-and-watch-closely.

Common follow-up: *Walk me through the alerting setup you would put on a logical replication topology.*

Q18 · CREATE INDEX CONCURRENTLY and Parallel Index Builds

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, JP Morgan India · Postgres version: 11+

The question:

> Explain CREATE INDEX CONCURRENTLY and parallel index builds. Are they the same thing?

The 30-second answer: No. CREATE INDEX CONCURRENTLY (CIC) avoids the ACCESS EXCLUSIVE lock so writes can continue during the build. Parallel index builds use multiple worker processes to build the index faster. They compose — you can CIC and still get parallel workers for B-tree indexes since Postgres 11.

Step-by-step thought process: 1. CIC scans the table twice — once to build the index, once to catch updates that arrived during the build. So it is slower than a regular CREATE INDEX, but it does not block writes. 2. Parallel index build is controlled by `max_parallel_maintenance_workers` (default 2) and the `parallel_workers` storage parameter on the table. 3. Only B-tree CIC supported parallel workers in 11; GIN, GiST, BRIN parallel index builds came later (BRIN in 16, GIN still serial as of 16). 4. CIC can fail and leave an INVALID index — check with `pg_index.indisvalid`. Drop the invalid index and retry. 5. CIC takes a SHARE UPDATE EXCLUSIVE lock — so it conflicts with itself, with VACUUM FULL, and with other DDL on the same table.

Edge cases to mention: - CIC cannot run inside a transaction block (`BEGIN`). - On very large tables, CIC can take hours; monitor `pg_stat_progress_create_index`. - A failed CIC leaves the invalid index taking disk space — drop it before retrying.

What NOT to say: - "CIC is just slower CREATE INDEX." Different concurrency contract.

Common follow-up: *What is `pg_stat_progress_create_index` and how do you use it?*

Q19 · Parallel Queries — When the Planner Picks Them and What Blocks Them

Tags: Difficulty: Medium · Asked at: Cred, Razorpay, Walmart Labs · Postgres version: 9.6+

The question:

> Walk me through Postgres parallel query. When does the planner pick parallel, and what is the most common reason a query that should parallelise does not?

The 30-second answer: The planner considers parallel when the estimated cost crosses `min_parallel_table_scan_size` / `min_parallel_index_scan_size` and the query uses parallel-safe functions. The most common blocker is a user-defined function that is not marked PARALLEL SAFE, or a query that runs inside a transaction with FOR UPDATE.

Step-by-step thought process: 1. Parallel scans: parallel sequential scan, parallel index scan, parallel index-only scan. 2. Above the scan: parallel hash join, parallel merge join, parallel aggregate. 3. The Gather (or Gather Merge) node sits at the top, collecting from workers. 4. Knobs: `max_parallel_workers_per_gather` (default 2), `max_parallel_workers` (cluster-wide cap), `parallel_setup_cost` and `parallel_tuple_cost` (how much the planner charges per worker). 5. Blockers: PL/pgSQL functions default to PARALLEL UNSAFE; CTEs that write; cursor-using queries; SELECT FOR UPDATE; foreign-data-wrapper scans (unless the FDW declares parallel-safe).

Edge cases to mention: - Parallel workers each consume work_mem — a parallel hash join with 4 workers can use 4x the memory of a serial one. - For small tables, the parallel setup cost dominates and serial is faster.

What NOT to say: - "More workers always faster." Diminishing returns past a few workers, and they compete for shared_buffers and disk bandwidth.

Common follow-up: *Show me an EXPLAIN ANALYZE where Gather makes the query slower than serial.*

Q20 · Hash Join vs Merge Join vs Nested Loop — How the Planner Chooses

Tags: Difficulty: Medium · Asked at: Goldman Sachs India, JP Morgan India, Cred · Postgres version: any

The question:

> When does the planner pick hash join, when merge join, and when nested loop?

The 30-second answer: Nested loop wins when the outer side is small and there is an index on the inner side — typical for OLTP point-lookup joins. Hash join wins when both sides are mid-to-large and fit in work_mem. Merge join wins when both inputs are already sorted on the join key (often because they come from indexes or an earlier sort).

Step-by-step thought process: 1. Nested loop cost: `outer_rows * inner_cost_per_lookup`. Cheap if outer_rows is tiny and inner is indexed. 2. Hash join: build a hash table on the smaller side in work_mem, then probe with the larger side. Cost is roughly `O(N + M)`. If the hash spills to disk (work_mem too small), it gets noticeably slower. 3. Merge join: sort both sides on the join key, then walk them in lockstep. Cost is dominated by the sort if not already sorted; otherwise it is `O(N + M)` with low constants. 4. The planner picks based on row estimates from pg_statistic. Bad stats means bad join choice — see Q22. 5. Force or disable for testing: `SET enable_nestloop = off` (etc.) — only ever for diagnosis, never in production.

Edge cases to mention: - A nested loop on a non-indexed inner side at high outer cardinality is the classic "this query worked fine until production data grew." - Parallel hash join (Postgres 11+) parallelises both the build and the probe.

What NOT to say: - "Hash join is always best." It loses on small outer with indexed inner.

Common follow-up: *Show me an EXPLAIN ANALYZE plan where the planner chose nested loop but should have picked hash.*

Q21 · EXPLAIN ANALYZE BUFFERS — What Each Number Actually Means

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred · Postgres version: any

The question:

> Read this EXPLAIN ANALYZE output for me. What is the difference between shared hit, shared read, and shared dirtied?

The 30-second answer: shared hit = pages served from shared_buffers (cache hit, fast). shared read = pages read from disk (or OS page cache). shared dirtied = pages this query modified, which the bgwriter will eventually write back. The actual disk-vs-OS-cache distinction is invisible to Postgres.

Step-by-step thought process: 1. Always run `EXPLAIN (ANALYZE, BUFFERS)` — the BUFFERS option is what makes the output diagnostic. 2. Compare shared hit to shared read: a query that consistently shows high shared read is not enjoying the shared_buffers cache. 3. shared dirtied appears on writes (UPDATE / DELETE) and on SELECTs that update hint bits — a freshly imported table often shows surprisingly high dirtied on first SELECTs. 4. local hit / read / dirtied refers to temp tables (per-session). 5. temp hit / read / written refers to spilled work_mem (sorts, hashes) — non-zero values mean your query exceeded work_mem and went to disk.

Edge cases to mention: - `temp written > 0` is your signal to either bump work_mem for this query (via `SET LOCAL`) or fix the plan. - BUFFERS without ANALYZE shows estimated reads, not actual — always pair with ANALYZE. - `EXPLAIN (ANALYZE, BUFFERS, VERBOSE, SETTINGS)` gives you the full diagnostic package.

What NOT to say: - "shared read means disk." It may be OS page cache; Postgres cannot distinguish.

Common follow-up: *A query has 0 shared hit and 50,000 shared read on first run, then 50,000 shared hit on second run. What happened?*

Q22 · ANALYZE and pg_statistic — Why Stale Stats Wreck Plans

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Meesho · Postgres version: any

The question:

> Your query was fast yesterday, slow today, same data, same Postgres version. Where do you look?

The 30-second answer: First suspect: stale statistics. The planner uses pg_statistic to estimate selectivity. If you bulk-loaded data without an ANALYZE, or if autovacuum is starved, the row-count estimates can be off by orders of magnitude, picking a nested loop where a hash join would win.

Step-by-step thought process: 1. Run `ANALYZE table_name` manually and re-EXPLAIN. If the plan changes, you found it. 2. Check `pg_stat_user_tables` — `last_analyze` and `last_autoanalyze` tell you when stats were last refreshed. 3. Compare row counts: EXPLAIN shows `(cost=... rows=N)` for estimates vs `(actual ... rows=M)` for reality. If rows estimate is off by more than 10x at any node, the planner had bad info. 4. Statistics target: default is 100. For a column with skewed distribution, bump it: `ALTER TABLE t ALTER COLUMN c SET STATISTICS 1000` then ANALYZE. 5. For correlated columns where the planner assumes independence, use extended statistics — see Q24.

Edge cases to mention: - Autovacuum runs ANALYZE based on a threshold of changed rows (`autovacuum_analyze_threshold` + `autovacuum_analyze_scale_factor`). For very large tables, the default scale factor (10%) means stats refresh only after 10% of the table changes — too slow for some workloads. - After a major version upgrade, you should run ANALYZE on the entire database before serving traffic.

What NOT to say: - "Postgres tracks stats live." It samples on ANALYZE, not on every write.

Common follow-up: *How do you tune autoanalyze to be more aggressive on a specific table?*

Q23 · Extended Statistics — CREATE STATISTICS for Correlated Columns

Tags: Difficulty: Hard · Asked at: Cred, Razorpay, Goldman Sachs India · Postgres version: 10+ (more types in 12+, 14+)

The question:

> Your filter `WHERE city = 'Bengaluru' AND pincode = '560001'` is estimating 100 rows but actually returns 50,000. The planner picks nested loop and the query crawls. Fix it.

The 30-second answer: city and pincode are correlated — every pincode is in exactly one city — but the planner multiplies their individual selectivities and underestimates. Create extended statistics: `CREATE STATISTICS s_city_pin (dependencies) ON city, pincode FROM users; ANALYZE users;` and the planner will use the functional-dependency knowledge.

Step-by-step thought process: 1. Three kinds of extended stats: `dependencies` (functional dependency, A => B), `ndistinct` (joint number of distinct combinations), `mcv` (most-common-values list across multiple columns, Postgres 12+). 2. Default planner assumption is column independence. When two columns are correlated, the multiplied selectivity underestimates row count. 3. After CREATE STATISTICS, you must ANALYZE the table for the new stats to be populated. 4. View results in `pg_statistic_ext` and `pg_stats_ext`. 5. Extended stats are most useful for: city/pincode, state/country, brand/category, model/year.

Edge cases to mention: - ndistinct stats help GROUP BY estimates — useful for hash aggregate vs sort aggregate decisions. - mcv (multi-column most common values) helps when specific combinations are way more common than independence would predict. - Extended stats have a non-zero ANALYZE cost — do not blanket-create them on every column pair.

What NOT to say: - "Just add an index." An index helps the scan but does not fix the row-count estimate that misleads the join choice.

Common follow-up: *Show me the pg_stats_ext output for the example above.*

```
CREATE STATISTICS s_users_city_pin (dependencies, ndistinct, mcv)
ON city, pincode FROM users;
ANALYZE users;
```

Q24 · Reading EXPLAIN — Spot the Plan Killer

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Walmart Labs · Postgres version: any

The question:

> Here is an EXPLAIN ANALYZE plan with a Nested Loop estimating 1 row and actually returning 200,000. What three things do you suspect?

The 30-second answer: (1) Stale or insufficient statistics on the join columns; (2) correlated predicates without extended statistics; (3) a function in the WHERE clause that the planner cannot estimate (any user-defined function, casts that defeat indexes).

Step-by-step thought process: 1. Estimate-vs-actual ratio at each node is the diagnostic — large skew at the lowest node propagates upward. 2. If the bad estimate is on a single column filter, ANALYZE + increase statistics target. 3. If it is on a multi-column filter, CREATE STATISTICS (see Q23). 4. If it is on a function call, mark the function STABLE / IMMUTABLE if it actually is, or rewrite to inline the expression so the planner can use stats. 5. Other suspects: a partial index that the planner is not using because its predicate is parameterised; a LATERAL subquery whose estimated rows are off.

Edge cases to mention: - For deeply parameterised queries (`WHERE x = $1`), the planner generates a generic plan after 5 executions — sometimes this generic plan is worse than the custom plans. Set `plan_cache_mode = force_custom_plan` for that session if needed. - `auto_explain` extension logs slow plans automatically — set up in production.

What NOT to say: - "Just rewrite as a CTE." CTEs in Postgres 12+ are inlined by default, so the rewrite changes nothing semantically.

Common follow-up: *Show me how you would use auto_explain to catch slow queries in production.*

Q25 · Declarative Partitioning — When to Reach for It and the Three Strategies

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Zerodha · Postgres version: 10+ (mature in 11+)

The question:

> When does partitioning pay off, and what are the three partition strategies?

The 30-second answer: Partition when a single table grows beyond ~100M rows and your queries reliably filter on a partition key (time, tenant_id, region). Three strategies: RANGE (most common, for time-series), LIST (for enums or known small sets), HASH (for even spread when no natural range exists).

Step-by-step thought process: 1. RANGE: `PARTITION BY RANGE (created_at)` then one partition per month. Pruning kicks in on `WHERE created_at >= ...`. 2. LIST: `PARTITION BY LIST (region)` with one partition per region — useful for multi-tenant where tenants are independent. 3. HASH: `PARTITION BY HASH (user_id) WITH (modulus 8, remainder 0..7)` — for even spread when no business-meaningful partition key exists. 4. Always use declarative partitioning, never the legacy inheritance-based form. 5. Combine: range-then-hash sub-partitioning is supported and useful for very large time-series with hot-key skew.

Edge cases to mention: - Each partition is a separate table with its own indexes. Index design must be replicated per partition (Postgres handles this automatically for indexes defined on the parent in 11+). -

Updates that move a row to a different partition (an UPDATE that changes the partition key) work in Postgres 11+ but are slow — modelling so this rarely happens is wise.

What NOT to say: - "Always partition large tables." Below ~100M rows, partitioning often hurts more than helps because of planning overhead.

Common follow-up: *Walk me through partition pruning at plan time vs run time.*

```
CREATE TABLE orders (id bigserial, created_at timestamptz, ...)
  PARTITION BY RANGE (created_at);

CREATE TABLE orders_2026_05
  PARTITION OF orders FOR VALUES FROM ('2026-05-01') TO ('2026-06-01');
```

Q26 · Partition Pruning — Plan-Time vs Execution-Time

Tags: Difficulty: Hard · Asked at: Cred, Razorpay, Goldman Sachs India · Postgres version: 11+

The question:

> What is partition pruning, and when does it happen at plan time vs execution time?

The 30-second answer: Pruning skips partitions that cannot contain matching rows. Plan-time pruning happens when the planner can resolve the predicate against the partition bounds at planning (literal values, known constants). Execution-time pruning happens when the predicate involves parameters or expressions resolved per row — common with prepared statements and parameterised joins.

Step-by-step thought process: 1. `EXPLAIN` shows pruning result: `Subplans Removed: N` at the Append node. 2. For `WHERE created_at >= '2026-05-01'`, plan-time pruning eliminates older partitions during planning — best case. 3. For `WHERE created_at >= $1`, plan-time cannot prune (does not know $1's value), so the plan includes all partitions but execution-time pruning skips them at scan time. 4. For nested loop joins where the inner side is partitioned by the join key, execution-time pruning kicks in per outer row. 5. The setting `enable_partition_pruning = on` (default true) controls plan-time. Execution-time is always on if applicable.

Edge cases to mention: - Functions on the partition key defeat pruning: `WHERE date_trunc('day', created_at) = '2026-05-26'` does not prune. Rewrite as a range. - Pruning works for RANGE, LIST, and HASH partitions; HASH pruning needs the exact value.

What NOT to say: - "Partition pruning is automatic on all queries." It needs a predicate the planner can match against partition bounds.

Common follow-up: *Why does `WHERE created_at::date = current_date` not prune?*

Q27 · Partition-Wise Join — Speeding Up Joins Across Partitioned Tables

Tags: Difficulty: Hard · Asked at: Razorpay, JP Morgan India, Cred · Postgres version: 11+

The question:

> You have two tables partitioned the same way (by month). A join between them is slow. What flag fixes it, and what is the planner doing?

The 30-second answer: Turn on `enable_partitionwise_join = on`. With it, the planner joins corresponding partitions pairwise (May vs May, June vs June) instead of unioning all partitions on each side and doing one giant join. Plan-time pruning then eliminates matching partitions per pair.

Step-by-step thought process: 1. Default is off because it can explode planning time for many-partition tables. 2. Both tables must be partitioned on equivalent keys (same column types and ranges). 3. Companion flag: `enable_partitionwise_aggregate = on` for per-partition aggregation before the final merge. 4. Memory cost: each per-partition join needs its own work_mem allocation. Watch shared memory. 5. Use when partition counts are moderate (tens, not thousands) and per-partition joins are big enough to benefit.

Edge cases to mention: - If partitions are not aligned on bounds (one table monthly, the other quarterly), partition-wise join cannot apply. - Plan compilation time scales with partition count — for 365 daily partitions per side, the planner may take seconds.

What NOT to say: - "It is on by default." It is off by default for good reason.

Common follow-up: *Show me the EXPLAIN ANALYZE plan with partition-wise join on vs off for the orders-payments join.*

Q28 · Partitioning Pitfalls — Foreign Keys, Unique Indexes, and Cross-Partition Updates

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Walmart Labs · Postgres version: 11+

The question:

> What are the three biggest partitioning gotchas to know before you propose partitioning a 500M-row table?

The 30-second answer: (1) Unique constraints must include the partition key; (2) foreign keys referencing a partitioned table need careful handling (full support arrived in Postgres 12); (3) UPDATEs that move rows across partitions are slow and acquire stricter locks.

Step-by-step thought process: 1. A unique constraint on (id) alone is not enforceable across partitions — Postgres only enforces uniqueness within each partition unless you include the partition key. So you typically need `UNIQUE (id, created_at)`. 2. Foreign keys: in Postgres 11 you could not have an FK *referencing* a partitioned table; this was fixed in 12. FKs *from* a partitioned table to a non-partitioned one have always worked. 3. Cross-partition update: changing a row's partition key UPDATEs to (in effect) DELETE + INSERT, with all the lock and TOAST costs that implies. 4. Other classics: each partition has its own indexes (more disk, more vacuum work); detach-old-partition for archival has gotchas around indexes. 5. `DETACH PARTITION CONCURRENTLY` (Postgres 14+) makes archival less disruptive.

Edge cases to mention: - A primary key on a partitioned table must include the partition column. - If you drop and recreate partitions frequently, statistics on the parent can drift.

What NOT to say: - "Partitioning is free if you set it up right." Adds maintenance burden everywhere.

Common follow-up: *How do you archive a 6-month-old partition without locking writes?*

Q29 · Generated Columns — STORED Only in Postgres

Tags: Difficulty: Easy · Asked at: Swiggy, Meesho, Razorpay · Postgres version: 12+

The question:

> Postgres supports generated columns. What is the syntax, and what is the catch versus other databases?

The 30-second answer: `GENERATED ALWAYS AS (expression) STORED`. Postgres only supports STORED — the value is computed at INSERT/UPDATE and physically stored. There is no VIRTUAL (compute-on-read) variant as of Postgres 16. The catch: every change to a base column rewrites the generated column too.

Step-by-step thought process: 1. Common use case: pre-compute a normalised email lower, a tsvector for full-text search, a denormalised total_amount = price * quantity. 2. Generated columns can be indexed like any other column — usually the whole point. 3. Cannot reference other generated columns or volatile functions. 4. Updates to base columns auto-recompute; you cannot manually UPDATE a generated column. 5. In Postgres 17, VIRTUAL generated columns landed for some use cases; in 15-16, STORED is your only option.

Edge cases to mention: - The expression must be IMMUTABLE — `now()` is not allowed. - Adding a generated column on a populated table acquires a long ACCESS EXCLUSIVE lock (it has to rewrite every row). Add the column nullable first, then backfill, then add a non-null version — common pattern.

What NOT to say: - "Use VIRTUAL for performance." Not available in 15-16.

Common follow-up: *When would you use a generated column instead of an expression index?*

```
ALTER TABLE orders
  ADD COLUMN total_amount numeric
  GENERATED ALWAYS AS (price * quantity - COALESCE(discount, 0)) STORED;
CREATE INDEX ON orders (total_amount);
```

Q30 · Generated Columns vs Expression Indexes — When to Pick Which

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Walmart Labs · Postgres version: 12+

The question:

> I have a column with a computed value used in WHERE clauses. Should I use a generated column with a normal index, or an expression index on the underlying column?

The 30-second answer: Expression index when the computed value is only used in WHERE clauses and never SELECTed — saves disk space and write cost. Generated column when the application also reads the value (SELECT total_amount FROM orders) or you need a NOT NULL constraint on the computed value.

Step-by-step thought process: 1. Expression index: `CREATE INDEX ON orders ((price * quantity))` — the planner uses it for `WHERE price * quantity > 1000`. 2. Generated column: physical storage, can be SELECTed, can have constraints (CHECK, NOT NULL). 3. Write cost: generated column writes the computed value to disk on every INSERT/UPDATE. Expression index writes the index entry but not the table column. 4. Disk cost: generated column adds a column width per row; expression index adds an index entry per row. Often similar order of magnitude. 5. Statistics: extended stats on a generated column can be more refined than on an expression — generated wins for complex planner reasoning.

Edge cases to mention: - A FILTER predicate (`WHERE x = $1 AND price * quantity > 1000`) hits the expression index only if it appears verbatim in the query — formatting matters. - Adding either to a large table is expensive (rewrites table or builds full index). Use CIC for the index and the nullable-add-and-backfill pattern for the column.

What NOT to say: - "Always generated, always indexed." Wastes write throughput.

Common follow-up: *If the formula changes, what happens to existing rows?*

Q31 · ANY_VALUE Aggregate — The Postgres 16 Convenience

Tags: Difficulty: Easy · Asked at: Meesho, Cred, Razorpay · Postgres version: 16

The question:

> Postgres 16 added the ANY_VALUE aggregate. What problem does it solve?

The 30-second answer: ANY_VALUE returns an arbitrary value from the group — useful when the SQL standard's GROUP BY rules force you to either group by a column or aggregate it, but you do not care which value you get. Replaces the awkward MIN/MAX hack and signals intent.

Step-by-step thought process: 1. Classic shape: `SELECT user_id, any_value(name), count(*) FROM users GROUP BY user_id` — name is functionally dependent on user_id, but the SQL standard does not let you select it without aggregation or grouping. 2. Before 16, you wrote `MIN(name)` or `MAX(name)` — works, but lies about intent and forces a comparison. 3. ANY_VALUE is the cleaner, faster signal: "any value from the group, I do not care which." 4. Postgres has historically been lenient about grouping (it allows columns that are functionally dependent on the grouping columns), but other DBs are stricter — ANY_VALUE makes queries portable. 5. Implementation is essentially "first value seen" — no comparison overhead.

Edge cases to mention: - The "arbitrary" return is implementation-defined and may change between executions — do not rely on which value comes back. - For deterministic first-value selection, use `(array_agg(x ORDER BY y))[1]` or `first_value(x) OVER (PARTITION BY g ORDER BY y)`.

What NOT to say: - "ANY_VALUE picks the first inserted value." Not guaranteed.

Common follow-up: *Why is MIN sometimes faster than ANY_VALUE on a particular column?*

```
SELECT u.user_id,
       any_value(u.full_name) AS name,
       count(o.id) AS orders_count,
       sum(o.total) AS lifetime_value
FROM users u JOIN orders o ON o.user_id = u.user_id
GROUP BY u.user_id;
```

Q32 · IS DISTINCT FROM — The NULL-Safe Equality

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman Sachs India · Postgres version: any

The question:

> What is IS DISTINCT FROM and when do I actually need it?

The 30-second answer: IS DISTINCT FROM is NULL-safe inequality — it treats NULL as a comparable value, so `NULL IS DISTINCT FROM NULL` is false (they are equal) and `1 IS DISTINCT FROM NULL` is true. Use it in change-detection queries, MERGE join conditions, and anywhere `<>` would mis-handle NULLs.

Step-by-step thought process: 1. Plain `=` and `<>` return NULL when either side is NULL — which is never true in a WHERE clause. So `WHERE a <> b` skips rows where either is NULL. 2. `WHERE a IS DISTINCT FROM b` treats NULLs as comparable and returns true / false, never NULL. 3. Common application: a CDC trigger comparing OLD and NEW row versions to detect actual changes — `IF NEW.x IS DISTINCT FROM OLD.x THEN ...`. 4. Also useful in MERGE join conditions if your join key can be NULL. 5. Symmetric form: `a IS NOT DISTINCT FROM b` returns true when equal-or-both-NULL.

Edge cases to mention: - Indexable: a B-tree on the column supports `IS DISTINCT FROM` only via a partial index plus rewrite. Often easier to rewrite as `(a <> b OR (a IS NULL) <> (b IS NULL))`. - In jsonb, JSON null is not SQL NULL — `IS DISTINCT FROM` does not look inside jsonb values.

What NOT to say: - "IS DISTINCT FROM is the same as <>." It is not, on NULL inputs.

Common follow-up: *Write the trigger function that logs only actual changes to the orders table.*

Q33 · IS DISTINCT FROM in Practice — Change-Detection Triggers

Tags: Difficulty: Medium · Asked at: Swiggy, Razorpay, Cred · Postgres version: any

The question:

> Write a BEFORE UPDATE trigger that only logs to an audit table when at least one of (status, total_amount, customer_id) actually changed.

The 30-second answer: The trigger body returns NEW unchanged, but conditionally inserts into audit only when at least one column differs — using IS DISTINCT FROM to handle the case where one side was NULL and the other became a value (or vice versa).

Step-by-step thought process: 1. Plain `OLD.status <> NEW.status` is wrong: if either is NULL, the comparison is NULL, and the audit row is skipped on legitimate NULL <-> value transitions. 2. `OLD.status IS DISTINCT FROM NEW.status` correctly returns true on those transitions. 3. Combine with OR for multi-column change detection. 4. Inside trigger function, return NEW (or NULL to skip the update — rarely what you want for audit). 5. Audit table should record (table, row_id, changed_columns array, before jsonb, after jsonb, changed_by, changed_at).

Edge cases to mention: - If you only want to log when *all* of N columns changed, use AND. For *any*, use OR. - For high-throughput tables, consider sample-only auditing (every Nth row) or routing to a logical-replication-fed audit cluster.

What NOT to say: - "Use OLD. <> *NEW.* to compare whole rows." Row equality with NULLs is the same NULL-trap.

Common follow-up: *Why might you compare jsonb columns with IS DISTINCT FROM and still miss "logical" changes?*

```
CREATE OR REPLACE FUNCTION orders_audit_trg() RETURNS trigger AS $$
BEGIN
  IF OLD.status IS DISTINCT FROM NEW.status
     OR OLD.total_amount IS DISTINCT FROM NEW.total_amount
     OR OLD.customer_id IS DISTINCT FROM NEW.customer_id THEN
    INSERT INTO orders_audit (order_id, before, after, changed_at)
    VALUES (OLD.id, to_jsonb(OLD), to_jsonb(NEW), now());
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Q34 · Row-Level Lock Modes — FOR UPDATE vs FOR NO KEY UPDATE vs FOR SHARE vs FOR KEY SHARE

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, JP Morgan India · Postgres version: 9.3+

The question:

> Walk me through the four row-level lock modes in Postgres and when to use each.

The 30-second answer: FOR UPDATE is the strongest — taken by `SELECT ... FOR UPDATE` and by any plain UPDATE that touches a unique-key column. FOR NO KEY UPDATE is taken by plain UPDATEs that do not touch unique keys — weaker so FK checks do not contend. FOR SHARE blocks concurrent FOR UPDATE. FOR KEY SHARE is the weakest, used internally by FK checks.

Step-by-step thought process: 1. The reason for the split: foreign-key constraint checks need to make sure the referenced row will not have its key changed. They take FOR KEY SHARE on the parent row. 2. If your UPDATE on the child does not change the FK column, it would unnecessarily block other updates on the parent if it took FOR UPDATE. So Postgres uses FOR NO KEY UPDATE on the child UPDATE — which does not conflict with FOR KEY SHARE. 3. Net effect: ordinary parent-child updates rarely block each other; they only conflict when a parent row's key is actually being changed. 4. Manual locks: `SELECT ... FOR UPDATE` for read-then-write patterns; `FOR UPDATE NOWAIT` to fail fast; `FOR UPDATE SKIP LOCKED` for work-queue patterns. 5. View what is locked: `pg_locks` joined with `pg_stat_activity`.

Edge cases to mention: - `FOR UPDATE OF table1` (not of all tables in the SELECT) restricts which table's rows get locked — useful in multi-table SELECTs. - Row-level locks are held until end of transaction; they do not auto-release on the next statement.

What NOT to say: - "FOR UPDATE is just an exclusive lock." There is a hierarchy, and the lower-conflict modes were added specifically to reduce FK contention.

Common follow-up: *What is the difference between FOR UPDATE NOWAIT and FOR UPDATE SKIP LOCKED?*

Q35 · Advisory Locks — When You Need Application-Level Mutual Exclusion

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred · Postgres version: any

The question:

> What are advisory locks and when should I use them?

The 30-second answer: Advisory locks are application-defined locks identified by an integer (or pair of integers). Postgres tracks them but assigns no semantics — your application decides what they mean. Use them for distributed mutual exclusion when you do not want to lock a row or a table.

Step-by-step thought process: 1. `pg_advisory_lock(key)` blocks until acquired; `pg_try_advisory_lock(key)` returns false if not. Released by `pg_advisory_unlock(key)` or at session end. 2. Transaction-scoped variant: `pg_advisory_xact_lock(key)` — released at COMMIT or ROLLBACK, simpler and safer. 3. Common use: ensure only one worker runs a periodic job (a "leader election"), serialise an idempotent operation, prevent concurrent MERGE on the same key. 4. The key is just an integer — hash a business key into it: `pg_try_advisory_xact_lock(hashtext('cron_job:rollup'))`. 5. Do not over-rely on advisory locks for data correctness — they enforce mutex but not isolation. Use row locks for data, advisory for coordination.

Edge cases to mention: - Advisory locks are scoped to the entire cluster, not the database. Two different databases on the same cluster share the advisory-lock keyspace. - A connection-pooler with statement-level reuse (PgBouncer transaction mode) will break session-scoped advisory locks. Use xact variant or session mode pooling.

What NOT to say: - "Advisory locks replace row locks." They serve different purposes.

Common follow-up: *Write the SQL for a single-leader cron worker using advisory locks.*

```
-- Run inside a transaction
SELECT pg_try_advisory_xact_lock(hashtext('daily_rollup_2026_05_26'));
-- If true, do the work; if false, another worker has it
```

Q36 · deadlock_timeout and log_lock_waits — Tuning for Production Visibility

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, JP Morgan India · Postgres version: any

The question:

> Two parameters: deadlock_timeout and log_lock_waits. What do they control and how should I set them in production?

The 30-second answer: deadlock_timeout (default 1s) is how long Postgres waits before running deadlock detection. log_lock_waits (default off) logs any session that waited longer than deadlock_timeout for a lock — your single best visibility tool for lock contention.

Step-by-step thought process: 1. Deadlock detection is expensive (a graph traversal), so Postgres does not run it immediately — it waits deadlock_timeout first, on the assumption that most waits resolve quickly. 2. log_lock_waits piggybacks on the same timeout: if a lock wait exceeds it, the wait is logged with the PID holding the lock and what kind of lock. 3. In production: set `log_lock_waits = on` and tune deadlock_timeout to your latency budget. 1s is fine for most apps; 100ms for low-latency payment systems. 4. Side effect of tuning down deadlock_timeout: more CPU spent on detection. Set in proportion to lock contention severity. 5. For per-statement timeout, use `lock_timeout` (raises error if lock not acquired in N ms) — better than letting a statement hang forever.

Edge cases to mention: - statement_timeout, lock_timeout, and idle_in_transaction_session_timeout are three different things. Configure all three in production. - log_lock_waits output is voluminous on contention; pair with a log aggregator.

What NOT to say: - "Reduce deadlock_timeout to detect deadlocks faster." Detection happens on conflict; the timeout is how long you tolerate waiting before the detection runs.

Common follow-up: *What should you set idle_in_transaction_session_timeout to and why?*

Q37 · GIN, GiST, and BRIN — When to Pick Each

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy · Postgres version: any (BRIN 9.5+)

The question:

> Postgres has GIN, GiST, and BRIN indexes in addition to B-tree. When does each shine?

The 30-second answer: GIN — many-to-one mappings (full-text search, jsonb containment, array contains). GiST — geometric, range, full-text, KNN. BRIN — very large, naturally-ordered tables (time-series logs) where a tiny block-range summary suffices. B-tree remains the default for ordered scalar lookups.

Step-by-step thought process: 1. GIN (Generalized Inverted Index): inverted list of values to row pointers. Big build, slow update, fast lookup. Standard for tsvector, jsonb, intarray. 2. GiST (Generalized Search Tree): balanced tree where operator support is pluggable. Used for ranges, geometry (with PostGIS), full-text, and exclusion constraints. 3. BRIN (Block Range Index): stores summary info (min/max) per block range (default 128 pages). Tiny index, near-zero maintenance, but only useful if data is physically clustered on the indexed column. Perfect for created_at on append-only logs. 4. B-tree: still the right answer for equality and range on scalar columns of modest selectivity. 5. INCLUDE columns (Postgres 11+) let B-tree act as a covering index.

Edge cases to mention: - BRIN works only if the column is correlated with physical order. If rows are inserted out of order, BRIN scans hit too many blocks. - GIN supports fastupdate (pending list) — improves write throughput but can cause VACUUM bursts. - GiST with KNN (`<->` operator) makes "nearest 10 points" queries fast.

What NOT to say: - "GIN is for jsonb only." Far broader — full-text, arrays, custom types.

Common follow-up: *For an append-only audit log with a created_at column, which index would you create on created_at and why?*

Q38 · Partial Indexes, Expression Indexes, and INCLUDE Columns

Tags: Difficulty: Medium · Asked at: Cred, Razorpay, Walmart Labs · Postgres version: any (INCLUDE 11+)

The question:

> Three index tricks: partial, expression, INCLUDE. Walk me through each with a use case.

The 30-second answer: Partial: `CREATE INDEX ON orders (user_id) WHERE status = 'pending'` — only indexes the rows the query cares about, much smaller. Expression: `CREATE INDEX ON users (lower(email))` — lets `WHERE lower(email) = $1` use the index. INCLUDE: `CREATE INDEX ON orders (user_id) INCLUDE (total_amount)` — stores extra columns in the leaf for index-only scans.

Step-by-step thought process: 1. Partial index: smaller, faster, ideal for skewed data where queries always filter on a known predicate (`status = 'pending'` , `deleted_at IS NULL`). 2. Expression index: required when your query applies a function to the column (`lower()` , `date_trunc()` , jsonb path extraction). 3. INCLUDE columns: not part of the index key (no sort or uniqueness role) but stored in the leaf so SELECTs can be index-only-scanned without a heap fetch. 4. Combine: `CREATE INDEX ON orders (user_id) INCLUDE (total_amount) WHERE status = 'pending'` — partial, with included payload column. 5. EXPLAIN shows index-only scan when the visibility map is up to date (depends on VACUUM) — without it you get an index scan with heap fetch.

Edge cases to mention: - Partial indexes are not used for queries whose predicate is not implied by the partial-index predicate. The planner must prove the query's WHERE entails the index's predicate. - Expression indexes need the same expression literally in the query — `lower(email)` matches but `email ILIKE 'X%'` does not.

What NOT to say: - "INCLUDE adds the columns to the key." It does not — they are payload, not part of ordering or uniqueness.

Common follow-up: *For a soft-delete pattern with deleted_at, what partial index would you create?*

```
CREATE INDEX idx_orders_pending_user ON orders (user_id, created_at)
  INCLUDE (total_amount, status)
  WHERE deleted_at IS NULL AND status = 'pending';
```

Q39 · Autovacuum — What It Does and When to Tune It

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, JP Morgan India · Postgres version: any

The question:

> What does autovacuum do, and what's the most common production tuning?

The 30-second answer: Autovacuum runs VACUUM (reclaim dead-tuple space, update visibility map) and ANALYZE (refresh statistics) automatically based on dead-tuple thresholds. The most common production tuning is to make it more aggressive on hot tables — lower `autovacuum_vacuum_scale_factor` from the default 0.2 (20% of rows must change) to 0.05 or even 0.01 on heavy-update tables.

Step-by-step thought process: 1. The trigger formula is `threshold + scale_factor * total_rows`. Default threshold is 50 rows, scale factor is 0.2 — so autovacuum kicks in when ~20% of a table is dead. 2. For a billion-row table, 20% is 200M dead tuples before vacuum runs — way too much. Override per-table: `ALTER TABLE t SET (autovacuum_vacuum_scale_factor = 0.02)`. 3. Throttle controls: `autovacuum_vacuum_cost_limit` (default 200) sets how much "work" each autovacuum cycle does before sleeping. Bump it on fast SSDs. 4. Concurrency: `autovacuum_max_workers` (default 3) caps how many tables can be vacuumed simultaneously. 5. Monitor: `pg_stat_user_tables` for `n_dead_tup`, `last_autovacuum`, `n_mod_since_analyze`.

Edge cases to mention: - `vacuum_freeze_min_age` / `vacuum_freeze_table_age` control when transactions are frozen to prevent XID wraparound — must be vacuumed before they wrap. Postgres will eventually force an aggressive vacuum to prevent wraparound, which can cause incidents. - Hot Updates (HOT, when no indexed column changes) bypass index bloat — fillfactor < 100 on tables with many UPDATEs encourages HOT.

What NOT to say: - "Vacuum locks the table." Standard VACUUM does not — only VACUUM FULL takes an ACCESS EXCLUSIVE lock.

Common follow-up: *A table has 100M dead tuples and autovacuum never seems to catch up. What's happening?*

Q40 · VACUUM FULL vs pg_repack — Reclaiming Disk Without Downtime

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, JP Morgan India · Postgres version: any

The question:

> Your orders table is 800 GB on disk but autovacuum reports only 200 GB of live data. VACUUM FULL would reclaim it but locks the table. What do you do?

The 30-second answer: Use pg_repack (a contrib extension or external utility). It rebuilds the table in the background by triggering log writes into a side table, building a new compacted copy, and swapping under a brief ACCESS EXCLUSIVE lock at the end. Reclaims space and reorganises by index without long downtime.

Step-by-step thought process: 1. VACUUM (the regular kind) does not return disk to the OS — it just marks pages reusable inside the table. 2. VACUUM FULL rewrites the table compactly, but takes an ACCESS EXCLUSIVE lock for the full duration — unacceptable for a 24/7 production table. 3. pg_repack creates a log table, sets a trigger to capture all writes during the rebuild, builds a new compacted heap and indexes alongside, replays the log, then briefly swaps the table names. 4. End-of-process lock is

short (seconds, not hours). Some writes during the swap will fail and need retry. 5. Alternative for partitioned tables: drop the old partition entirely instead of compacting it.

Edge cases to mention: - pg_repack needs enough disk to hold a second copy of the table during rebuild. - It does not work on tables without a primary key or unique index (it needs a way to identify rows during the log replay). - For severe bloat, the root cause is often an old open transaction or a leaked replication slot — fix the cause before repacking, or it will refill.

What NOT to say: - "VACUUM FULL is fine if I do it at night." For a payments table at 800 GB, the lock window will be hours.

Common follow-up: *Why might a long-running transaction on a replica block VACUUM on the primary, and what is* `hot_standby_feedback` *?*

Section 2 — Snowflake (40 Questions)

Senior data interview questions. Snowflake is the most-asked cloud DW in Indian product-co data interviews. Expect architecture deep-dives, cost-control questions, and "design an incremental pipeline" prompts.

Q41 · Explain Snowflake's separation of storage and compute

Tags: Difficulty: Easy · Asked at: Swiggy, Razorpay, Flipkart, Meesho · Snowflake area: Architecture

The question:

> Walk me through Snowflake's three-layer architecture and why separating storage from compute is a big deal.

The 30-second answer: Snowflake has three independent layers — cloud storage (S3/Blob/GCS holds compressed columnar micro-partitions), compute (virtual warehouses are MPP clusters spun up on demand), and cloud services (metadata, optimizer, security, transactions). You can scale compute up/down or run multiple warehouses on the same data with zero contention, and you only pay storage once.

Step-by-step thought process: 1. Storage layer — your tables live as immutable micro-partitions in object storage. Snowflake manages compression, encryption, and the format; you never touch the files. 2. Compute layer — a virtual warehouse is a sized cluster (XS to 6XL) that reads from storage, runs queries, and can be suspended when idle. Each warehouse is an isolated workload. 3. Services layer — handles SQL compilation, query plan, metadata lookup (which micro-partitions to scan), security, and transaction coordination. This layer is global per account. 4. Why it matters — in a traditional MPP DW (Teradata, on-prem Exadata), storage and compute are coupled. To add compute you also pay for storage, and concurrent workloads compete for the same nodes. In Snowflake, your BI workload and

your ETL workload can run on separate warehouses, neither slows the other, and they hit the same single copy of data. 5. Result — you can right-size each workload independently, suspend idle compute, and avoid the "noisy neighbour" problem.

Edge cases to mention: - Storage costs are billed separately and are usually a small fraction of compute costs. - The services layer itself is free up to ~10% of daily compute spend; beyond that, cloud services usage is billed.

What NOT to say: - Do not say "Snowflake is just Postgres in the cloud" — architecturally it is closer to a redesigned MPP DW with object storage as the bottom tier. - Do not confuse "warehouse" (Snowflake compute cluster) with the general term "data warehouse" — different things.

Common follow-up: *If two warehouses query the same table at the same time, do they see consistent data?*

Q42 · What is a virtual warehouse and how do you size one?

Tags: Difficulty: Easy · Asked at: Razorpay, Dream11, Slice, InMobi · Snowflake area: Architecture

The question:

> What is a virtual warehouse in Snowflake and how do you decide what size to use?

The 30-second answer: A virtual warehouse is an MPP compute cluster sized from X-Small (1 node) up to 6X-Large (512 nodes), with each step roughly doubling the node count, compute power, and credit-per-hour cost. Sizing is workload-driven — bigger sizes finish a single heavy query faster but waste credits on small queries.

Step-by-step thought process: 1. Sizes follow t-shirt naming — XS, S, M, L, XL, 2XL, 3XL, 4XL, 5XL, 6XL. Each step up roughly doubles cost per hour but also roughly halves runtime for a parallelisable query. 2. Pick size by query characteristics, not by table size — a small aggregation on a huge clustered table may run fine on XS, while a complex 10-way join on medium tables may need L. 3. Default starting point — most ETL workloads start at M or L; interactive BI starts at S or M; data science exploratory at XS or S. 4. The right way to size — pick a starting size, run your representative workload, look at query profile for "remote disk spillage" (means warehouse is too small) and queue time (means too few clusters). Adjust. 5. Crucial cost rule — billing is per-second after a 60-second minimum. A 4XL running for 30 seconds costs the same as one running for 60 seconds, so for short bursty queries a small warehouse is often cheaper overall.

Edge cases to mention: - Doubling size does not always halve runtime — only if the query is parallelisable. A single small query that hits one micro-partition will not run faster on a 6XL. - Spillage to remote disk (visible in query profile) is the strongest signal you need a bigger warehouse.

What NOT to say: - Do not say "always use the biggest warehouse for ETL" — that is a fast way to burn credits. - Do not confuse warehouse size with concurrency — for concurrency, you scale out via multi-cluster, not up.

Common follow-up: *If your warehouse is showing high queue time, do you scale up or scale out?*

Q43 · Multi-cluster warehouses — what problem do they solve?

Tags: Difficulty: Medium · Asked at: Flipkart, Meesho, Jupiter, Razorpay · Snowflake area: Architecture

The question:

> Explain multi-cluster warehouses. When would you use one over a single-cluster warehouse?

The 30-second answer: A multi-cluster warehouse lets Snowflake spin up additional clusters of the same size when query concurrency spikes, then shut them down when the load drops. You use it for BI/ad-hoc workloads with many concurrent users; you do not need it for serial ETL jobs.

Step-by-step thought process: 1. Single-cluster warehouse — one cluster, fixed size. If 50 users hit it at once, queries queue up. 2. Multi-cluster — you set a min and max cluster count. Snowflake starts more clusters when queue time grows, retires them when traffic drops. 3. Scaling policies — Standard (aggressive, adds clusters quickly to clear the queue) and Economy (conservative, lets queries queue longer to keep clusters fully utilised, optimising cost). 4. Use case — Tableau/Looker dashboards hit by 100s of analysts in business hours. A multi-cluster M warehouse with min=1, max=5, Standard policy handles concurrency spikes cleanly. 5. Not for ETL — your nightly load job runs serially on one warehouse; adding clusters does not help one query. Use multi-cluster only when concurrent independent queries are the bottleneck.

Edge cases to mention: - Min clusters > 1 is "Maximised" mode — always runs that many clusters, useful when you want guaranteed concurrency without ramp-up delay. - Each cluster bills independently per second; multi-cluster with max=5 can quietly cost 5x.

What NOT to say: - Do not confuse multi-cluster (more clusters of same size) with sizing up (single bigger cluster). They solve different problems. - Do not enable multi-cluster as a default — it is a concurrency tool, not a performance tool.

Common follow-up: *Which scaling policy is better for an analyst-heavy workload — Standard or Economy?*

Q44 · Auto-suspend and auto-resume — best practice settings?

Tags: Difficulty: Easy · Asked at: Swiggy, Razorpay, Slice, InMobi · Snowflake area: Cost

The question:

> What do auto-suspend and auto-resume do, and what values would you set for an ETL warehouse vs a BI warehouse?

The 30-second answer: Auto-suspend stops a warehouse after N seconds of idle, auto-resume restarts it when a query arrives. For ETL warehouses set auto-suspend to 60 seconds (lowest possible — kill it the moment the job ends). For BI warehouses set 300-600 seconds to preserve warm cache for analyst sessions.

Step-by-step thought process: 1. The cost mechanism — Snowflake bills per second after a 60-second minimum. A warehouse left running idle is burning credits. 2. Auto-suspend value choice depends on cache value — if your workload benefits from warehouse-local result cache (BI dashboards re-hitting same data), keeping it warm 5-10 minutes saves more than it costs. 3. ETL warehouses — workloads are scheduled, gaps between jobs are long, no cache reuse. Set auto-suspend = 60s. 4. BI warehouses — analysts come and go, sessions cluster. Set auto-suspend = 300-600s so the next query within 10 minutes hits a warm warehouse. 5. Auto-resume should almost always be ON — otherwise users get errors when warehouse is suspended. The only time you turn it off is for tightly-controlled cost environments where you manually start warehouses.

Edge cases to mention: - Resume takes 1-3 seconds for small sizes, longer for large. Plan first-query latency accordingly. - The 60-second minimum bill means very short auto-suspend values do not help if queries arrive every 30s.

What NOT to say: - Do not say "set auto-suspend to 0" — minimum is typically 60 seconds. - Do not leave warehouses with no auto-suspend — this is the #1 cause of unexpected Snowflake bills.

Common follow-up: *What is lost when a warehouse suspends?*

```
ALTER WAREHOUSE etl_wh SET AUTO_SUSPEND = 60 AUTO_RESUME = TRUE;
ALTER WAREHOUSE bi_wh  SET AUTO_SUSPEND = 600 AUTO_RESUME = TRUE;
```

Q45 · What are micro-partitions?

Tags: Difficulty: Medium · Asked at: Razorpay, Dream11, Flipkart, Meesho · Snowflake area: Architecture

The question:

> Explain what a Snowflake micro-partition is, including its size and physical layout.

The 30-second answer: A micro-partition is an immutable, columnar storage unit of roughly 50-500 MB uncompressed (smaller compressed). Snowflake automatically partitions every table into micro-partitions as data is ingested, and stores column min/max/null-count metadata for each — which powers automatic pruning at query time.

Step-by-step thought process: 1. Physical layout — each micro-partition stores a horizontal slice of rows but in columnar format (each column stored contiguously, compressed independently). 2. Automatic — you do not choose partition boundaries. As you insert data, Snowflake batches rows into micro-partitions, generally in insertion order. 3. Immutability — micro-partitions are never modified. An UPDATE or DELETE writes new micro-partitions and marks old ones for retention/expiry. This enables time travel and zero-copy cloning. 4. Metadata — for every micro-partition Snowflake stores per-column min, max, null count, distinct count estimate. This metadata lives in the services layer and is what the optimiser uses to prune. 5. Pruning — when you filter WHERE order_date = '2026-05-01', Snowflake checks each micro-partition's min/max for order_date and skips any that cannot match. On large tables this can cut scan to <1% of the data.

Edge cases to mention: - Compressed micro-partition size on disk is typically 16-50 MB; the 50-500 MB number is uncompressed. - Pruning effectiveness depends on natural clustering — data inserted in date order will prune date filters very well; randomly inserted data will not.

What NOT to say: - Do not say "I create partitions in Snowflake" — you do not. Partitioning is automatic and transparent. - Do not equate micro-partitions with Hive/Spark partitions — they are conceptually different (micro-partitions are much smaller and auto-managed).

Common follow-up: *How is this different from how Postgres or MySQL stores data?*

Q46 · How does Snowflake prune micro-partitions?

Tags: Difficulty: Medium · Asked at: Flipkart, Razorpay, InMobi, Swiggy · Snowflake area: Performance

The question:

> Walk through exactly how Snowflake decides which micro-partitions to read for a query.

The 30-second answer: At compile time the optimiser pushes WHERE-clause predicates against the per-micro-partition min/max metadata stored in the services layer, eliminates micro-partitions that cannot contain matching rows, and scans only the survivors. You see this in query profile as "partitions scanned vs partitions total".

Step-by-step thought process: 1. The metadata table — services layer holds, for every micro-partition of every table, min/max/null-count for each column. 2. Predicate evaluation — for WHERE order_date BETWEEN '2026-05-01' AND '2026-05-07', the optimiser checks each micro-partition's order_date min and max. If max < '2026-05-01' or min > '2026-05-07', skip. 3. Composite predicates — multiple WHERE

clauses combine; a micro-partition must pass all of them to be scanned. 4. Limits — pruning works on min/max comparisons, so equality, range, and IN-list predicates prune well. LIKE '%pattern%' and function-wrapped columns (e.g., DATE(created_ts)) generally cannot use micro-partition pruning. 5. Verify in query profile — the "TableScan" node shows "Partitions scanned: X / Total: Y". If X is close to Y, you are not pruning; consider clustering, predicate rewrite, or a search optimisation service.

Edge cases to mention: - Joins also benefit indirectly via dynamic pruning — Snowflake can use the build side of a hash join to prune the probe side at runtime. - High-cardinality string columns often have wide min/max ranges per partition, reducing pruning effectiveness.

What NOT to say: - Do not say "Snowflake reads everything and filters in memory" — it does aggressive pruning at compile time. - Do not confuse pruning with caching; pruning is about which files to read, caching is about reusing previously read data.

Common follow-up: *What happens if you wrap your filter column in a function?*

Q47 · Why does Snowflake not need traditional indexes?

Tags: Difficulty: Medium · Asked at: Razorpay, Meesho, Dream11, Slice · Snowflake area: Architecture

The question:

> Why doesn't Snowflake have traditional B-tree indexes like Postgres or MySQL?

The 30-second answer: Snowflake is an analytical columnar store, not an OLTP row store. Its access pattern is "scan many rows of few columns" not "find one row by key". Micro-partition pruning + columnar compression replaces the role of indexes for analytical workloads; for point lookups Snowflake offers Search Optimisation Service as an opt-in.

Step-by-step thought process: 1. Workload mismatch — B-tree indexes shine when you need to find 1 row out of millions by key. Analytical queries scan billions of rows and aggregate. 2. Cost of indexes — in OLTP, indexes slow writes (every insert updates every index). Snowflake's immutable micro-partition model would have to rewrite index structures constantly. 3. Replacement mechanisms — micro-partition pruning (skip whole files), columnar compression (read only needed columns), clustering keys (encourage co-located data for filter columns). 4. Point-lookup case — if you need true OLTP-style "WHERE id = ?" on a huge table, enable Search Optimisation Service, which builds equality/IN-list indexes per column at extra cost. 5. Mental model — Snowflake is built around the assumption that pruning + parallel scan is faster than maintaining auxiliary structures, for the workloads it targets.

Edge cases to mention: - Primary key and unique constraints can be declared but are not enforced — they are metadata hints for query optimiser only. - Hybrid Tables (Unistore) introduce row-store indexes for OLTP-style access; this is a newer separate table type, not standard Snowflake tables.

What NOT to say: - Do not say "Snowflake has hidden indexes" — micro-partition metadata is not an index, it is per-file statistics. - Do not assume a query that would use an index in Postgres will be slow in Snowflake — pruning usually handles it.

Common follow-up: *When would you turn on Search Optimisation Service?*

Q48 · Why are micro-partitions immutable, and what does this enable?

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Swiggy, InMobi · Snowflake area: Architecture

The question:

> Why does Snowflake make micro-partitions immutable, and what features does this enable?

The 30-second answer: Immutability lets Snowflake retain old versions of data files cheaply (since nothing rewrites in place), which enables time travel, fail-safe, zero-copy cloning, and consistent snapshot isolation without locking. The cost is that UPDATE/DELETE writes new partitions instead of modifying old ones.

Step-by-step thought process: 1. The mechanism — when you UPDATE a row, Snowflake writes a new micro-partition containing the new row version and marks the old partition as superseded but retained. 2. Time travel — because old micro-partitions are kept for the retention window (1-90 days), you can query "as of timestamp X" by routing the scan to the version of partitions active at that time. 3. Zero-copy cloning — a CLONE creates a new table pointing to the same micro-partitions. Only when one side writes does Snowflake create new partitions for that side. No data copied at clone time. 4. Snapshot isolation — every query gets a consistent view at its start time. Long-running queries are not affected by concurrent writes because the writes go to new partitions. 5. The tradeoff — heavy update/delete workloads accumulate "dead" partition versions and inflate storage. Vacuuming is automatic but lags. Heavy DML also burns compute since each UPDATE rewrites whole partitions.

Edge cases to mention: - Time travel storage is charged only for the bytes of replaced/deleted micro-partitions, not the live data. - Fail-safe is a non-configurable 7-day Snowflake-internal retention after time travel ends — for permanent tables only.

What NOT to say: - Do not say "UPDATE is free" — it has both compute cost (rewriting partitions) and storage cost (retaining old versions). - Do not promise zero-copy clone is literally zero cost — clone is free at creation but diverges as either side writes.

Common follow-up: *If a table has heavy UPDATE workloads, how does this affect cost?*

Q49 · What is a clustering key and when do you need one?

Tags: Difficulty: Medium · Asked at: Flipkart, Razorpay, Meesho, Dream11 · Snowflake area: Performance

The question:

> Explain clustering keys in Snowflake. When do you actually need to define one?

The 30-second answer: A clustering key tells Snowflake which columns to co-locate when reorganising micro-partitions, improving pruning for queries that filter on those columns. You need one only for very large tables (typically > 1 TB) where natural insertion order does not align with frequent filter patterns.

Step-by-step thought process: 1. What it does — Snowflake's automatic clustering service periodically reorganises micro-partitions so rows with similar clustering-key values sit in the same partition. Better clustering → tighter min/max → more pruning. 2. When natural clustering is enough — if you always insert by date and filter by date, your data is already date-clustered; no key needed. 3. When you need it — large tables where common filter columns differ from insert order. Example: 5 TB events table inserted in event_time order, but most queries filter by user_id. 4. How to choose — pick the most-frequently-filtered low-to-medium cardinality column(s). Avoid high-cardinality keys (every value unique → no co-location benefit) and avoid changing-frequently keys (re-clustering churn). 5. Verify benefit — use SYSTEM$CLUSTERING_INFORMATION to see clustering depth before/after, and compare partition-scan ratios in query profile.

Edge cases to mention: - Clustering keys are not literal partition boundaries — they are hints for the auto-clustering service. - Up to ~4 columns or expressions in a clustering key; more than that rarely helps.

What NOT to say: - Do not cluster every table by default — small tables and well-naturally-ordered tables waste money. - Do not equate clustering keys with primary keys — they have nothing to do with uniqueness.

Common follow-up: *If you cluster on user_id, what happens when you bulk-load a million rows for one user?*

```
ALTER TABLE events CLUSTER BY (event_date, user_id);
SELECT SYSTEM$CLUSTERING_INFORMATION('events');
```

Q50 · Automatic clustering vs explicit clustering — costs

Tags: Difficulty: Hard · Asked at: Razorpay, Meesho, Swiggy, Jupiter · Snowflake area: Cost

The question:

> What does automatic clustering cost, and how does Snowflake decide when to recluster?

The 30-second answer: Automatic clustering is a background service that re-organises a clustered table's micro-partitions; it consumes serverless credits billed separately from your warehouses. Snowflake reclusters when clustering depth degrades beyond a threshold, typically after large inserts/updates that disrupt the key order.

Step-by-step thought process: 1. Triggering — Snowflake monitors clustering depth (a measure of overlap between micro-partition value ranges). When depth exceeds an internal threshold, re-clustering kicks off. 2. The cost — serverless credits, not your warehouse credits. You see this in ACCOUNT_USAGE.AUTOMATIC_CLUSTERING_HISTORY. 3. Worst-case scenarios — frequent random-key updates on a clustered table, or constantly inserting out-of-order data, can cause continuous re-clustering and big bills. 4. Choosing what to cluster — better to cluster columns that are insert-stable (date) than ones updated post-insert (status). High DML churn on a clustering column is expensive. 5. Suspending — you can SUSPEND RECLUSTER on a table to pause the service, useful when you know you are doing a one-off massive load and will manually resume after.

Edge cases to mention: - Re-clustering can lag behind ingestion; query performance may stay good due to pruning even while re-cluster runs in the background. - For very volatile tables, sometimes the right answer is no clustering key + a downstream pre-clustered materialised view.

What NOT to say: - Do not say "automatic clustering is always on by default" — it is on only for tables with a defined CLUSTER BY. - Do not assume re-clustering is instant — it can take hours on huge tables.

Common follow-up: *How do you investigate a spike in automatic clustering credits?*

```
SELECT TABLE_NAME, SUM(CREDITS_USED)
FROM SNOWFLAKE.ACCOUNT_USAGE.AUTOMATIC_CLUSTERING_HISTORY
WHERE START_TIME >= DATEADD(DAY, -7, CURRENT_TIMESTAMP())
GROUP BY 1 ORDER BY 2 DESC;
```

Q51 · Clustering depth — what does it actually measure?

Tags: Difficulty: Hard · Asked at: Razorpay, Flipkart, InMobi, Dream11 · Snowflake area: Performance

The question:

> What is clustering depth and how do you interpret it?

The 30-second answer: Clustering depth is the average number of micro-partitions that overlap in value range at any point along the clustering key. Lower is better — depth of 1 means each value lives in exactly one partition (perfect), high depth means values are scattered across many partitions (poor pruning).

Step-by-step thought process: 1. The intuition — for a clustered column, imagine sorting all micro-partitions by their min value. At any given key value, count how many partitions have a min/max range that covers it. That count is the depth at that point. 2. Average depth — Snowflake reports the average across the key range. A depth of ~1 is ideal; depth growing with table size is a sign clustering is degrading. 3. Reading the output — SYSTEM$CLUSTERING_INFORMATION returns partition_count, average_overlaps, average_depth, partition_depth_histogram. The histogram shows how many partitions fall at each depth level. 4. What good looks like — for a date-clustered table, average_depth should hover near 1-3 even after years of inserts. 5. What bad looks like — average_depth in double digits or rapidly climbing means many partitions cover overlapping ranges; pruning is degrading; consider re-clustering or different key.

Edge cases to mention: - After a bulk load that violates key order (e.g., backfilling historical data into a date-clustered table), depth will jump until automatic clustering catches up. - Sometimes a small table just has high depth and there is nothing meaningful to do — clustering benefits are workload-dependent.

What NOT to say: - Do not confuse clustering depth with clustering width or partition count — they measure different things. - Do not panic at depth = 5 — context matters; check whether your queries actually need finer pruning.

Common follow-up: *If average depth is 1 but a column has 10 distinct values and 1000 partitions, what is the implication?*

Q52 · When should you NOT use a clustering key?

Tags: Difficulty: Medium · Asked at: Meesho, Swiggy, Razorpay, Slice · Snowflake area: Cost

The question:

> Give examples of cases where defining a clustering key would be a bad idea.

The 30-second answer: Do not cluster small tables, frequently-updated columns, high-cardinality (near-unique) columns, or tables where natural insert order already aligns with query filters. Each of these either wastes re-clustering credits or provides no pruning benefit.

Step-by-step thought process: 1. Small tables — under a few hundred GB, scan is fast enough; the auto-clustering credits will dwarf any savings. 2. Already-ordered data — if you ingest by event_date in batches and filter by event_date, micro-partitions are naturally date-ordered; explicit clustering is redundant. 3. Frequently-updated key — if you cluster by status (open/closed) and orders flip status

often, every status change can trigger re-clustering churn. 4. Very high cardinality — clustering by user_id where user_id has 100M distinct values means each micro-partition still covers many users; pruning gain is marginal. 5. Multi-column wide keys — defining 6+ columns rarely helps; the auto-clustering service has to balance all of them and pruning benefit per column drops.

Edge cases to mention: - Composite keys with cardinality stacking (e.g., country, then date) can work well when queries filter on the leading column. - For tables you ingest once and never update, set the key once at create time and let the initial sort do the work.

What NOT to say: - Do not say "cluster everything to be safe" — this is the most common cost mistake in Snowflake. - Do not blindly cluster on the primary key — primary key is usually high-cardinality and a bad clustering choice.

Common follow-up: *How do you decide whether a clustering key is paying for itself?*

Q53 · Time travel — what is it and what are the retention limits?

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Flipkart, Meesho · Snowflake area: Architecture

The question:

> What is time travel in Snowflake and what are the retention periods?

The 30-second answer: Time travel lets you query a table as it existed at any point within its retention window, undrop a recently-dropped object, and clone from a historical state. Standard edition gets up to 1 day of retention; Enterprise edition gets up to 90 days. Default retention is 1 day.

Step-by-step thought process: 1. The use cases — recover from accidental DELETE/UPDATE, audit historical state, build clones from a known good point in time. 2. Setting retention — DATA_RETENTION_TIME_IN_DAYS at account, database, schema, or table level. Most teams set 7-30 days on critical tables. 3. The syntax — SELECT * FROM orders AT(TIMESTAMP => '2026-05-25 10:00:00'), or BEFORE(STATEMENT => 'query_id'), or AT(OFFSET => -3600) for relative time. 4. UNDROP — recovers a dropped table/schema/database within retention. UNDROP TABLE orders. 5. Cost — retained micro-partitions cost storage. A table with heavy DML and 90-day retention can accumulate significant time-travel storage.

Edge cases to mention: - Transient and temporary tables have no fail-safe and at most 1-day time travel. - After retention ends, permanent tables enter fail-safe (Snowflake-internal 7 days, not user-queryable, recovery requires support ticket).

What NOT to say: - Do not say "time travel is always 90 days" — it depends on edition and explicit setting. - Do not rely on fail-safe as a recovery mechanism — it requires a support escalation, not a self-serve query.

Common follow-up: *Can you time travel across a TRUNCATE TABLE?*

```
SELECT * FROM orders AT(OFFSET => -60 * 60);
SELECT * FROM orders BEFORE(STATEMENT => '019abc...');
UNDROP TABLE orders;
```

Q54 · Querying historical data — AT vs BEFORE

Tags: Difficulty: Medium · Asked at: Swiggy, Razorpay, Slice, InMobi · Snowflake area: Architecture

The question:

> What is the difference between AT and BEFORE when querying time travel?

The 30-second answer: AT returns the table state at and including the given timestamp/statement, BEFORE returns the state immediately before. BEFORE(STATEMENT => 'qid') is the common pattern for "what did the table look like just before this bad UPDATE ran".

Step-by-step thought process: 1. AT semantics — gives the snapshot as of the moment specified, with any commits at that exact moment visible. 2. BEFORE semantics — gives the snapshot immediately prior, excluding the specified event. 3. Practical use — when an analyst ran a bad UPDATE and you have the query_id, use BEFORE(STATEMENT => 'query_id') to see pre-update state. Using AT would include the update. 4. Both accept TIMESTAMP, OFFSET (seconds back from now), or STATEMENT (query_id). Choose based on what reference point is easiest to express. 5. Recovery pattern — CREATE TABLE orders_recovered CLONE orders BEFORE(STATEMENT => 'bad_qid'), then swap or merge the recovered state in.

Edge cases to mention: - The query_id used in BEFORE/AT must be within the table's retention window; older query IDs do not work even if you have them logged. - Time travel respects transactions — if a multi-statement transaction modified the table, you see the table either before or after the commit, not mid-transaction.

What NOT to say: - Do not confuse AT(OFFSET => -60) with BEFORE — OFFSET is just a way of specifying time, AT/BEFORE control inclusivity. - Do not assume time travel works across all DDL — some DDL truncates the time travel chain for that object.

Common follow-up: *How would you actually recover from an accidental DELETE on a 2 TB table?*

```
CREATE TABLE orders_restored CLONE orders BEFORE(STATEMENT => '019abc-def-...');
INSERT INTO orders SELECT * FROM orders_restored WHERE id NOT IN (SELECT id FROM orders);
```

Q55 · Time travel cost — what are you actually paying for?

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Meesho, Dream11 · Snowflake area: Cost

The question:

> If I set DATA_RETENTION_TIME_IN_DAYS = 30 on a 1 TB table with daily updates, what is the cost impact?

The 30-second answer: Storage cost grows by the volume of micro-partitions superseded by updates over the 30-day window. If 5% of partitions are rewritten daily, after 30 days you are retaining ~150% of base table size in time-travel storage. There is no compute cost for retention itself.

Step-by-step thought process: 1. The mechanism — every UPDATE/DELETE writes new partitions and marks old ones for retention. Retained partitions sit in object storage at the regular per-TB rate. 2. The math — base 1 TB + daily churn of (say) 5% × 30 days = 1.5 TB of retained data, billed at storage rates. 3. Pure-INSERT tables — if you only INSERT (no UPDATE/DELETE), nothing accumulates in time-travel storage because no partitions are superseded. 4. Heavy DML tables — a high-update OLTP-style table with long retention can have time-travel storage many times larger than the live data. 5. Mitigation — set retention to the minimum your recovery needs require. Critical fact tables: 7 days. Staging tables: 1 day. Reference data: 30+ days OK because low churn.

Edge cases to mention: - ACCOUNT_USAGE.TABLE_STORAGE_METRICS shows ACTIVE_BYTES vs TIME_TRAVEL_BYTES vs FAILSAFE_BYTES — the place to look for cost diagnostics. - TRUNCATE TABLE retains the data for the time-travel window; if you truncate to reduce storage, you also need to wait or set retention to 0.

What NOT to say: - Do not say "time travel is free" — storage is real money, especially on high-DML tables. - Do not set 90-day retention as a default — pick it per table based on recovery requirements.

Common follow-up: *How would you audit which tables are driving time-travel storage cost?*

Q56 · Zero-copy cloning — how does it actually work?

Tags: Difficulty: Medium · Asked at: Swiggy, Razorpay, Meesho, Jupiter · Snowflake area: Architecture

The question:

> Explain zero-copy cloning. How can it be instant on a multi-TB table?

The 30-second answer: A CLONE creates a new database/schema/table that references the same underlying micro-partitions as the source, recorded as metadata only. No data is copied. As either source

or clone writes, new partitions are created and the metadata diverges, so storage grows only with divergence.

Step-by-step thought process: 1. Clone operation — CREATE TABLE dev_orders CLONE prod_orders. Metadata-only operation, finishes in seconds even on TB-scale tables. 2. Storage model — the clone's catalog entry points to the same micro-partition file list. Both objects share storage until either is modified. 3. Divergence on write — if you UPDATE 10 rows in the clone, Snowflake writes new micro-partitions for those rows. The original table is untouched. Clone now stores (shared partitions + new partitions for changed rows). 4. Cloning with time travel — CREATE TABLE recovered CLONE orders AT(TIMESTAMP => ...) lets you clone from a historical state for forensic or recovery work. 5. Common patterns — full prod-to-dev database clone for testing schema changes, point-in-time clones for audit/forensic, monthly snapshot clones for compliance.

Edge cases to mention: - You can clone a whole database, including all schemas and tables, in one DDL statement. - Pipes, streams, and tasks are not always cloned; check the docs for which child objects propagate.

What NOT to say: - Do not say "clone = backup" — clone shares storage; deleting source partitions affects the clone only after time travel. - Do not assume cloning is literally free forever — divergence costs storage and DML on the clone costs compute.

Common follow-up: *If I clone prod to dev and then DROP the prod table, what happens to dev?*

```
CREATE OR REPLACE DATABASE dev_db CLONE prod_db;
CREATE TABLE orders_audit_20260501 CLONE orders AT(TIMESTAMP => '2026-05-01 00:00:00');
```

Q57 · Dev/staging from prod — clone vs replicate vs share

Tags: Difficulty: Medium · Asked at: Flipkart, Razorpay, Slice, InMobi · Snowflake area: Architecture

The question:

> You want analysts to test on prod-like data in a dev environment. Clone, replicate, or share — which would you pick?

The 30-second answer: For dev/staging in the same account, zero-copy clone is almost always the right answer — instant, no extra storage initially, fully writable. Replication is for cross-region/account disaster recovery. Secure data sharing is for read-only consumption by an external account.

Step-by-step thought process: 1. Clone — same account, same region, instant, writable, diverges over time. Use for dev/test/QA environments where teams need to run DML. 2. Replication — copies data to another account or region. Used for DR, regional latency, or cross-account analytics. Has ongoing cost

and latency. 3. Secure data sharing — consumer accounts see your data live (read-only) without copying. Use for B2B data sharing, marketplace, internal multi-account setups where consumers should not write. 4. Refresh cadence — clone can be refreshed periodically (drop + re-clone) if dev data needs to stay fresh. Replication can be scheduled. Sharing is always live. 5. Cost shape — clone: storage only on divergence. Replication: full storage + transfer. Sharing: provider pays storage, consumer pays compute on their warehouse.

Edge cases to mention: - For PII-sensitive dev environments, clone is still risky because data is real; consider masking policies before sharing dev access. - Cross-region replication has minutes-to-hours lag depending on volume.

What NOT to say: - Do not suggest CTAS (CREATE TABLE AS SELECT) to copy prod to dev — that is full copy, slow and expensive. - Do not assume sharing lets consumers write back — it does not, it is read-only.

Common follow-up: *How would you mask PII before letting devs query a cloned table?*

Q58 · What is a Snowflake Stream?

Tags: Difficulty: Medium · Asked at: Swiggy, Razorpay, Dream11, Flipkart · Snowflake area: Architecture

The question:

> Explain Snowflake streams. What do they capture and what is the typical use case?

The 30-second answer: A stream is a change-data capture (CDC) object that records insert/update/delete activity on a source table since the last time the stream was consumed. You use streams to drive incremental processing — read the stream, transform, write to a target, and the stream advances.

Step-by-step thought process: 1. The mechanism — stream tracks an offset (a "stream position") on the source table. It exposes a virtual table showing changed rows with METADATA$ACTION ('INSERT' or 'DELETE') and METADATA$ISUPDATE flags. 2. Stream types — Standard (tracks net changes), Append-only (only inserts, cheaper), and Insert-only (for external tables). 3. Consumption pattern — INSERT INTO target SELECT ... FROM my_stream — this DML action advances the stream offset, so the next read sees only newer changes. 4. Why use it — instead of running full-table comparisons or watermarking by timestamp, the stream tells you exactly which rows changed since you last looked. Especially useful for SCD2 builds and incremental aggregates. 5. Combined with Tasks — streams + tasks is the canonical pattern for incremental Snowflake-native pipelines (vs running dbt jobs externally).

Edge cases to mention: - A stream becomes stale if not consumed within the data retention period of the source table; pick stream extension carefully. - Streams on views are supported with limitations; streams on external tables follow different rules.

What NOT to say: - Do not say "streams are like Kafka inside Snowflake" — they are change logs on tables, not message queues. - Do not assume reading from a stream advances it — only a DML transaction that consumes the stream advances its offset.

Common follow-up: *What happens if my task that consumes the stream fails mid-run?*

```
CREATE STREAM orders_stream ON TABLE orders;
SELECT * FROM orders_stream;  -- shows changes since last consumed
```

Q59 · Snowflake Tasks — what are they and how do they schedule?

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Meesho, InMobi · Snowflake area: Architecture

The question:

> What is a Snowflake task and what scheduling options does it support?

The 30-second answer: A task is a scheduled SQL job that runs a statement (or calls a stored procedure) on a defined cadence — either a cron expression or "every N minutes" interval. Tasks can chain into DAGs via AFTER clauses, and can be gated on streams having data via WHEN SYSTEM$STREAM_HAS_DATA.

Step-by-step thought process: 1. Basic syntax — CREATE TASK my_task WAREHOUSE = wh SCHEDULE = '5 MINUTE' AS INSERT INTO ... . 2. Cron syntax — SCHEDULE = 'USING CRON 0 6 * * * UTC' for daily 6 AM UTC runs. 3. Chaining — child tasks declare AFTER parent_task to form DAGs. Common in multi-step pipelines (stage → transform → load → aggregate). 4. Conditional execution — WHEN SYSTEM$STREAM_HAS_DATA('my_stream') makes a task no-op when the upstream stream is empty, avoiding wasted compute. 5. Serverless tasks — instead of attaching a warehouse, set USER_TASK_MANAGED_INITIAL_WAREHOUSE_SIZE; Snowflake provisions an ephemeral warehouse per execution. Useful for unpredictable workloads.

Edge cases to mention: - Tasks have to be explicitly resumed (ALTER TASK ... RESUME) after creation; they start suspended. - Maximum task tree depth and number of child tasks have limits — for very large DAGs, prefer an external orchestrator.

What NOT to say: - Do not present tasks as a replacement for Airflow/dbt-Cloud for complex pipelines — tasks are great for in-Snowflake transformation but lack cross-system observability. - Do not forget to grant USAGE on the warehouse to the task owner role.

Common follow-up: *Streams + tasks vs running dbt on a schedule — when do you pick each?*

```
CREATE TASK refresh_aggregates
  WAREHOUSE = etl_wh
  SCHEDULE = '15 MINUTE'
  WHEN SYSTEM$STREAM_HAS_DATA('orders_stream')
AS
  INSERT INTO daily_revenue SELECT ... FROM orders_stream;

ALTER TASK refresh_aggregates RESUME;
```

Q60 · Building an incremental pipeline with streams + tasks

Tags: Difficulty: Hard · Asked at: Swiggy, Flipkart, Razorpay, Dream11 · Snowflake area: Architecture

The question:

> Design an incremental pipeline that takes raw orders, derives a daily revenue table, and updates near real-time.

The 30-second answer: Create a stream on raw_orders, a task on a 5-minute schedule gated by SYSTEM$STREAM_HAS_DATA that MERGEs new/changed orders into daily_revenue, ensure the MERGE handles both inserts and updates idempotently. Use a transactional consume + apply so failed runs do not skip data.

Step-by-step thought process: 1. Source — raw_orders is your ingestion table (loaded by Snowpipe or batch COPY). 2. Capture changes — CREATE STREAM raw_orders_stream ON TABLE raw_orders. This records inserts/updates/deletes. 3. Build the task — CREATE TASK upd_daily_revenue WAREHOUSE = etl_wh SCHEDULE = '5 MINUTE' WHEN SYSTEM$STREAM_HAS_DATA('raw_orders_stream') AS MERGE INTO daily_revenue USING (SELECT date, SUM(amount) ... FROM raw_orders_stream GROUP BY date) src ON tgt.date = src.date WHEN MATCHED THEN UPDATE ... WHEN NOT MATCHED THEN INSERT .... 4. Idempotency — MERGE pattern means re-running the task with the same data produces the same result. If a task fails before commit, the stream offset is not advanced; the next run reprocesses the same delta. 5. Observability — log run metadata (rows processed, task_run_id) to a metadata table. Monitor TASK_HISTORY view for failures.

Edge cases to mention: - Append-only stream is cheaper if raw_orders never gets updates; standard stream is needed if rows are updated/deleted in source. - Long-running pipelines should split into smaller tasks chained via AFTER; one giant task that does everything is hard to retry partially.

What NOT to say: - Do not use INSERT-only without MERGE if source updates exist — you will double-count. - Do not skip WHEN SYSTEM$STREAM_HAS_DATA — empty runs still consume warehouse minimum bills.

Common follow-up: *How do you handle late-arriving data in this pipeline?*

Q61 · Streams + Tasks vs scheduled dbt runs

Tags: Difficulty: Hard · Asked at: Meesho, Razorpay, Slice, Jupiter · Snowflake area: Architecture

The question:

> When would you use Snowflake streams + tasks vs scheduled dbt runs for the same incremental transformation?

The 30-second answer: Use streams + tasks when transformations are simple, latency-sensitive (sub-15 min), and live entirely inside Snowflake. Use dbt scheduled runs when you have complex DAGs, testing/documentation needs, cross-source data lineage, and version-controlled SQL as your team's source of truth.

Step-by-step thought process: 1. Streams + tasks strengths — low latency (1-5 min refresh), no external orchestrator, change-aware (only process deltas), no extra cost on top of warehouse. 2. Streams + tasks weaknesses — no native testing, no documentation framework, deploying changes requires SQL DDL discipline, limited cross-task observability, no version control out of the box. 3. dbt strengths — full DAG modelling with refs, tests (unique, not_null, custom), documentation, version control, integration with CI/CD, cross-source models (joins to non-Snowflake data via federated tables). 4. dbt weaknesses — typically batch-scheduled (every 15-60 min minimum), each run scans incremental boundaries (timestamp-based), no native CDC awareness. 5. Hybrid — many teams use streams to capture changes, expose a "deltas" view, then let dbt incremental models consume that view. Best of both.

Edge cases to mention: - For sub-1-minute latency, neither pure tasks nor dbt fit — use Dynamic Tables or Snowpipe Streaming with a target lag. - dbt's incremental models can use Snowflake streams via dbt-snowflake adapter features.

What NOT to say: - Do not present this as "one is always better" — they coexist in most production data stacks. - Do not skip testing if you go pure streams + tasks; build your own assertion framework.

Common follow-up: *How would you add tests to a streams + tasks pipeline?*

Q62 · Stream staleness and stream extension

Tags: Difficulty: Hard · Asked at: Razorpay, Flipkart, InMobi, Dream11 · Snowflake area: Architecture

The question:

> What does it mean for a stream to become stale, and how do you prevent it?

The 30-second answer: A stream becomes stale when its position falls outside the source table's data retention window — the stream no longer has access to the changes it tracked, and the next consume errors out. Prevent it by raising source table DATA_RETENTION_TIME_IN_DAYS or by consuming the stream more frequently.

Step-by-step thought process: 1. The mechanism — a stream is essentially a pointer to a position in the source's time-travel history. If retention expires past that pointer, the stream cannot read what it pointed to. 2. Detecting staleness — SYSTEM$STREAM_GET_TABLE_TIMESTAMP returns the timestamp the stream is reading from. Compare to the source's retention to see how close to expiry. 3. Stream extension — some streams support extended retention via a parameter on the source table (MAX_DATA_EXTENSION_TIME_IN_DAYS). This extends retention if a stream still needs it. 4. Mitigation order — first, make sure your task is actually running. Second, raise retention on the source table to comfortably exceed your maximum task downtime. Third, set MAX_DATA_EXTENSION_TIME_IN_DAYS as a safety net. 5. Recovery — if a stream is already stale, you must DROP and CREATE the stream and re-bootstrap (full reload). No way to recover the missed changes.

Edge cases to mention: - Append-only streams on append-only tables have different staleness characteristics; check the docs for your stream type. - A stalled task that does not consume the stream can cause silent staleness; alert on TASK_HISTORY status != SUCCEEDED.

What NOT to say: - Do not assume stream extension is unlimited — it adds storage cost and has practical ceilings. - Do not silently re-create stale streams in production — you will skip data without realising.

Common follow-up: *How would you monitor stream lag in production?*

Q63 · Dynamic tables — what are they and how do they differ from materialised views?

Tags: Difficulty: Hard · Asked at: Swiggy, Razorpay, Meesho, Flipkart · Snowflake area: Architecture

The question:

> Explain dynamic tables. How are they different from materialised views?

The 30-second answer: A dynamic table is a declaratively-defined table whose contents are derived from a query, automatically refreshed by Snowflake to meet a target lag (e.g., "stay within 5 minutes of source"). Materialised views are limited to a single base table, restricted SQL, and refresh on read. Dynamic tables support joins, complex SQL, multiple sources, and pipeline-style chaining.

Step-by-step thought process: 1. Definition syntax — CREATE DYNAMIC TABLE dt_name TARGET_LAG = '5 minutes' WAREHOUSE = wh AS SELECT ... FROM .... 2. Refresh model — Snowflake decides when to refresh based on TARGET_LAG. You declare the freshness SLA, Snowflake schedules

accordingly. 3. Vs materialised views — MVs auto-maintain on each base table change but are restricted to single-table queries (no joins), limited aggregations, and have strict SQL constraints. Dynamic tables remove those restrictions. 4. Vs streams + tasks — dynamic tables are declarative ("this is the result I want, with this lag"). Streams + tasks are procedural ("on this schedule, run this DML"). Dynamic tables remove orchestration boilerplate. 5. Chaining — dynamic table B can be defined on top of dynamic table A; Snowflake propagates refresh through the chain to maintain end-to-end lag.

Edge cases to mention: - TARGET_LAG = DOWNSTREAM means "refresh only when a downstream dynamic table needs you" — useful for chained DAGs. - Some operations (non-deterministic UDFs, certain window functions) disable incremental refresh and force full refresh.

What NOT to say: - Do not call dynamic tables "the new materialised views" — they are a different abstraction and coexist with MVs. - Do not assume zero lag — TARGET_LAG is a target, not a guarantee, and very low lag drives compute cost up.

Common follow-up: *How does Snowflake decide between incremental and full refresh on a dynamic table?*

```
CREATE OR REPLACE DYNAMIC TABLE daily_revenue
  TARGET_LAG = '15 minutes'
  WAREHOUSE = etl_wh
AS
  SELECT order_date, SUM(amount) AS revenue FROM orders GROUP BY 1;
```

Q64 · TARGET_LAG — what does it really control?

Tags: Difficulty: Medium · Asked at: Razorpay, Dream11, InMobi, Slice · Snowflake area: Cost

The question:

> What does TARGET_LAG on a dynamic table actually control, and how does it affect cost?

The 30-second answer: TARGET_LAG declares the maximum staleness you accept — Snowflake refreshes the dynamic table often enough to stay within that bound. Lower lag means more frequent refreshes and higher warehouse cost. Setting it too aggressively (e.g., 1 minute on a complex pipeline) can multiply costs without business benefit.

Step-by-step thought process: 1. The meaning — TARGET_LAG = '5 minutes' means "the dynamic table will be at most 5 minutes behind the source at any read". 2. Snowflake's scheduling — Snowflake computes how long a refresh takes and schedules the next one so the lag stays within target. 3. Cost effect — short lag → more refreshes per hour → more warehouse minutes consumed. Halving lag often doubles cost. 4. Sensible defaults — for downstream BI dashboards, 15-60 min is usually enough. For

operational alerting, 1-5 min. For batch reporting, 1+ hour or DOWNSTREAM. 5. Chained pipelines — when stacking dynamic tables, set leaf-level lag based on the business SLA; intermediate tables can use DOWNSTREAM to refresh only when needed.

Edge cases to mention: - If source data changes faster than refresh can complete, dynamic table lag will silently grow beyond target until the refresh catches up. - TARGET_LAG cannot be lower than the refresh execution time; setting it too low just means continuous refresh.

What NOT to say: - Do not set TARGET_LAG = '1 minute' as default — verify the business actually needs sub-5-min freshness. - Do not assume lag is free — every refresh runs on a warehouse and burns credits.

Common follow-up: *How would you monitor whether a dynamic table is actually meeting its target lag?*

Q65 · Dynamic tables vs Snowpipe Streaming — when to use each

Tags: Difficulty: Hard · Asked at: Swiggy, Razorpay, Flipkart, Meesho · Snowflake area: Architecture

The question:

> When would you use dynamic tables vs Snowpipe Streaming?

The 30-second answer: Snowpipe Streaming is for high-throughput, low-latency row-by-row ingestion from upstream producers (Kafka, app servers). Dynamic tables are for declarative transformation on top of ingested data with a lag SLA. They are complementary, not competitors — Snowpipe Streaming feeds raw tables, dynamic tables transform downstream.

Step-by-step thought process: 1. Snowpipe Streaming responsibility — get data from outside Snowflake into a raw table with sub-second latency, exactly-once semantics, no batching required. 2. Dynamic table responsibility — transform the raw table into business-ready models, maintained automatically with a target lag. 3. Typical architecture — Kafka → Snowpipe Streaming → raw_events → dynamic table cleaned_events (lag 1 min) → dynamic table daily_aggregates (lag 30 min) → BI dashboards. 4. Why this layering works — each layer has its own SLA. Raw is real-time, cleaned is near-real-time, aggregates are eventual. Costs scale with freshness need per layer. 5. Decision rule — ask "is this an ingestion problem or a transformation problem?" Snowpipe Streaming for the first, dynamic tables for the second.

Edge cases to mention: - For files in object storage, classic Snowpipe (event-driven COPY) is cheaper than Snowpipe Streaming. - Dynamic tables cannot directly read from Kafka or external sources; they need data already in Snowflake.

What NOT to say: - Do not present these as alternatives to each other — they sit at different layers of the stack. - Do not use Snowpipe Streaming for in-warehouse transformation — wrong tool.

Common follow-up: *How would Snowpipe Streaming and dbt fit together in the same stack?*

Q66 · Snowpipe — how does file-based ingestion work?

Tags: Difficulty: Medium · Asked at: Flipkart, Razorpay, Meesho, InMobi · Snowflake area: Architecture

The question:

> Explain Snowpipe. How does it ingest files automatically?

The 30-second answer: Snowpipe is a serverless continuous file ingestion service — you define a pipe with a COPY INTO statement and a source stage; new files arriving in the stage trigger an automatic COPY. It can be event-driven (S3/Blob/GCS notifications) or polled via REST API.

Step-by-step thought process: 1. The setup — CREATE STAGE points at a cloud bucket. CREATE PIPE wraps a COPY INTO target FROM @stage statement. AUTO_INGEST = TRUE wires up cloud event notifications. 2. How files trigger ingest — new object lands in bucket → cloud sends event → Snowflake's queue picks it up → Snowpipe runs the COPY using serverless compute → bills per second of compute used. 3. Latency — usually 30-90 seconds from file landing to data queryable. Not real-time but near-real-time. 4. Idempotency — Snowpipe deduplicates by file metadata (filename + checksum); the same file is not loaded twice within the de-dup window (~14 days). 5. Monitoring — PIPE_USAGE_HISTORY shows ingest volume, latency, credit consumption. COPY_HISTORY shows per-file load status.

Edge cases to mention: - File size sweet spot is 100-250 MB per file; many tiny files inflate per-file overhead. - Errors in COPY (bad data) write to LOAD_HISTORY; you decide whether to skip or fail the file via ON_ERROR option.

What NOT to say: - Do not confuse Snowpipe with Snowpipe Streaming — different products with different APIs and latency profiles. - Do not say "Snowpipe runs on my warehouse" — it uses serverless compute, billed separately.

Common follow-up: *How do you handle a corrupt file that Snowpipe keeps trying to load?*

```
CREATE PIPE orders_pipe AUTO_INGEST = TRUE AS
  COPY INTO raw_orders FROM @orders_stage
  FILE_FORMAT = (TYPE = PARQUET) ON_ERROR = 'SKIP_FILE';
```

Q67 · Snowpipe vs Snowpipe Streaming — cost shape

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Dream11, Slice · Snowflake area: Cost

The question:

> What is the cost difference between Snowpipe (file-based) and Snowpipe Streaming?

The 30-second answer: Snowpipe charges per file processed plus serverless compute time, so it favours larger batched files. Snowpipe Streaming charges per row written, with predictable per-row pricing, and favours steady high-volume streams. For low file count → Snowpipe Streaming is often cheaper; for batched files → Snowpipe wins.

Step-by-step thought process: 1. Snowpipe pricing model — per-file overhead + serverless compute. 1,000 files × 1 KB each is expensive due to overhead; 10 files × 100 MB each is cheap. 2. Snowpipe Streaming pricing — credits per row ingested. Predictable scaling, no per-file overhead. 3. Latency tradeoff — Snowpipe ~30-90s, Snowpipe Streaming sub-second to a few seconds depending on flush interval. 4. Operational shape — Snowpipe needs files landed in stage; Snowpipe Streaming needs a producer using the SDK (Kafka connector, Java SDK). 5. Decision matrix — file producers (CDC tools, hourly batch exports, data dumps) → Snowpipe. Continuous event producers (clickstream, IoT, Kafka) → Snowpipe Streaming.

Edge cases to mention: - For mixed workloads, you can use both — Snowpipe Streaming for hot events, Snowpipe for daily reconciliation batch files. - Snowflake's Kafka Connector now uses Snowpipe Streaming under the hood; older versions used Snowpipe.

What NOT to say: - Do not force one tool for everything — they have genuinely different pricing optima. - Do not assume sub-second latency from Snowpipe — its design target is 30-90s.

Common follow-up: *Your producer can write to S3 OR call the Snowpipe Streaming SDK — which would you pick at 100k events/sec?*

Q68 · COPY INTO — what should you know

Tags: Difficulty: Medium · Asked at: Flipkart, Meesho, Razorpay, InMobi · Snowflake area: Architecture

The question:

> Walk through the key options in COPY INTO when loading from a stage.

The 30-second answer: COPY INTO target FROM @stage with options for file format, column mapping, error handling, and validation. Critical knobs: FILE_FORMAT, ON_ERROR (continue/skip/abort), FORCE (re-load already-loaded files), PURGE (delete files after load), VALIDATION_MODE (dry-run to inspect errors).

Step-by-step thought process: 1. FILE_FORMAT — defines parser (CSV, JSON, PARQUET, AVRO, ORC, XML) plus delimiter, header, compression. Best practice: define a named file format object once, reference everywhere. 2. ON_ERROR — CONTINUE (load valid rows, log bad), SKIP_FILE (skip whole file on any error), SKIP_FILE_N (skip after N errors), ABORT_STATEMENT (fail fast). 3. FORCE — set TRUE to re-load files already in load history. Useful for reprocessing after a transform change. 4. PURGE — delete loaded files from the stage after success. Good for cost control on staging buckets. 5. VALIDATION_MODE — RETURN_ERRORS or RETURN_N_ROWS lets you dry-run a load to see what would happen, without committing.

Edge cases to mention: - Column transformation in COPY INTO — you can SELECT-transform during the COPY (e.g., COPY INTO t FROM (SELECT $1::INT, $2::STRING ... FROM @stage)). - Pattern matching — PATTERN = '.*[.]json' lets you filter files in the stage.

What NOT to say: - Do not say "always FORCE = TRUE to be safe" — that disables Snowflake's deduplication and you will double-load. - Do not skip VALIDATION_MODE on first runs of a new pipeline.

Common follow-up: *How would you handle a column-order change in upstream files?*

```
COPY INTO orders FROM @orders_stage
  FILE_FORMAT = (FORMAT_NAME = 'PARQUET_FMT')
  ON_ERROR = 'SKIP_FILE'
  PURGE = TRUE;
```

Q69 · SQL UDFs vs JavaScript UDFs vs Python UDFs — when each?

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Dream11 · Snowflake area: Architecture

The question:

> Compare SQL UDFs, JavaScript UDFs, and Python UDFs in Snowflake. When would you pick each?

The 30-second answer: SQL UDFs are inlined into the query plan, fastest, but limited to SQL expressions. JavaScript UDFs handle imperative logic and JSON manipulation but run in a sandbox per row. Python UDFs use Snowpark and let you use Python libraries (pandas, numpy, scikit-learn) for heavy logic, including vectorised UDFs for batch processing.

Step-by-step thought process: 1. SQL UDF — pure SQL function body. Fastest, fully inlinable, optimiser can push predicates through. Use for simple expression reuse (e.g., normalise phone number format). 2. JavaScript UDF — runs JS engine per row in a sandbox. Good for procedural logic, regex, JSON traversal that SQL cannot express cleanly. Slower than SQL but flexible. 3. Python UDF — Snowpark Python runtime. Pre-imported packages (numpy, pandas, scikit-learn) available. Two flavours:

scalar UDF (per row) and vectorised UDF (pandas batch). 4. Vectorised Python UDF — receives a pandas Series, returns a pandas Series. Hugely faster than per-row Python for large batches. Best for ML scoring, statistical transforms. 5. Decision rule — SQL UDF if you can express it in SQL. JS UDF if you need procedural logic with no heavy libraries. Python UDF if you need NumPy/Pandas/ML, especially vectorised.

Edge cases to mention: - All UDF types support secure variants — hides the function body from non-owners. - Python UDFs have package allowlist (Anaconda-managed channel); not every PyPI package is available.

What NOT to say: - Do not assume Python UDFs are always slower — vectorised Python UDFs can outperform JS for batch transforms. - Do not use JS UDF for ML scoring — Python UDF is the right tool.

Common follow-up: *How does Snowflake sandbox Python UDFs for security?*

```
CREATE FUNCTION normalize_phone(p STRING) RETURNS STRING LANGUAGE SQL
  AS $$ REGEXP_REPLACE(p, '[^0-9]', '') $$;

CREATE FUNCTION score_row(features ARRAY) RETURNS FLOAT LANGUAGE PYTHON
  RUNTIME_VERSION = '3.10' PACKAGES = ('numpy', 'scikit-learn')
  HANDLER = 'predict' AS $$ ... $$;
```

Q70 · UDF performance and security model

Tags: Difficulty: Hard · Asked at: Razorpay, Meesho, Flipkart, InMobi · Snowflake area: Performance

The question:

> What is the performance and security model for non-SQL UDFs in Snowflake?

The 30-second answer: JavaScript and Python UDFs run in isolated sandboxes per query, with no network access (unless explicitly granted via external access integrations) and no filesystem write. They are slower per row than native SQL because of sandbox overhead and serialisation, but vectorised Python UDFs reduce overhead by batching.

Step-by-step thought process: 1. Sandbox isolation — UDF execution runs in restricted JVM (JS) or Python runtime, no syscalls outside allowlist, no network by default. 2. External access — you can grant network egress via External Access Integration objects, but it must be explicitly created and granted; nothing implicit. 3. Per-row overhead — every row enters the runtime, serialisation cost. Vectorised UDFs amortise this by passing a batch. 4. Memory limits — UDFs have per-query memory caps; very large payloads (large arrays/JSON per row) can OOM. 5. Caching — UDF results are not cached the same way scalar SQL results are; the optimiser may not always inline or memoise.

Edge cases to mention: - Secure UDFs hide function body from non-owner roles; necessary for shared functions that contain logic you do not want exposed. - Java UDFs and stored procedures exist too with similar sandboxing; choose based on team language familiarity.

What NOT to say: - Do not assume UDFs can talk to external APIs by default — they cannot. - Do not write a per-row Python UDF when a vectorised UDF would suffice on a large table.

Common follow-up: *How would you give a Python UDF access to a third-party REST API?*

Q71 · Secure views, row access policies, dynamic masking

Tags: Difficulty: Hard · Asked at: Razorpay, Flipkart, Jupiter, Slice · Snowflake area: Security

The question:

> Explain secure views, row access policies, and dynamic data masking. How do they differ?

The 30-second answer: Secure views hide their definition from non-owners and prevent optimiser side-channels that could leak data. Row access policies filter which rows a role sees. Dynamic masking policies transform column values at query time (e.g., hash PII). They compose — a single object can have all three layered.

Step-by-step thought process: 1. Secure view — CREATE SECURE VIEW. Two effects: (a) view definition not visible to non-owners; (b) optimiser does not push predicates into the view in ways that could expose underlying data via timing attacks. 2. Row access policy — attached to a table or view, a function returning boolean based on session context (current_role, current_user, custom mapping table). Returns rows where the policy returns TRUE. 3. Dynamic masking policy — attached to a column. A function that returns the column value or a masked version based on current_role. E.g., return phone as-is to PII_ANALYST role, return masked '****' to ANALYST role. 4. Composition — a table can have row access policy on it AND masking policies on individual columns AND be queried through a secure view. All three apply. 5. Use case — shared analytics warehouse where compliance requires that contractors see only Indian-region rows and PII is masked except for the fraud team.

Edge cases to mention: - Masking policies can use external mapping tables (e.g., a "users_to_regions" table) to compute the mask, enabling fine-grained control without one policy per role. - Row access policies have a performance cost (extra filter); test with EXPLAIN on policy-protected tables.

What NOT to say: - Do not say "regular view with WHERE clause = secure view" — secure views have semantic protections against side-channel leaks. - Do not implement PII masking via a wrapper view alone — masking policies are the supported, audited mechanism.

Common follow-up: *If a row access policy and a column masking policy both apply, what is the order of evaluation?*

```
CREATE MASKING POLICY mask_email AS (val STRING) RETURNS STRING ->
  CASE WHEN CURRENT_ROLE() IN ('PII_ANALYST') THEN val
       ELSE REGEXP_REPLACE(val, '.+@', '***@') END;

ALTER TABLE customers MODIFY COLUMN email SET MASKING POLICY mask_email;
```

Q72 · Snowflake RBAC — role hierarchy and best practices

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Swiggy, Jupiter · Snowflake area: Security

The question:

> How does Snowflake's role-based access control work? Describe a typical role hierarchy.

The 30-second answer: Snowflake's RBAC is role-based with role inheritance — a role gets all privileges of roles granted to it. Best practice is a layered hierarchy: object-access roles (read/write per dataset) → functional roles (analyst, engineer) → user roles. Privileges grant on objects, roles grant on roles, users are granted roles.

Step-by-step thought process: 1. Primitives — privileges (SELECT, INSERT, USAGE, OWNERSHIP, etc.) on objects (databases, schemas, tables, warehouses). Granted to roles, not users. 2. Roles grant to roles — GRANT ROLE access_finance_read TO ROLE analyst. The analyst role now inherits finance read access. 3. Users grant from roles — GRANT ROLE analyst TO USER abhishek. The user can SET ROLE analyst to act as that role. 4. Hierarchy pattern — at the bottom, access roles (e.g., DB_FINANCE_RO, DB_FINANCE_RW) hold privileges on objects. In the middle, functional roles (ANALYST, ENGINEER) inherit access roles. At top, user roles map users to functional roles. 5. Why this pattern — onboarding/offboarding becomes "grant/revoke a functional role" rather than thousands of object grants. Audit becomes easier.

Edge cases to mention: - Roles can be granted to other roles to form a DAG (not just a tree). - Ownership is special — a role owns objects it creates; ownership transfer is explicit (GRANT OWNERSHIP).

What NOT to say: - Do not grant privileges directly to users — Snowflake does not support that; all privileges flow through roles. - Do not collapse all access into ACCOUNTADMIN — it bypasses least-privilege principles.

Common follow-up: *What happens to permissions when an object is recreated?*

Q73 · Future grants and managed access schemas

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Meesho, Slice · Snowflake area: Security

The question:

> What are future grants and managed access schemas? When do you need them?

The 30-second answer: Future grants apply to objects created after the grant statement, so newly-created tables in a schema automatically inherit predefined permissions. Managed access schemas centralise grant ownership in the schema owner, preventing object owners from granting privileges on their own objects.

Step-by-step thought process: 1. Future grants — GRANT SELECT ON FUTURE TABLES IN SCHEMA finance TO ROLE analyst. New tables in finance automatically get SELECT for analyst. Without this, every new table needs manual grants. 2. Why critical — in an ETL-heavy environment where new tables appear weekly, future grants prevent permission drift. 3. Managed access schemas — by default, the owner of an object can grant privileges on it. In a managed access schema, only the schema owner can grant privileges, regardless of object ownership. 4. Why use managed — prevents engineers from accidentally granting wider access than the security team intended. Centralises grant control. 5. Combination — managed access schema + future grants = secure-by-default. New tables get the predefined grant set automatically, and engineers cannot override.

Edge cases to mention: - Future grants only apply at the schema level (e.g., all FUTURE tables in schema X). You cannot future-grant for a single object pattern. - Multiple future grants for the same role + object type stack; you can revoke specific ones if needed.

What NOT to say: - Do not say "future grants apply retroactively to existing objects" — they do not; existing objects need explicit grants. - Do not use future grants without auditing periodically — they can quietly broaden access if roles change.

Common follow-up: *How would you set up secure-by-default access for a new schema for a new team?*

```
GRANT USAGE ON SCHEMA finance TO ROLE analyst;
GRANT SELECT ON FUTURE TABLES IN SCHEMA finance TO ROLE analyst;
GRANT SELECT ON FUTURE VIEWS IN SCHEMA finance TO ROLE analyst;
```

Q74 · System roles — ACCOUNTADMIN, SYSADMIN, USERADMIN, SECURITYADMIN

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Jupiter, InMobi · Snowflake area: Security

The question:

> Walk through Snowflake's system roles and the principle behind separating them.

The 30-second answer: ACCOUNTADMIN is the super-role (account-wide everything). SYSADMIN owns data infrastructure (databases, warehouses). USERADMIN manages users and roles. SECURITYADMIN manages grants and monitors usage. The separation lets you grant power on one axis (e.g., data) without granting power on another (e.g., users).

Step-by-step thought process: 1. ACCOUNTADMIN — full account control including billing. Should be used only by 2-3 named humans with break-glass procedures, not for daily work. 2. SYSADMIN — root of the object hierarchy. Creates databases, schemas, warehouses. Functional roles should inherit from SYSADMIN, not from ACCOUNTADMIN. 3. USERADMIN — creates and manages users and roles. Does not grant privileges directly to objects. 4. SECURITYADMIN — grants and revokes privileges on objects, manages access. Pairs with USERADMIN for full identity + access management. 5. Why the separation — least-privilege. A user-management tool can use USERADMIN without being able to see data. A data-platform tool can use SYSADMIN without being able to manage users.

Edge cases to mention: - Custom roles should be in a hierarchy under SYSADMIN, so SYSADMIN retains visibility/control over objects created by custom roles. - ACCOUNTADMIN must not be auto-granted; treat it like root and use rarely.

What NOT to say: - Do not assume SYSADMIN can manage users — it cannot, by default. - Do not skip GRANT ROLE custom_role TO ROLE SYSADMIN — without it, SYSADMIN loses sight of custom-role-owned objects.

Common follow-up: *Why must you grant your custom roles to SYSADMIN?*

Q75 · Snowflake caching layers — result, metadata, warehouse

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Flipkart, Dream11 · Snowflake area: Performance

The question:

> Snowflake has three caching layers. Explain each.

The 30-second answer: Result cache — query result stored in services layer for 24 hours, returned instantly for identical re-runs (any warehouse). Metadata cache — table statistics in services layer, used for queries like COUNT(*). Warehouse local cache — per-warehouse SSD cache of recently-scanned micro-partitions, cleared when warehouse suspends.

Step-by-step thought process: 1. Result cache — at services layer, scoped to your account. If you run the exact same query (same text, same role, same data version) within 24 hours, Snowflake serves the

cached result with no warehouse usage at all. 2. Result cache invalidation — any change to underlying data invalidates the cached result. Tables with constant ingestion never benefit. 3. Metadata cache — Snowflake stores per-table summary stats (row counts, min/max). Queries like SELECT COUNT(*) or SELECT MAX(date) can be served from metadata with no scan, no warehouse needed. 4. Warehouse local cache — each warehouse has SSD cache of recently-read micro-partitions. Subsequent queries on the same partitions read from SSD, much faster than re-reading object storage. 5. Cache loss on suspend — when a warehouse auto-suspends, its local cache is lost. Resuming a warehouse means a cold cache.

Edge cases to mention: - Result cache requires deterministic queries; functions like CURRENT_TIMESTAMP or sequences break cacheability. - For BI dashboards with predictable repeating queries, warm warehouse + result cache is the killer combo.

What NOT to say: - Do not confuse result cache with warehouse cache — they live at different layers and behave differently. - Do not assume metadata cache works for arbitrary aggregations — it is specific to operations that map to stored stats.

Common follow-up: *If you SELECT COUNT()* FROM big_table, which cache serves it?*

Q76 · Reading a query profile

Tags: Difficulty: Hard · Asked at: Razorpay, Flipkart, Meesho, InMobi · Snowflake area: Performance

The question:

> What do you look for when reading a Snowflake query profile?

The 30-second answer: Start at "Most Expensive Nodes" — usually a TableScan or Join. Check partitions scanned vs total (pruning effectiveness). Check spillage (warehouse too small). Check whether the build side of joins is large (causes broadcast inefficiency). Check skew in any operator that shows uneven row distribution.

Step-by-step thought process: 1. Open the profile — Snowsight → Query History → click query → Query Profile tab. 2. Total elapsed time — overall, vs compilation time. High compilation suggests a complex plan; high execution suggests a heavy scan or join. 3. TableScan — check "Partitions Scanned" / "Total Partitions". Pruning ratio under 50% on a filtered query is a red flag. 4. Joins — look at row counts on each side. If the build side is huge (>100M rows), broadcast may have failed and Snowflake fell back to a slow strategy. 5. Spillage — "Bytes spilled to local storage" or "Bytes spilled to remote storage" indicates warehouse memory pressure. Local spill is OK in moderation; remote spill is a strong signal to scale up. 6. Skew — operators showing one partition processing 10x more rows than others suggest data skew (e.g., a hot key).

Edge cases to mention: - Long "Initialisation" phase often means cold warehouse cache; consider running representative queries first to warm. - "Synchronisation" time accumulates from inter-cluster communication; large in wide joins.

What NOT to say: - Do not just look at total time — diagnose where time is spent. - Do not optimise without measuring; the profile shows truth, your guesses do not.

Common follow-up: *Your query profile shows 95% time in remote disk I/O. What do you do?*

Q77 · EXPLAIN in Snowflake — what does it tell you?

Tags: Difficulty: Medium · Asked at: Razorpay, Dream11, Flipkart, Slice · Snowflake area: Performance

The question:

> What does EXPLAIN show in Snowflake, and when do you use it vs the query profile?

The 30-second answer: EXPLAIN shows the planned execution tree, operators, estimated row counts, and partitions assigned per scan — all without running the query. Use EXPLAIN to validate plan shape before running expensive queries; use query profile after running to see actual runtime metrics.

Step-by-step thought process: 1. EXPLAIN USING TEXT/JSON returns a plan. Includes operators (TableScan, Join, Aggregate, Sort), estimated rows, estimated partitions, join strategy (hash, broadcast). 2. Pre-run validation — for a query that may scan TBs, EXPLAIN tells you the estimated scan volume and whether pruning is engaged, before you burn warehouse minutes. 3. Limitations — EXPLAIN gives estimates, not actuals. Optimiser estimates can be off, particularly for complex joins or skewed data. 4. EXPLAIN vs query profile — EXPLAIN is what the optimiser planned. Query profile is what actually ran. They can differ; investigate large gaps. 5. Practical use — write a query → EXPLAIN → check partitions and join order → fix predicates or hints if needed → run → check profile.

Edge cases to mention: - EXPLAIN does not show data values; you cannot use it to debug correctness, only plan shape. - For very complex queries, the JSON output is more parseable than the text format.

What NOT to say: - Do not assume EXPLAIN row counts match runtime row counts — they are estimates. - Do not skip EXPLAIN for very large ad-hoc queries; it is free and catches obvious mistakes.

Common follow-up: *EXPLAIN shows a hash join but query profile shows a nested loop. What happened?*

```
EXPLAIN USING TEXT
SELECT o.id, c.name FROM orders o JOIN customers c ON o.cust_id = c.id WHERE o.date = '2026-05-01
```

Q78 · Query history and warehouse history — production debugging

Tags: Difficulty: Medium · Asked at: Swiggy, Razorpay, Jupiter, InMobi · Snowflake area: Performance

The question:

> What views would you query to debug a sudden cost spike or slow queries?

The 30-second answer: For queries: SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY for execution time, bytes scanned, credits used. For warehouses: WAREHOUSE_METERING_HISTORY for credit usage over time. For tasks: TASK_HISTORY. For pipes: PIPE_USAGE_HISTORY. ACCOUNT_USAGE views have up to 45 min lag; INFORMATION_SCHEMA equivalents are real-time but scoped to last 7-14 days.

Step-by-step thought process: 1. Identify symptom — sudden cost spike → look at WAREHOUSE_METERING_HISTORY by warehouse and hour to find the culprit warehouse and time window. 2. Drill into queries — QUERY_HISTORY filtered by warehouse_name and start_time within the spike window. Order by execution_time DESC or bytes_scanned DESC. 3. Per-query forensics — pick a heavy query → click into Query Profile → see what scanned, joined, spilled. 4. For background services — AUTOMATIC_CLUSTERING_HISTORY, MATERIALIZED_VIEW_REFRESH_HISTORY, PIPE_USAGE_HISTORY each have their own usage views, billed separately from warehouses. 5. Build dashboards — most teams build Snowsight or BI dashboards on top of ACCOUNT_USAGE for daily cost monitoring with alerts on anomalies.

Edge cases to mention: - ACCOUNT_USAGE has 365 days of history; INFORMATION_SCHEMA has 7-14 days. Use ACCOUNT_USAGE for trend analysis. - Be careful with cost attribution — queries can run on warehouses owned by different teams; aggregate by role or user as well.

What NOT to say: - Do not assume INFORMATION_SCHEMA is up-to-date with everything — it has narrower retention. - Do not skip RESOURCE_MONITORS for proactive cost control; setting limits prevents runaway warehouses.

Common follow-up: *How would you set up an alert for queries scanning more than 1 TB?*

```
SELECT user_name, warehouse_name, query_text, total_elapsed_time, credits_used_cloud_services
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE start_time >= DATEADD(HOUR, -24, CURRENT_TIMESTAMP())
ORDER BY total_elapsed_time DESC LIMIT 50;
```

Q79 · Cost optimisation — practical playbook

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Flipkart, Meesho · Snowflake area: Cost

The question:

> Walk through the cost optimisation steps you would take on a Snowflake account.

The 30-second answer: Right-size warehouses (smallest that meets latency SLA), enforce auto-suspend (60s for ETL, 300-600s for BI), use multi-cluster only where concurrency demands it, audit large queries via QUERY_HISTORY, set resource monitors with credit limits, and review high-DML clustered tables for re-clustering churn.

Step-by-step thought process: 1. Warehouse audit — for each warehouse, look at WAREHOUSE_METERING_HISTORY to see daily credit burn. Identify always-on warehouses with low utilisation. 2. Right-size — for each warehouse, sample QUERY_HISTORY for time spent in queue vs execution. If no queueing and queries fast: try one size smaller. If queueing: add multi-cluster. 3. Auto-suspend — set 60s on every ETL warehouse, 300-600s on BI warehouses. Audit any warehouse without auto-suspend set. 4. Query culprits — top 20 queries by credits over the last 7 days. Often a handful of bad queries (no filter, full scan, unnecessary cross-joins) drive 30% of cost. 5. Background services — check AUTOMATIC_CLUSTERING_HISTORY, MATERIALIZED_VIEW_REFRESH_HISTORY, PIPE_USAGE_HISTORY for surprises. A misconfigured clustered table can quietly burn thousands of credits. 6. Resource monitors — set monthly credit caps per warehouse to fail-safe against runaway costs. Notification at 80%, suspend at 100%. 7. Storage — review tables with high TIME_TRAVEL_BYTES via TABLE_STORAGE_METRICS. Reduce retention where business does not need 30+ days.

Edge cases to mention: - Storage cost is typically 5-15% of total — focus optimisation effort on compute first. - Cloud services usage > 10% of compute is billed; runaway DDL or metadata-heavy queries can push you over.

What NOT to say: - Do not just shrink warehouses blindly — measure first. - Do not skip resource monitors; one bad query can burn a month's budget overnight.

Common follow-up: *Your bill jumped 3x last month — how do you find what changed?*

Q80 · Materialised views vs query-on-demand — cost tradeoff

Tags: Difficulty: Hard · Asked at: Swiggy, Razorpay, Dream11, Flipkart · Snowflake area: Cost

The question:

> When does it make sense to use a materialised view vs just computing the aggregate on demand each time?

The 30-second answer: Materialised views amortise compute cost when the same expensive aggregate is queried many times between source updates. They cost continuous maintenance compute as source changes. If query frequency × on-demand cost > MV maintenance + (cheaper) read cost, use the MV; otherwise compute on demand.

Step-by-step thought process: 1. The maintenance cost — MV is auto-refreshed on every base table change. Heavy DML on the base table means heavy MV refresh credits. 2. The query cost saved — every read of the MV is cheap (pre-computed, often small). Versus the underlying complex aggregate which may scan a large fact table. 3. Break-even math — if the on-demand query costs C credits and runs N times per day, on-demand cost = N × C. If MV maintenance costs M per day and read cost is negligible, MV breaks even when N × C > M. 4. When MV wins — read-heavy aggregates, source updated occasionally, many concurrent dashboard reads. 5. When on-demand wins — source updated continuously (high M), query run rarely (low N), or large result cache covers the queries.

Edge cases to mention: - Snowflake MVs have restrictions: single base table, limited SQL constructs, no joins. For richer transforms, use dynamic tables. - Result cache often makes the on-demand approach surprisingly cheap for repeated identical queries.

What NOT to say: - Do not say "always materialise expensive aggregates" — high source churn turns this into a money pit. - Do not skip dynamic tables when MV restrictions bite — they cover more SQL with similar cost shape.

Common follow-up: *If the same dashboard query runs 1,000 times a day on a stable source, why might result cache make a MV unnecessary?*

End of Section 2 — Snowflake (40 questions).

Section 3 — Google BigQuery (30 Questions)

BigQuery comes up in two kinds of interviews: data engineering / analytics roles at product companies that run their analytics on GCP (Swiggy, Razorpay, Meesho, Dream11, Glance, Sharechat, Disney+Hotstar, Flipkart-Ads), and consulting / GCC roles serving Google Cloud customers (Searce, Niveus, Quantiphi, Deloitte GCP practice, TCS GCP CoE).

Interviewers here probe three things in order: do you understand the **storage-compute split** and the **bytes-scanned billing model**, can you **write nested SQL** without flattening everything, and can you **read a query plan and bring the bill down**. The 30 questions below are sequenced in that order — architecture first, then physical design (partitioning, clustering), then nested data, then operational topics (scripting, scheduling, cost, MVs, ML, multi-region).

Indian English. Direct. Numbers and limits are accurate as of late 2025 / early 2026.

Q81 · Why is BigQuery called serverless, and how does that change the way you write SQL?

Tags: Difficulty: Easy · Asked at: Swiggy, Razorpay, Quantiphi, Searce · BQ area: Architecture

The question:

> Walk me through what "serverless" actually means in BigQuery. Why does the team here say "you stop thinking about clusters"?

The 30-second answer: You never provision, size, or patch a node. Google runs a multi-tenant Dremel cluster and hands you slots on demand. You only declare the query — BigQuery decides parallelism, shuffle, and machine count.

Step-by-step thought process: 1. Traditional MPP warehouses (on-prem Teradata, Redshift classic) make you pick node count and instance type up front. You pay for nodes whether queries run or not. 2. BigQuery splits storage from compute. Storage lives in Colossus as columnar Capacitor files. Compute lives in a shared pool of slots. 3. When you submit a query, the query coordinator parses it, plans stages, and grabs slots from the pool. On the on-demand model, you get up to 2000 slots per project by default (burstable higher) and pay only for bytes scanned. 4. Because compute is elastic, you stop optimising for "use all the cores" and start optimising for "scan fewer bytes" — partition pruning, clustering, SELECT only the columns you need. 5. Failure handling is hidden — if a worker dies mid-query, BigQuery reassigns the shard. You do not write retry logic.

Edge cases to mention: - Serverless does not mean unlimited — you still hit slot contention during peak hours and project-level concurrency limits (100 concurrent interactive queries by default). - DDL is still synchronous; very large CTAS jobs can starve other workloads in the same reservation.

What NOT to say: - "BigQuery has no compute" — it absolutely has compute, you just do not see the VMs. - "Storage is free" — storage is cheap (~$0.02/GB/month active), but it is billed.

Common follow-up: *Then how does Google charge you — by query, by node-hour, or by bytes?*

Q82 · Explain Dremel and the columnar storage format. Why does column pruning matter so much?

Tags: Difficulty: Medium · Asked at: Google GCC, Niveus, Dream11, Flipkart · BQ area: Architecture

The question:

> Tell me what happens physically when I run `SELECT user_id FROM events WHERE event_date = '2026-05-01'` on a 2 TB table.

The 30-second answer: Dremel reads only the `user_id` column file and only the partition for that date — so instead of 2 TB, the engine might scan 4 GB. Columnar storage + partition pruning is the whole cost story.

Step-by-step thought process: 1. BigQuery stores each column separately in Capacitor format — a proprietary columnar layout with heavy compression (dictionary, RLE, delta). 2. When the query planner sees `SELECT user_id`, it opens only the `user_id` column files. Other 80 columns are never read from disk. 3. The `WHERE event_date = '2026-05-01'` clause is matched against the partition metadata. If the table is partitioned on `event_date`, only that one day's files are opened. 4. Dremel then fans the read across hundreds of leaf workers, each scanning a shard. Results stream up a tree of mixers that aggregate them. 5. Billing meter records bytes read from the columns that were actually opened — not the on-disk file size, but the logical uncompressed bytes of the columns referenced.

Edge cases to mention: - `SELECT *` defeats column pruning entirely — every column file is opened. This is the single biggest cost mistake juniors make. - Functions like `COUNT(*)` are special — they hit table metadata and scan 0 bytes. - `SELECT * EXCEPT(col)` is read as wildcard expansion — it still scans all columns except the excluded ones.

What NOT to say: - "BigQuery uses Parquet" — it does not, internally. Capacitor is similar in spirit but proprietary. (You can export to Parquet.)

Common follow-up: *If columnar is so good, why do we still need partitioning on top of it?*

Q83 · Why does BigQuery charge by bytes scanned and not by query time?

Tags: Difficulty: Easy · Asked at: Searce, Quantiphi, TCS GCP CoE · BQ area: Architecture

The question:

> A junior says "this query took 4 seconds so it should be cheap." Why is that wrong?

The 30-second answer: On the on-demand model, billing is decoupled from runtime. A 4-second query that scans 1 TB costs the same as a 4-minute query that scans 1 TB. You pay for data read, not clock time.

Step-by-step thought process: 1. The on-demand price is a flat rate per TB scanned (~$6.25/TiB in most regions as of 2025). 2. Runtime is a function of slots available. If your project has lots of free slots, even a 1 TB scan finishes in seconds. The bill stays the same. 3. This model rewards good physical

design — partitioning, clustering, columnar selection — and punishes lazy `SELECT *` queries. 4. The flat-rate / reservation model is different: you buy slot-hours and runtime does matter, because you can starve your reservation. 5. Caching is the one exception — cached results are free and instant. Identical queries within 24 hours hit the cache by default.

Edge cases to mention: - DML statements (INSERT, UPDATE, DELETE, MERGE) are billed on bytes scanned + bytes written for some operations. - Streaming inserts have a separate per-row charge. - Query cache is per-user by default; collaborative caching needs explicit setup.

What NOT to say: - "Just add more slots" — slots affect runtime, not on-demand billing.

Common follow-up: *When does flat-rate become cheaper than on-demand?*

Q84 · On-demand vs flat-rate / reservations — when do you switch?

Tags: Difficulty: Medium · Asked at: Razorpay, Meesho, Niveus, Searce · BQ area: Cost

The question:

> Our monthly BigQuery bill is around ₹8 lakh, all on on-demand. Should we move to reservations?

The 30-second answer: Once on-demand spend is predictable and high enough that flat-rate slot-months work out cheaper, switch. The break-even is usually around the cost of one annual slot commitment (~₹17–18 lakh/year for 500 Enterprise slots).

Step-by-step thought process: 1. On-demand: you pay per TB scanned, no commitment. Good for spiky / unpredictable workloads. 2. Flat-rate (reservations): you commit to a number of slots — pay by the hour, month, or year (annual is cheapest). Queries are free of bytes-scanned charges but compete for your fixed slot pool. 3. Calculate your average daily bytes scanned for the last 30 days from `INFORMATION_SCHEMA.JOBS_BY_PROJECT`. Multiply by per-TB price. Compare to slot-month price at the tier you would pick. 4. Reservations also unlock autoscaling — set baseline + max slots, BigQuery scales between them in 100-slot increments. 5. You can mix: keep dev / ad-hoc projects on on-demand, put production ELT into a reservation.

Edge cases to mention: - Reservations are per region — buying slots in `us-central1` does not help queries in `asia-south1`. - Idle slots in a reservation are wasted unless you enable slot sharing across assignments. - Reservations bill at slot-hour granularity for hourly commitments, slot-month for monthly.

What NOT to say: - "Reservations are always cheaper" — for spiky workloads they can be 2-3x more expensive than on-demand.

Common follow-up: *What is the difference between Standard, Enterprise, and Enterprise Plus editions?*

Q85 · BigQuery editions — Standard, Enterprise, Enterprise Plus

Tags: Difficulty: Medium · Asked at: Searce, Niveus, Quantiphi, Deloitte · BQ area: Architecture

The question:

> When would you put a customer on Enterprise Plus instead of Enterprise?

The 30-second answer: Enterprise Plus when you need column-level encryption with CMEK, cross-region disaster recovery, or longer slot commitment discounts. Enterprise is the workhorse tier. Standard is read-only ad-hoc analytics.

Step-by-step thought process: 1. **Standard edition**: cheapest per slot-hour, but limited to BigQuery basics — no fine-grained access control, no scheduled queries with custom service accounts, no time-travel beyond default. 2. **Enterprise**: full feature set most teams expect — row-level security, column-level access policies, materialized views, BigQuery ML, scheduled queries, Dataform. 3. **Enterprise Plus**: adds CMEK at column level, cross-region replicas of datasets for DR, longer time-travel (up to 7 days), and the deepest commitment discount tiers. 4. Editions only apply when you buy slot reservations. On-demand projects do not pick an edition. 5. You can mix editions within an organisation — reservations are scoped per project / folder.

Edge cases to mention: - Moving between editions is a reservation-config change, not a data migration. - Some features (e.g. fine-grained DLP integration) are Enterprise+ only and silently no-op on Standard.

What NOT to say: - "Enterprise Plus is just Enterprise with a higher discount" — feature gates matter.

Common follow-up: *How does autoscaling work inside a reservation?*

Q86 · Slot autoscaling — baseline, max, and how scaling triggers

Tags: Difficulty: Medium · Asked at: Dream11, Sharechat, Razorpay, Niveus · BQ area: Cost

The question:

> Set baseline 100, max 1000. What actually happens when a heavy query lands at 3 am?

The 30-second answer: BigQuery scales up in 100-slot blocks as soon as the scheduler sees demand exceed current capacity. Scale-down has a cool-down of about a minute to avoid thrash. You pay only for slot-seconds actually used above baseline.

Step-by-step thought process: 1. Baseline slots are always-on and billed at the committed rate (cheapest with annual commit). 2. Max slots define the autoscale ceiling. Anything between baseline and

max is billed at the autoscale rate per slot-second. 3. Scheduler looks at slot demand each second. If queries queued + running need more than current slots, it adds a 100-slot block. 4. Scale-down is conservative — slots are released after a cool-down to handle the next burst without re-warming. 5. Combine baseline = steady ELT load, max = peak BI traffic. This gives flat-rate economics for known load and on-demand-like elasticity for spikes.

Edge cases to mention: - Setting baseline = 0 is allowed (pure autoscale) but loses you the commitment discount. - Each reservation has its own autoscale range; assignments route projects to reservations. - Hard cap at organisation-level slot quota — even max cannot exceed it.

What NOT to say: - "It scales per slot" — minimum step is 100 slots.

Common follow-up: *How do you decide baseline vs max for a new workload?*

Q87 · Time-partitioned vs ingestion-time vs integer-range partitioning

Tags: Difficulty: Medium · Asked at: Swiggy, Meesho, Flipkart, Glance · BQ area: Schema

The question:

> I have a 5 TB events table. What partition strategy do you pick and why?

The 30-second answer: Partition on a `DATE` or `TIMESTAMP` column that you filter on most — usually `event_date`. Use ingestion-time only if no good business-date column exists. Use integer-range only for non-temporal keys like `user_id` buckets.

Step-by-step thought process: 1. **Time-unit column partitioning**: choose a DATE / TIMESTAMP / DATETIME column. Granularity can be HOUR, DAY, MONTH, or YEAR. Day is most common. 2. **Ingestion-time partitioning**: BigQuery auto-creates a pseudo-column `_PARTITIONTIME` based on when the row was inserted. Useful for raw logs where business date == insert date. 3. **Integer-range partitioning**: define start, end, interval over an INT64 column. Used for sharding by user_id, tenant_id, etc. 4. A table can have only one partition column. Pick the column you filter on in 80% of queries. 5. Partition limits: 10,000 partitions per table. Day-partitioned tables therefore cover ~27 years. Hour-partitioned tables cover ~13 months.

Edge cases to mention: - Cannot re-partition in place — you create a new table via CTAS with the new partition spec, then swap. - For ingestion-time, queries use `_PARTITIONTIME` or `_PARTITIONDATE` pseudo-columns in the filter. - Integer-range needs you to commit to the interval at create time.

What NOT to say: - "Partition by primary key" — there is no PK in BigQuery; partitioning is for pruning, not uniqueness.

Common follow-up: *What is the `_PARTITIONTIME` pseudo-column and when do I see it?*

Q88 · `_PARTITIONTIME` pseudo-column — when and how to use it

Tags: Difficulty: Easy · Asked at: Razorpay, Niveus, Dream11 · BQ area: Schema

The question:

> What is `_PARTITIONTIME` and when is it useful?

The 30-second answer: A virtual column on ingestion-time-partitioned tables that exposes the partition boundary as a TIMESTAMP. You filter on it to prune partitions; for column-partitioned tables you filter on the actual column instead.

Step-by-step thought process: 1. Created automatically on ingestion-time tables — you never declare it. 2. `_PARTITIONTIME` is a TIMESTAMP truncated to the day (or hour) boundary. `_PARTITIONDATE` is the same as a DATE. 3. Filter on it like a normal column: `WHERE _PARTITIONTIME = TIMESTAMP("2026-05-01")`. The planner uses this to skip other partitions. 4. You can also write to a specific partition via the partition decorator: `events$20260501`. 5. Column-partitioned tables do not expose `_PARTITIONTIME` — filter on the actual column instead.

Edge cases to mention: - `_PARTITIONTIME` is NULL for the streaming buffer (rows in the last few minutes before they land in a partition). - Cannot SELECT `_PARTITIONTIME *` — it is a pseudo-column, not part of `SELECT *`.

What NOT to say: - "It works on every BigQuery table" — only ingestion-time-partitioned tables.

Common follow-up: *How do you enforce that no one runs a full-table scan on this partitioned table?*

Q89 · Require partition filter — the one switch that saves you ₹2 lakh a month

Tags: Difficulty: Medium · Asked at: Meesho, Glance, Razorpay, Quantiphi · BQ area: Cost

The question:

> A junior wrote `SELECT * FROM events` on a 50 TB partitioned table. How do you make sure that never happens again?

The 30-second answer: Set `require_partition_filter = true` on the table. Any query missing a filter on the partition column will fail at planning time instead of scanning 50 TB.

Step-by-step thought process: 1. The table option `require_partition_filter` rejects queries that do not include a WHERE clause on the partition column. 2. Set it at creation time: `CREATE`

`TABLE ... PARTITION BY event_date OPTIONS(require_partition_filter = true)`. 3. Or alter an existing table: `ALTER TABLE events SET OPTIONS(require_partition_filter = true)`. 4. The check is syntactic on the partition column — it does not validate that the filter is selective, only that it exists. 5. Pair with a default query template in BI tools so analysts cannot forget.

Edge cases to mention: - A subquery that filters on the partition column still satisfies the check. - `WHERE event_date IS NOT NULL` satisfies the check but does not prune — it is a footgun. - View on top of a partitioned table inherits the requirement only if the view references the partition column in its WHERE.

What NOT to say: - "It blocks expensive queries" — it blocks unpruned queries, which is more specific.

Common follow-up: *What happens to old partitions you do not need anymore?*

Q90 · Partition expiration — automating data lifecycle

Tags: Difficulty: Easy · Asked at: Sharechat, Glance, Niveus · BQ area: Cost

The question:

> We are retaining 5 years of event data but legally only need 2 years. How do we stop paying for the rest?

The 30-second answer: Set `partition_expiration_days = 730` on the table. BigQuery will drop partitions older than that automatically. Storage cost falls accordingly.

Step-by-step thought process: 1. Each partition has its own lifecycle. Setting `partition_expiration_days` on the table applies to all partitions. 2. The expiration is measured from the partition boundary, not from row insert time. 3. Once expired, the partition is deleted in the background. Queries that touch expired partitions return zero rows for those days. 4. You can also set `table_expiration_ms` to drop the whole table at a future date — useful for temp / staging tables. 5. Combine with dataset-level default expiration so all new tables inherit the policy.

Edge cases to mention: - Expired data cannot be recovered via time-travel beyond the time-travel window (default 7 days). - Changing expiration on an existing table affects future deletions only — already-deleted partitions stay deleted.

What NOT to say: - "It moves data to cold storage" — there is no manual tier; long-term storage pricing kicks in automatically after 90 days of no edit.

Common follow-up: *What is long-term storage and how is it different from active storage?*

Q91 · Clustered tables — when, how, and the 4-column rule

Tags: Difficulty: Medium · Asked at: Swiggy, Razorpay, Dream11, Flipkart · BQ area: Performance

The question:

> When does clustering actually help, and why does BigQuery limit it to 4 columns?

The 30-second answer: Clustering sorts data within each partition by up to 4 columns, ordered by selectivity. Queries filtering on the leading clustering column get block-level pruning on top of partition pruning. Beyond 4 columns the cost outweighs the benefit.

Step-by-step thought process: 1. Clustering is declared at table creation: `CLUSTER BY user_id, country, device_type`. 2. Order of columns matters — leading columns are checked first. Filtering on `user_id` alone gets full benefit; filtering on `country` alone gets partial. 3. BigQuery groups rows into blocks. Each block stores min / max for clustering columns. Queries skip blocks whose range cannot match the filter. 4. Automatic re-clustering runs in the background — you do not pay for it, but heavy inserts may temporarily reduce effectiveness. 5. The 4-column limit reflects diminishing returns — block headers grow, and selectivity per column drops fast after the first 2-3.

Edge cases to mention: - No bytes-scanned reduction is shown in the dry-run estimate for clustering — it shows up in actual job stats only. - Clustering does not enforce uniqueness, ordering on read, or sorting in output. - High-cardinality columns (like `event_id`) make poor leading clustering columns — too many blocks, too little pruning.

What NOT to say: - "Clustering is like an index" — there is no separate index structure; the data itself is sorted.

Common follow-up: *When does it make sense to partition AND cluster?*

Q92 · Partition + cluster combo — when to use both

Tags: Difficulty: Medium · Asked at: Meesho, Razorpay, Niveus · BQ area: Performance

The question:

> Should I partition by date AND cluster by user_id, or just pick one?

The 30-second answer: Use both when queries filter on date (for partition pruning) AND on a high-selectivity column like user_id (for block pruning within the partition). The combo is the standard pattern for event tables in production.

Step-by-step thought process: 1. Partitioning prunes whole partitions. Clustering prunes blocks within a partition. 2. A query like `WHERE event_date = '2026-05-01' AND user_id = 12345` benefits from both: 1 partition scanned, only blocks containing user 12345 read. 3. Without clustering, the date partition still scans all users for that day — potentially 100 GB instead of 100 MB. 4. Partition first by the most common filter (usually date), then cluster by the next 1-4 filter columns by selectivity. 5. There is no extra storage cost for clustering, but maintenance does consume background slots.

Edge cases to mention: - If queries never filter on the clustering column, clustering does nothing and you should drop it. - Tables under ~1 GB do not benefit from clustering — block-level pruning needs enough blocks to matter.

What NOT to say: - "Pick clustering OR partitioning" — they solve different problems.

Common follow-up: *When should I NOT bother clustering at all?*

Q93 · When clustering is the wrong choice

Tags: Difficulty: Medium · Asked at: Searce, Niveus · BQ area: Schema

The question:

> Name three cases where you would not cluster a BigQuery table.

The 30-second answer: Small tables, write-heavy tables with random keys, and tables where the filter columns change every quarter. Clustering has overhead — it pays off only when there is a stable, selective filter.

Step-by-step thought process: 1. **Small tables (<1 GB)**: blocks are large relative to the table; pruning saves little. 2. **Random / write-heavy patterns**: streaming inserts land in temporary unclustered blocks. Heavy writes mean a constant lag in re-clustering benefit. 3. **Unstable query patterns**: if the team filters on a different column each quarter (e.g. dashboards changing), clustering on yesterday's hot column does not help. 4. **Already-aggregated tables**: a 10k-row daily rollup is fine without clustering. 5. **Wide string columns as leading cluster key**: dictionary size grows, pruning gets weaker.

Edge cases to mention: - Re-clustering does not affect query correctness, only performance, so a "wrong" clustering choice is rarely catastrophic — just wasteful. - You can drop and re-add clustering via CTAS into a new table.

What NOT to say: - "Clustering is always free" — automatic re-clustering uses background slots from your reservation.

Common follow-up: *How do nested and repeated columns change all of this?*

Q94 · ARRAY and STRUCT — when to use nested vs flat schema

Tags: Difficulty: Medium · Asked at: Swiggy, Flipkart, Razorpay, Glance · BQ area: Schema

The question:

> When do you nest an order's line items as ARRAY<STRUCT<...>> instead of a separate line_items table?

The 30-second answer: Nest when items are always queried with the parent, never updated independently, and you want join-free reads. Flatten when items have their own lifecycle, separate access patterns, or grow unboundedly.

Step-by-step thought process: 1. BigQuery is columnar — nested fields are stored as repeated columns, not as JSON. Reading a nested field is as cheap as reading a top-level column. 2. ARRAY<STRUCT<...>> avoids the join cost. A single scan of `orders` returns line items inline. 3. Updates are the catch — updating one line item rewrites the whole parent row (BigQuery has no row-level update for nested). 4. If line items are queried independently (e.g. "all sales of SKU-123 last month"), flat is better — column pruning on a flat `line_items` table is more selective. 5. Hybrid: nest for the OLTP-replica analytical layer; also materialise a flat fact table for SKU-level analysis.

Edge cases to mention: - Nested fields cannot be partition columns. A timestamp inside a STRUCT cannot drive partitioning. - Nested fields cannot be clustering columns either. - The 100 MB row limit applies — extremely large arrays will hit this.

What NOT to say: - "Nested is always faster" — for cross-parent analysis (SKU-level above), flat wins.

Common follow-up: *Show me an UNNEST query on this schema.*

Q95 · UNNEST patterns — flattening arrays in a query

Tags: Difficulty: Medium · Asked at: Razorpay, Meesho, Sharechat, Niveus · BQ area: Schema

The question:

> Given `orders(id, items ARRAY<STRUCT<sku STRING, qty INT64>>)`, write a query that lists every order line.

The 30-second answer: Use a CROSS JOIN with UNNEST to expand the array, then access STRUCT fields with dot notation.

Step-by-step thought process: 1. `UNNEST(array_col)` turns an array column into rows. Combined with a CROSS JOIN, each parent row is replicated per array element. 2. You can give the UNNEST a correlation alias to access struct fields cleanly. 3. To preserve parent context, include parent columns in the SELECT — they are automatically replicated. 4. Position of the element inside the array is available via `WITH OFFSET AS pos`. 5. LEFT JOIN UNNEST preserves rows whose array is empty or NULL — important for outer-style queries.

Edge cases to mention: - UNNEST(NULL) returns zero rows by default; LEFT JOIN UNNEST keeps the parent row with NULLs. - UNNEST of a scalar array (e.g. ARRAY) needs no struct accessor. - Filtering before UNNEST is cheaper than filtering after — pushdown of WHERE matters.

What NOT to say: - "I'll convert to JSON and parse" — UNNEST is the native, columnar-friendly path.

```
SELECT
  o.id AS order_id,
  item.sku,
  item.qty,
  pos
FROM `proj.ds.orders` AS o,
  UNNEST(o.items) AS item WITH OFFSET AS pos
WHERE o.order_date = DATE "2026-05-01";
```

Common follow-up: *How would you compute total quantity per order without UNNEST?*

Q96 · Aggregating inside an array without UNNEST

Tags: Difficulty: Medium · Asked at: Dream11, Swiggy, Flipkart · BQ area: Schema

The question:

> Same `orders.items` schema. Total quantity per order, in one pass.

The 30-second answer: Use a subquery on the array directly — `(SELECT SUM(item.qty) FROM UNNEST(o.items) AS item)`. This avoids the row blow-up of a CROSS JOIN UNNEST.

Step-by-step thought process: 1. BigQuery allows correlated subqueries on array columns. The subquery sees only the rows for that parent. 2. This is cheaper than CROSS JOIN UNNEST + GROUP BY parent_id because no shuffle is needed. 3. You can compute multiple aggregates in one go by selecting them as STRUCT. 4. ARRAY_LENGTH(items) gives a count without UNNEST at all. 5. EXISTS and ANY on array elements are also written as subqueries on UNNEST.

Edge cases to mention: - Subquery on an array must return exactly one column (or a STRUCT) for use in SELECT. - Empty array returns the aggregate's identity (SUM = NULL, COUNT = 0).

```
SELECT
  id,
  (SELECT SUM(item.qty) FROM UNNEST(items) AS item) AS total_qty,
  ARRAY_LENGTH(items) AS line_count
FROM `proj.ds.orders`;
```

What NOT to say: - "You must UNNEST first" — only when you want per-element rows in the final output.

Common follow-up: *How would you join two tables on an array element?*

Q97 · Joining on array elements

Tags: Difficulty: Hard · Asked at: Razorpay, Niveus, Dream11 · BQ area: Schema

The question:

> Given `orders.items` (array of SKUs) and `products(sku, name)`, list every product name purchased per order.

The 30-second answer: UNNEST the array, then JOIN the unnested SKU to products. Group back by order if you want array-shaped output.

Step-by-step thought process: 1. The array side must be flattened first — you cannot directly JOIN a scalar to an ARRAY. 2. CROSS JOIN UNNEST gives you one row per (order, item). 3. JOIN this to `products` on `sku`. 4. To reshape back into arrays, wrap in ARRAY_AGG with GROUP BY order_id. 5. For performance, filter both sides before the UNNEST — partition pruning still applies on the parent.

```
SELECT
  o.id,
  ARRAY_AGG(p.name) AS product_names
FROM `proj.ds.orders` AS o,
  UNNEST(o.items) AS item
JOIN `proj.ds.products` AS p
  ON p.sku = item.sku
WHERE o.order_date = DATE "2026-05-01"
GROUP BY o.id;
```

Edge cases to mention: - LEFT JOIN keeps orders with no matching products. - If items contain duplicates, ARRAY_AGG preserves duplicates — use ARRAY_AGG(DISTINCT ...) if not desired. - IGNORE NULLS on ARRAY_AGG drops NULLs from outer join misses.

What NOT to say: - "Use a subquery" — works for aggregates but not for joining external tables on inner elements.

Common follow-up: *How do you write procedural / loop logic in BigQuery?*

Q98 · Scripting basics — BEGIN / END, DECLARE, SET

Tags: Difficulty: Easy · Asked at: Searce, Niveus, Quantiphi · BQ area: Architecture

The question:

> I need to run three queries in sequence, passing a date variable. Show me the BigQuery script.

The 30-second answer: Wrap the statements in BEGIN ... END; declare variables with DECLARE; assign with SET. The whole script runs as one job.

Step-by-step thought process: 1. BigQuery scripting accepts multiple statements separated by `;`. The whole thing is one job in the UI. 2. `DECLARE run_date DATE DEFAULT CURRENT_DATE();` creates a session variable. 3. `SET run_date = DATE "2026-05-01";` reassigns it. 4. Variables interpolate into subsequent queries via direct reference (no `${}` syntax inside SQL). 5. BEGIN ... END blocks allow nesting, EXCEPTION WHEN ERROR THEN handlers, and IF / WHILE / LOOP control flow.

```
DECLARE run_date DATE DEFAULT CURRENT_DATE();

BEGIN
  CREATE OR REPLACE TABLE staging.daily_events AS
  SELECT * FROM raw.events WHERE event_date = run_date;

  CREATE OR REPLACE TABLE staging.daily_summary AS
  SELECT user_id, COUNT(*) AS n FROM staging.daily_events GROUP BY user_id;

  MERGE prod.user_daily t
  USING staging.daily_summary s
  ON t.user_id = s.user_id AND t.run_date = run_date
  WHEN MATCHED THEN UPDATE SET n = s.n
  WHEN NOT MATCHED THEN INSERT (user_id, run_date, n) VALUES (s.user_id, run_date, s.n);

EXCEPTION WHEN ERROR THEN
  SELECT @@error.message AS err;
  RAISE USING MESSAGE = "Daily run failed";
END;
```

Edge cases to mention: - Each statement inside the script is billed individually; the script wrapper itself is free. - DDL inside scripts holds metadata locks briefly — long DDL can block other concurrent DDL on the same table.

What NOT to say: - "Scripting is a separate language" — it is standard BigQuery SQL with extensions.

Common follow-up: *When would you use EXECUTE IMMEDIATE?*

Q99 · EXECUTE IMMEDIATE and stored procedures

Tags: Difficulty: Medium · Asked at: Niveus, Searce, Quantiphi, TCS · BQ area: Architecture

The question:

> Why and when would you wrap dynamic SQL in EXECUTE IMMEDIATE inside a stored procedure?

The 30-second answer: Use EXECUTE IMMEDIATE when the table name, column list, or WHERE clause must be built at runtime — for example, looping over a list of regional tables. Stored procs let you reuse this logic with parameters.

Step-by-step thought process: 1. EXECUTE IMMEDIATE takes a STRING containing SQL and runs it. Bind parameters via USING clause to avoid injection. 2. Create a stored procedure with `CREATE PROCEDURE proc_name(arg STRING) BEGIN ... END;`. 3. Procedures live in datasets like tables, are versioned, and can be granted to specific service accounts. 4. Typical use: a procedure that takes a date and dataset name, builds and runs a MERGE. 5. Stored procs run server-side — no roundtrip per statement, important for large scripts.

```
CREATE OR REPLACE PROCEDURE ops.rebuild_summary(target_date DATE)
BEGIN
  DECLARE sql_text STRING;
  SET sql_text = FORMAT("""
    CREATE OR REPLACE TABLE summary.daily_%s AS
    SELECT user_id, COUNT(*) AS n
    FROM raw.events
    WHERE event_date = DATE '%t'
    GROUP BY user_id
  """, FORMAT_DATE("%Y%m%d", target_date), target_date);
  EXECUTE IMMEDIATE sql_text;
END;

CALL ops.rebuild_summary(DATE "2026-05-01");
```

Edge cases to mention: - USING clause prevents SQL injection in dynamic strings. - Procedures can OUT parameters or return values via temp tables. - EXECUTE IMMEDIATE bills each statement separately.

What NOT to say: - "It is like a UDF" — UDFs return values per row, procedures orchestrate statements.

Common follow-up: *How do you schedule the procedure to run daily?*

Q100 · Scheduled queries — setup, ownership, monitoring

Tags: Difficulty: Easy · Asked at: Niveus, Searce, Quantiphi · BQ area: Architecture

The question:

> Walk me through scheduling that `rebuild_summary` procedure daily at 2 am IST.

The 30-second answer: Create a Scheduled Query in the BigQuery console or via the API, pick a cron-style schedule, and choose a service account to own it. Logs land in `INFORMATION_SCHEMA.SCHEDULED_QUERIES`.

Step-by-step thought process: 1. Open the BigQuery console -> Scheduled queries -> create. Or use `bq mk --transfer_config`. 2. The query body is a normal SQL statement — `CALL ops.rebuild_summary(@run_date)` works. 3. Built-in parameters: `@run_date`, `@run_time` — BigQuery passes these in at execution. 4. Ownership matters — by default the schedule runs as the user who created it. Switch to a dedicated service account for production so it survives the creator leaving the org. 5. Failures generate Pub/Sub notifications if configured; you can also query `region-asia-south1.INFORMATION_SCHEMA.SCHEDULED_QUERY_RUNS` for history.

Edge cases to mention: - Schedules are region-locked — the schedule lives in the same region as the destination dataset. - DST changes can shift cron times — pick a fixed UTC offset. - Concurrent runs of the same schedule are not deduped — long-running jobs can overlap.

What NOT to say: - "It is like cron" — closer in spirit to Cloud Scheduler with BigQuery integration.

Common follow-up: *Why would I use Dataform or Airflow instead?*

Q101 · Scheduled queries vs Dataform vs Airflow / Cloud Composer

Tags: Difficulty: Medium · Asked at: Razorpay, Meesho, Niveus, Searce · BQ area: Architecture

The question:

> When is a Scheduled Query enough, and when do I move to Dataform or Airflow?

The 30-second answer: Scheduled Queries for single SQL jobs with no dependencies. Dataform for DAGs of SQL transformations with testing. Airflow / Composer when you need cross-system orchestration (BigQuery + GCS + Dataflow + APIs).

Step-by-step thought process: 1. Scheduled Queries: zero infra, no DAG concept, no testing framework. One SQL, one cron. 2. Dataform: BigQuery-native SQLX with dependency graphs, assertions (tests), incremental models. Free; lives inside BigQuery. 3. Airflow / Composer: full Python DAGs, hundreds of operators, cross-service orchestration. Operational overhead — you manage a cluster. 4. Pick by complexity: 1 query -> Scheduled. 10-100 dependent queries -> Dataform. Multi-service pipeline -> Airflow. 5. Mix is common: Airflow triggers Dataform runs; Dataform manages the SQL graph; Scheduled Queries handle simple BI refreshes.

Edge cases to mention: - Dataform was acquired by Google and is now first-class — no longer a separate product to procure. - Composer is managed Airflow but you pay for the GKE cluster behind it — not cheap for small teams.

What NOT to say: - "Scheduled queries are deprecated" — they are not.

Common follow-up: *How do you check the cost of a specific scheduled query last month?*

Q102 · INFORMATION_SCHEMA.JOBS — querying your own cost history

Tags: Difficulty: Medium · Asked at: Razorpay, Meesho, Niveus, Searce · BQ area: Cost

The question:

> Show me how to find the top 10 most expensive queries last month.

The 30-second answer: Query `region-<your-region>.INFORMATION_SCHEMA.JOBS_BY_PROJECT` with a filter on `creation_time`, sort by `total_bytes_billed DESC`, and join to user / job_id for context.

Step-by-step thought process: 1. Each region has its own `INFORMATION_SCHEMA.JOBS_BY_PROJECT`, `JOBS_BY_USER`, `JOBS_BY_FOLDER`, `JOBS_BY_ORGANIZATION` views. 2. Key columns: `job_id`, `user_email`, `query`, `total_bytes_billed`, `total_slot_ms`, `creation_time`, `end_time`. 3. `total_bytes_billed` is the on-demand cost driver. `total_slot_ms` matters for reservation efficiency. 4. Round to days for trend dashboards; group by `user_email` to find top spenders. 5. Set a 30-day retention reminder — JOBS history is kept 180 days.

```
SELECT
  job_id,
  user_email,
  ROUND(total_bytes_billed / POW(1024, 4), 3) AS tib_billed,
  ROUND(total_bytes_billed / POW(1024, 4) * 6.25, 2) AS approx_usd,
  total_slot_ms,
  query
FROM `region-asia-south1`.INFORMATION_SCHEMA.JOBS_BY_PROJECT
WHERE creation_time BETWEEN TIMESTAMP "2026-04-01" AND TIMESTAMP "2026-05-01"
  AND statement_type = "SELECT"
ORDER BY total_bytes_billed DESC
LIMIT 10;
```

Edge cases to mention: - INFORMATION_SCHEMA queries scan 0 bytes — free to run. - Cross-region queries need explicit region prefix in the table name. - For org-wide view, you need the BigQuery Resource Viewer role at the org level.

What NOT to say: - "Use the billing export" — that is for ledger / invoice reconciliation. JOBS is for job-level forensics.

Common follow-up: *Bytes scanned vs slot-ms — when do I care about which?*

Q103 · Bytes scanned vs slot-ms — what each tells you

Tags: Difficulty: Medium · Asked at: Dream11, Razorpay, Quantiphi · BQ area: Cost

The question:

> A query scans 10 GB and uses 500 slot-seconds. Another scans 100 GB and uses 50 slot-seconds. Which matters more?

The 30-second answer: On on-demand, bytes scanned drives your bill; ignore slot-ms. On reservations, slot-ms tells you whether the query is starving your fixed pool; bytes are mostly cosmetic.

Step-by-step thought process: 1. Bytes scanned (`total_bytes_billed`) is the on-demand cost — flat ₹/TB scanned. 2. Slot-ms (`total_slot_ms`) is compute time — relevant only when slot capacity is finite (reservations). 3. A query with low bytes but high slot-ms usually has heavy aggregation, joins, or window functions on small data. 4. A query with high bytes but low slot-ms is a wide-shallow scan with no shuffle — typical `SELECT *` over a partition. 5. Optimisation playbook differs: on-demand -> prune bytes (partition, column, filter). Reservation -> tune algorithm (joins, window, aggregation).

Edge cases to mention: - Reservation projects still report bytes scanned in JOBS — useful for capacity planning even if not billed. - Slot-ms includes time across multiple workers — divide by your slot count for wall-clock estimate.

What NOT to say: - "Slot-ms is the cost" — only on flat-rate / autoscale, and only relative to your commitment.

Common follow-up: *Practical ways to reduce bytes scanned right now?*

Q104 · Partition pruning, column pruning, and the cost playbook

Tags: Difficulty: Easy · Asked at: Meesho, Sharechat, Niveus · BQ area: Cost

The question:

> Give me your top 5 levers to cut a query's bytes scanned.

The 30-second answer: 1. SELECT only needed columns. 2. Add a partition-column filter. 3. Cluster on the next most-selective filter. 4. Use approximate functions where exact is not needed. 5. Materialise expensive joins as MVs.

Step-by-step thought process: 1. **Column pruning**: never `SELECT *`. List the columns. Each unread column is bytes saved. 2. **Partition filter**: `WHERE event_date = ...` cuts the table to one partition's worth of bytes. 3. **Clustering**: filtering on the leading cluster column skips blocks within the partition. 4. **Approximate functions**: `APPROX_COUNT_DISTINCT`, `APPROX_QUANTILES` use HLL sketches — same order of magnitude bytes but 10-100x faster, and helps stay within slot budgets. 5. **Materialised views**: pre-aggregate; queries against the MV scan tiny bytes.

Edge cases to mention: - `WHERE date_col > CURRENT_DATE - 7` works for partition pruning if `date_col` is the partition column. - Function on the partition column can defeat pruning: `WHERE DATE(timestamp_col) = ...` may not prune on a TIMESTAMP partition. - `LIMIT` does NOT reduce bytes scanned — it filters after the scan.

What NOT to say: - "Add more slots to make it cheaper" — slots speed it up, not cheapen it on on-demand.

Common follow-up: *Tell me about materialised views.*

Q105 · Materialised views — refresh, supported aggregates, smart tuning

Tags: Difficulty: Medium · Asked at: Razorpay, Niveus, Searce, Quantiphi · BQ area: Performance

The question:

> When is a materialised view better than a regular view, and when is it worse?

The 30-second answer: MV when the same expensive aggregate is read repeatedly and the base table updates slowly relative to read frequency. Worse than a view when the base table churns constantly — refresh cost can dominate.

Step-by-step thought process: 1. A materialised view stores precomputed results and refreshes incrementally as the base table changes. 2. Refresh is automatic — you do not schedule it. It runs in the background using your reservation slots. 3. Supported aggregates: SUM, COUNT, MIN, MAX, AVG, COUNT(DISTINCT) (approximate), HLL. No window functions, no joins of more than a couple of tables historically — check current limits. 4. **Smart tuning**: BigQuery automatically rewrites queries against the base table to use the MV when applicable. The user does not have to change SQL. 5. Trade-off: MV refresh consumes slots and storage. If reads are rare, the maths does not work.

Edge cases to mention: - MVs cannot reference UDFs, scripts, or non-deterministic functions. - Manual refresh: `CALL BQ.REFRESH_MATERIALIZED_VIEW(...)` if you want to force one. - Smart tuning can be disabled per MV if you want strict referencing.

What NOT to say: - "MV is just a cached view" — caching is per-user, per-24h, expires on base change. MVs persist and incrementally refresh.

Common follow-up: *Where does BI Engine fit in?*

Q106 · BI Engine — in-memory acceleration for dashboards

Tags: Difficulty: Easy · Asked at: Niveus, Searce, Razorpay · BQ area: Performance

The question:

> When does enabling BI Engine actually move the needle?

The 30-second answer: When dashboards re-run the same aggregate queries dozens of times per hour. BI Engine caches column data in RAM per region — sub-second response for the cached tables, lower slot usage on the cluster.

Step-by-step thought process: 1. Allocate a BI Engine reservation in GB per region. RAM caches the columns of tables you frequently query. 2. Queries that fit the cache are answered from RAM in <1 second; bytes-billed are reduced. 3. Works transparently with BigQuery SQL — no rewrite. Looker / Looker Studio / Power BI dashboards benefit automatically. 4. Best fit: BI dashboards with concentrated time-range queries on a small set of partitioned tables. 5. Cost: BI Engine slots are billed per GB-hour. Worth it when dashboard concurrency is high.

Edge cases to mention: - Some SQL constructs fall back to standard BigQuery (e.g. certain UDFs, certain joins) — partial acceleration is normal. - Cache eviction is LRU; cold tables get pushed out. - Per-region — multi-region datasets need BI Engine in both regions if you want acceleration in both.

What NOT to say: - "BI Engine is for ELT" — it is read-acceleration, not transformation.

Common follow-up: *Can BigQuery train ML models too?*

Q107 · BigQuery ML — CREATE MODEL basics and where it fits

Tags: Difficulty: Easy · Asked at: Swiggy, Meesho, Quantiphi, Dream11 · BQ area: Architecture

The question:

> Walk me through training a logistic regression for churn prediction without leaving BigQuery.

The 30-second answer: Use `CREATE MODEL` with `model_type = 'logistic_reg'`, point it at a SELECT with your features and label, then `ML.PREDICT` for inference. No external ML stack needed for the interview-context use cases.

Step-by-step thought process: 1. `CREATE OR REPLACE MODEL ds.churn_model OPTIONS(model_type='logistic_reg', input_label_cols=['churned']) AS SELECT ... FROM training_set;` 2. BigQuery runs training as a regular SQL job — uses slots from your reservation / on-demand. 3. Model lives in a dataset like a table. Inspect with `ML.EVALUATE`, `ML.WEIGHTS`, `ML.FEATURE_INFO`. 4. Inference: `SELECT * FROM ML.PREDICT(MODEL ds.churn_model, (SELECT * FROM new_users))`. 5. Supported models: linear / logistic regression, K-means, ARIMA+ (forecasting), boosted trees, deep neural nets, matrix factorisation, plus Vertex AI integration for hosted models.

Edge cases to mention: - Training cost is bytes scanned + slot-ms; large training jobs can be expensive. - For serious ML, BigQuery ML is a first-cut; Vertex AI is the production platform. - Model versions are not auto-managed — you handle naming.

What NOT to say: - "BigQuery ML competes with TensorFlow" — it complements it. For interview context, mention BQML for SQL-friendly inference and Vertex AI for heavy lifting.

Common follow-up: *How do you query data that lives outside BigQuery?*

Q108 · BigLake, federated queries, and external tables

Tags: Difficulty: Medium · Asked at: Searce, Niveus, Quantiphi, TCS · BQ area: Architecture

The question:

> Data is in GCS Parquet, in Cloud Spanner, and in Cloud SQL Postgres. Can I query all three from BigQuery?

The 30-second answer: Yes. External tables for GCS files (Parquet / CSV / JSON), federated queries for Cloud SQL / Spanner via EXTERNAL_QUERY, and BigLake for governance on GCS with row / column security.

Step-by-step thought process: 1. **External tables on GCS**: `CREATE EXTERNAL TABLE` over a URI pattern. Data stays in GCS; BigQuery reads on query. 2. **BigLake**: external tables with BigQuery-grade access control (row / column policies, fine-grained ACLs). Lets you treat a data lake like a warehouse. 3. **Federated queries**: `SELECT * FROM EXTERNAL_QUERY("connection_id", "SELECT ...")` pushes a query to Cloud SQL or Spanner; results stream back. 4. Performance trade-off: federated queries do not use BigQuery's columnar storage — you pay for slot time AND the source database's compute. 5. Best practice: use external / federated for occasional joins; for repeated heavy use, ELT into native BigQuery tables.

Edge cases to mention: - External Parquet supports partition discovery via Hive-style paths. - Federated queries pay egress if the source is in a different region. - BigLake tables can also span Iceberg / Delta formats in 2025.

What NOT to say: - "Use Dataflow first" — federated queries are simpler when ad-hoc.

Common follow-up: *What does the GA4 export schema look like in BigQuery?*

Q109 · GA4 export schema in BigQuery — quick orientation

Tags: Difficulty: Easy · Asked at: Swiggy, Meesho, Sharechat, Glance · BQ area: Schema

The question:

> GA4 dumps to BigQuery daily. What are the key tables and how is the schema shaped?

The 30-second answer: GA4 creates daily-sharded `events_YYYYMMDD` tables (and a streaming `events_intraday_*`) in your linked dataset. Each row is one event, with deeply nested `event_params`, `user_properties`, and `items` arrays.

Step-by-step thought process: 1. Schema: `event_date`, `event_timestamp`, `event_name`, `user_pseudo_id`, plus `event_params ARRAY<STRUCT<key STRING, value STRUCT<string_value, int_value, double_value, float_value>>>`. 2. Daily tables:

`events_20260525`, `events_20260524`, etc. Use wildcard: `FROM dataset.events_*` with `_TABLE_SUFFIX BETWEEN ...` for partition-like pruning. 3. Streaming buffer: `events_intraday_YYYYMMDD` holds today's events until daily export consolidates them. 4. To extract a parameter value, UNNEST event_params and filter on key. 5. The schema is unstable across GA4 changes — write resilient queries that handle NULL value variants.

```
SELECT
  event_date,
  event_name,
  (SELECT value.string_value
   FROM UNNEST(event_params)
   WHERE key = "page_location") AS page_url,
  user_pseudo_id
FROM `analytics_123456789.events_*`
WHERE _TABLE_SUFFIX BETWEEN "20260501" AND "20260525"
  AND event_name = "page_view";
```

Edge cases to mention: - Wildcards prune on `_TABLE_SUFFIX` filter — without it, you scan all days. - Intraday and daily tables can briefly overlap during the daily export window. - GA4 BigQuery export is free up to 1M events/day on standard properties.

What NOT to say: - "It is the same as Universal Analytics export" — UA was sessions-shaped; GA4 is events-shaped.

Common follow-up: *How does cross-region work for datasets and queries?*

Q110 · Dataset locations — regional, multi-region, and copy costs

Tags: Difficulty: Medium · Asked at: Niveus, Searce, Razorpay · BQ area: Architecture

The question:

> My team in Bengaluru wants `asia-south1`. Marketing in the US wants `US` multi-region. How do I share data without paying double?

The 30-second answer: A dataset has a fixed location set at creation. Queries cannot cross locations. To share, you either copy datasets (with egress cost) or use Analytics Hub / Cross-region replication for read-only access.

Step-by-step thought process: 1. Dataset location is regional (e.g. `asia-south1`) or multi-region (e.g. `US`, `EU`). Set at creation, immutable after. 2. All tables in a dataset share the location. All jobs run in that location. 3. Cross-region copy: `bq cp --location` copies tables across regions. Egress is billed as cross-region network transfer. 4. **Cross-region dataset replication**: BigQuery can maintain a

secondary read-only copy of a dataset in another region. Pricing covers replication egress + storage in both regions. 5. **Analytics Hub**: publish a listing that lets subscribers in the same region query the data without copying. Cross-region requires replication first.

Edge cases to mention: - Multi-region storage is more expensive per GB than single region, but spans multiple zones for higher durability. - Some features (e.g. specific ML model types) are region-restricted — check before designing. - A query referencing two datasets must have them in the same location, or you must replicate one.

What NOT to say: - "Just use the global region" — there is no global. `US` and `EU` are multi-region but not global.

Common follow-up: *If I move from `US` to `asia-south1` to cut latency, what is the migration shape?*

End of Section 3 — BigQuery

You now have 30 BigQuery-specific questions covering the architecture-to-cost arc that interviewers walk down. Pair this with Section 1 (Snowflake) and Section 2 (Databricks) of File 06 to round out your modern-warehouse coverage. The next file (`07-cheatsheet.md`) gives you the one-page printable summary for the morning of the interview.

Sections 4 + 5 — MySQL 8.x + SQL Server 2022

This file covers engine-specific behaviour for the two RDBMS systems you are most likely to be quizzed on in Indian backend / data-role interviews: **MySQL 8.x** (dominant in product startups, e-commerce, fintech consumer apps) and **SQL Server 2022** (dominant in GCCs of US BFSI / retail — JP Morgan, Goldman, Wells Fargo, Morgan Stanley, Walmart Labs, Target, AmEx).

The questions assume you already know cross-engine fundamentals (joins, normalisation, indexes, transactions, isolation levels). What is tested here is the **release-specific** stuff — what landed in MySQL 8.0 (the line every interviewer cares about) versus what is new in SQL Server 2022 versus what was always there but interviewers love to probe.

Part A — MySQL 8.x (25 Qs)

Version assumptions: MySQL 8.0 GA (April 2018) and the 8.4 LTS line. We treat InnoDB as the default storage engine throughout. Where a feature landed in a specific dot release (hash join in 8.0.18, for example) we call it out explicitly.

Q111 · Window functions in MySQL 8.0

Tags: Difficulty: Easy · Asked at: Flipkart, Swiggy, Razorpay, Zomato · Engine: MySQL 8 · Topic: Window functions

The question:

> MySQL was the last major RDBMS to get window functions. When did they land, and what is the basic syntax?

The 30-second answer: Window functions arrived in MySQL 8.0 (April 2018). Before that you faked them with self-joins or user variables. Syntax is standard SQL: `function() OVER (PARTITION BY ... ORDER BY ... ROWS/RANGE ...)`.

Step-by-step thought process: 1. Before 8.0, the "rank within group" pattern in MySQL was the infamous `SET @rank := IF(@prev = col, @rank+1, 1)` hack — error-prone, order-dependent, and broken under modern optimizer hints. 2. 8.0 added all the SQL standard window functions in one shot: `ROW_NUMBER`, `RANK`, `DENSE_RANK`, `LAG`, `LEAD`, `FIRST_VALUE`, `LAST_VALUE`, `NTH_VALUE`, `NTILE`, plus aggregate-as-window (`SUM() OVER`, etc.). 3. The `OVER` clause has three parts: `PARTITION BY` (resets the window per group), `ORDER BY` (orders rows within the partition), and an optional frame clause (`ROWS BETWEEN ... AND ...`). 4. Named windows: `WINDOW w AS (PARTITION BY x ORDER BY y)` then `SUM(amt) OVER w` — useful when many functions share the same window.

Edge cases to mention: - Aggregate window functions without `ORDER BY` default to the entire partition, not a running total. Add `ORDER BY` to get running behaviour. - `RANK` and `DENSE_RANK` produce ties; `ROW_NUMBER` does not. Choosing between them is a common interview trap. - The default frame for ordered windows is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` — which includes ties at the current row. Use `ROWS` for strict positional framing.

What NOT to say: - Do not claim window functions are "slow" in MySQL 8 — modern optimizer handles them well, often better than the old user-variable hacks. - Do not say they replace `GROUP BY`. They are complementary: `GROUP BY` collapses rows, window functions enrich rows.

Common follow-up: *Why was MySQL so late to ship window functions when Postgres had them since 8.4 (2009)?*

```
SELECT
  order_id, customer_id, amount,
  ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) AS recency_rank,
  SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_date
                    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total
FROM orders;
```

Q112 · Top-N-per-group with window functions

Tags: Difficulty: Medium · Asked at: Flipkart, Meesho, Paytm, Cred · Engine: MySQL 8 · Topic: Window functions

The question:

> Give me the top 3 highest-spending orders per customer. Show both the pre-8.0 and post-8.0 approach.

The 30-second answer: Post-8.0: use `ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY amount DESC)` in a CTE, filter outer query for `rn <= 3`. Pre-8.0 you needed correlated subqueries or session variables — both ugly and slow at scale.

Step-by-step thought process: 1. The clean modern pattern is a CTE that assigns `ROW_NUMBER()` per partition, ordered by the ranking column descending. 2. The outer query filters `WHERE rn <= 3`. You cannot filter on a window function in the same `SELECT` — windows are computed after `WHERE` but before `SELECT`, so you must wrap in a derived table or CTE. 3. Choose `ROW_NUMBER` if you want exactly 3 rows even with ties. Use `RANK` or `DENSE_RANK` if ties at position 3 should all be included. 4. For very large tables, ensure the partition column is indexed and the order column is part of the same composite index for best performance.

Edge cases to mention: - Ties at boundary: customer with two orders both at ₹5000 — `ROW_NUMBER` arbitrarily picks one as "3rd", which is non-deterministic across runs. Add a tiebreaker column (`order_id`, `created_at`) to make it stable. - `LIMIT` does not work for top-N-per-group — `LIMIT` is global, not per partition. This is a classic beginner mistake. - For exactly N=1 (the "latest record per group" pattern) you can sometimes beat window functions with a `LEFT JOIN ... WHERE other.id IS NULL` self-anti-join, but readability suffers.

What NOT to say: - Do not recommend the user-variable approach in 2026 — it is undefined behaviour in modern MySQL and the optimizer may evaluate the SELECT list in any order.

Common follow-up: *What index would you build to make this query fast on a 100M-row orders table?*

```
WITH ranked AS (
  SELECT order_id, customer_id, amount,
         ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY amount DESC, order_id) AS rn
  FROM orders
)
SELECT * FROM ranked WHERE rn <= 3;
```

Q113 · LAG, LEAD, and gaps-and-islands

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Walmart Labs, Cred · Engine: MySQL 8 · Topic: Window functions

The question:

> A user has a series of login timestamps. Find streaks of consecutive daily logins. How do you approach this with window functions?

The 30-second answer: This is the classic **gaps-and-islands** problem. Compute `LAG(login_date) OVER (PARTITION BY user_id ORDER BY login_date)`. Flag rows where the gap is not 1 day, then take a running sum of the flag to assign a "streak ID", then group by user + streak_id.

Step-by-step thought process: 1. First CTE: compute `prev_date = LAG(login_date)` per user. 2. Second step: a row starts a new streak if `prev_date IS NULL` or `DATEDIFF(login_date, prev_date) > 1`. Mark this with `is_new_streak = 1`. 3. Third step: `SUM(is_new_streak) OVER (PARTITION BY user_id ORDER BY login_date)` gives each row a streak ID. Rows in the same streak share an ID. 4. Final aggregation: `GROUP BY user_id, streak_id`, take `MIN(login_date)` and `MAX(login_date)` and `COUNT(*)` as streak length.

Edge cases to mention: - Same-day duplicates — if a user logs in twice on the same date, `DATEDIFF = 0` which is not `> 1`, so it does not break the streak. Usually you `DISTINCT login_date` first. - Time zones — daily streaks must use a consistent timezone. Indian apps often store UTC; convert to IST before extracting the date. - Leap day, DST: in India there is no DST, so this is mostly a concern for global products.

What NOT to say: - Do not try to solve this with a self-join on `t1.date = t2.date + 1` — it works for small data but is `O(n²)` and falls over at scale.

Common follow-up: *How would you compute the longest streak ever achieved by each user?*

```
WITH gapped AS (
  SELECT user_id, login_date,
         LAG(login_date) OVER (PARTITION BY user_id ORDER BY login_date) AS prev_date
  FROM (SELECT DISTINCT user_id, login_date FROM logins) d
),
flagged AS (
  SELECT user_id, login_date,
         CASE WHEN prev_date IS NULL OR DATEDIFF(login_date, prev_date) > 1 THEN 1 ELSE 0 END AS n
  FROM gapped
),
streaks AS (
  SELECT user_id, login_date,
         SUM(new_streak) OVER (PARTITION BY user_id ORDER BY login_date) AS streak_id
  FROM flagged
)
SELECT user_id, streak_id, MIN(login_date) AS start_d, MAX(login_date) AS end_d, COUNT(*) AS days
FROM streaks
GROUP BY user_id, streak_id;
```

Q114 · CTEs vs derived tables in MySQL 8

Tags: Difficulty: Easy · Asked at: Swiggy, Zomato, Flipkart, BharatPe · Engine: MySQL 8 · Topic: CTEs

The question:

> What is the difference between a CTE and a derived table in MySQL 8? Is one faster than the other?

The 30-second answer: CTEs (`WITH` clause) arrived in 8.0. Functionally identical to derived tables in most cases — the optimizer can inline a CTE into the outer query. CTEs win on readability and enable recursion. Performance is essentially the same.

Step-by-step thought process: 1. A derived table is `SELECT ... FROM (SELECT ... ) AS sub` — an inline subquery in the FROM clause. 2. A CTE is `WITH sub AS (SELECT ...) SELECT ... FROM sub` — same semantics, declared up front. 3. MySQL 8 treats non-recursive CTEs as derived tables internally. They get **merged** into the outer query if the optimizer thinks merging is cheaper, otherwise **materialized** as a temp table. 4. Recursive CTEs (`WITH RECURSIVE`) have no derived-table equivalent — they are a genuine new capability.

Edge cases to mention: - CTEs referenced multiple times: MySQL may materialize once and reuse, or inline twice. Check `EXPLAIN` to be sure. Materializing helps when the CTE is expensive; inlining helps when it is cheap and the outer query can push down predicates. - CTEs cannot be `UPDATE` d or `DELETE` d the same way some other engines allow. - You can reference earlier CTEs from later ones in the same `WITH` chain, but not the other way round.

What NOT to say: - Do not claim CTEs are an "optimization fence" in MySQL — they are not, unlike older Postgres versions before 12. - Do not say CTEs are always slower because they "materialize" — that is only sometimes true and depends on the plan.

Common follow-up: *When would you intentionally want a CTE to materialize rather than inline?*

```
WITH recent_orders AS (
  SELECT * FROM orders WHERE order_date >= CURRENT_DATE - INTERVAL 30 DAY
)
SELECT customer_id, COUNT(*) AS recent_count
FROM recent_orders
GROUP BY customer_id;
```

Q115 · Recursive CTEs — organisation hierarchy

Tags: Difficulty: Medium · Asked at: Walmart Labs, JP Morgan, Goldman Sachs, Razorpay · Engine: MySQL 8 · Topic: CTEs

The question:

> You have an `employees` table with `id`, `name`, `manager_id`. Print every employee along with their full reporting chain up to the CEO.

The 30-second answer: Use a recursive CTE. Anchor member selects the CEO (`manager_id IS NULL`). Recursive member joins `employees` to the CTE on `e.manager_id = cte.id`, accumulating depth and path. Final select pulls from the CTE.

Step-by-step thought process: 1. Anchor: `SELECT id, name, 0 AS depth, CAST(name AS CHAR(1000)) AS path FROM employees WHERE manager_id IS NULL`. 2. Recursive: `SELECT e.id, e.name, c.depth+1, CONCAT(c.path, ' > ', e.name) FROM employees e JOIN cte c ON e.manager_id = c.id`. 3. Combine with `UNION ALL` (not `UNION` — `UNION` removes duplicates and is much slower). 4. MySQL has a default recursion limit set by `cte_max_recursion_depth` (default 1000). For deeper trees you set it explicitly with `SET cte_max_recursion_depth = 10000;`.

Edge cases to mention: - Cycles: if data has `A reports to B, B reports to A`, the recursion will loop until the limit hits. Detect cycles by tracking visited IDs in the path string and checking `FIND_IN_SET`. - Type widening: the `CAST(name AS CHAR(1000))` is critical. If you omit it, the path column inherits the type of the anchor (e.g. `VARCHAR(50)`) and recursive concatenation will silently truncate. - Performance: index `manager_id` for the join, otherwise each recursion level does a full scan.

What NOT to say: - Do not solve this with application-level recursion in a loop — it is N round-trips and embarrassingly slow.

Common follow-up: *How would you find all direct + indirect reports of a given manager?*

```
WITH RECURSIVE chain AS (
  SELECT id, name, manager_id, 0 AS depth, CAST(name AS CHAR(1000)) AS path
  FROM employees WHERE manager_id IS NULL
  UNION ALL
  SELECT e.id, e.name, e.manager_id, c.depth + 1, CONCAT(c.path, ' > ', e.name)
  FROM employees e JOIN chain c ON e.manager_id = c.id
)
SELECT id, name, depth, path FROM chain ORDER BY path;
```

Q116 · JSON datatype basics

Tags: Difficulty: Easy · Asked at: Swiggy, Zomato, Cred, Razorpay · Engine: MySQL 8 · Topic: JSON

The question:

> What is the JSON column type in MySQL? How does it differ from storing JSON in a `TEXT` column?

The 30-second answer: The JSON type (added in 5.7, refined in 8.0) stores JSON in an internal binary format. Validates on insert, supports efficient access to sub-elements via `JSON_EXTRACT` and the `->` operator, and unlike `TEXT` cannot store malformed JSON.

Step-by-step thought process: 1. With `TEXT`, the database is dumb to the content — you can store `"{"name": "Aman"` } (truncated, invalid) and only find out when parsing in the application. 2. With `JSON`, the engine validates syntax at write time. Invalid input raises an error. 3. Internal storage is a parsed binary representation, so reading individual keys does not require re-parsing the whole document each time. 4. JSON columns can be indexed indirectly via generated columns (more on this in a later question) — but not directly with a normal B-tree.

Edge cases to mention: - JSON columns cannot have a default value in the traditional sense before 8.0.13. From 8.0.13 onwards you can use `DEFAULT (JSON_OBJECT(...))` with the expression default syntax. - Storage overhead: binary JSON is roughly the same size as the textual form, sometimes a bit larger due to inline length prefixes. - Whitespace and key order are not preserved — MySQL re-serializes. If you need byte-for-byte fidelity, use `TEXT`.

What NOT to say: - Do not say JSON columns are "free-form NoSQL inside MySQL" without qualification — they are slower to write than scalar columns and you lose strong typing.

Common follow-up: *When should you use JSON columns vs a properly normalised relational design?*

Q117 · JSON_EXTRACT and the -> / ->> operators

Tags: Difficulty: Easy · Asked at: Flipkart, Meesho, Razorpay, PhonePe · Engine: MySQL 8 · Topic: JSON

The question:

> How do you pull a specific value out of a JSON column? What is the difference between `->` and `->>`?

The 30-second answer: `JSON_EXTRACT(col, '$.path')` returns a JSON value. The `col -> '$.path'` operator is shorthand for the same. The `->>` operator is shorthand for `JSON_UNQUOTE(JSON_EXTRACT(...))` — strips the surrounding quotes for string values.

Step-by-step thought process: 1. `JSON_EXTRACT(profile, '$.address.city')` returns `"Bengaluru"` — with quotes, as a JSON string. 2. `profile -> '$.address.city'` is equivalent shorthand. 3. `profile ->> '$.address.city'` returns `Bengaluru` — no quotes, ready to compare with a `VARCHAR` directly. 4. The `$` is the root document. Path expressions use dot for object keys, `[n]` for array indexes, and `*` as a wildcard (e.g. `$.items[*].price`).

Edge cases to mention: - Missing key returns `NULL`, not an error. Useful for sparse documents but easy to miss in tests. - Comparing a JSON-extracted value with `=`: `WHERE profile->'$.city' = 'Bengaluru'` will fail to match because the LHS is `"Bengaluru"` (with quotes). Use `->>` to get the unquoted string. - Type coercion: if you extract a number with `->>` you get a string, not a number. Cast explicitly when comparing.

What NOT to say: - Do not say JSON path expressions support filter conditions like XPath — MySQL 8 supports a limited subset including range and equality filters but not full XPath.

Common follow-up: *How would you index a value inside a JSON column?*

```
SELECT id, profile->>'$.name' AS name, profile->>'$.address.city' AS city
FROM users
WHERE profile->>'$.address.city' = 'Bengaluru';
```

Q118 · JSON_TABLE for shredding JSON into rows

Tags: Difficulty: Hard · Asked at: Walmart Labs, Cred, Razorpay, Zomato · Engine: MySQL 8 · Topic: JSON

The question:

> You have a JSON column storing an array of order line items. How do you query this as if it were a relational table?

The 30-second answer: Use `JSON_TABLE` (added in 8.0). It takes a JSON document and a path expression, and returns a table where each match becomes a row with columns extracted via `COLUMNS (...)` clause.

Step-by-step thought process: 1. Basic form: `JSON_TABLE(orders.items, '$[*]' COLUMNS (sku VARCHAR(50) PATH '$.sku', qty INT PATH '$.qty', price DECIMAL(10,2) PATH '$.price')) AS items`. 2. The path `$[*]` walks every element of the array. Each element produces one row. 3. Each `COLUMNS` clause defines a relational column with its type and the path within the array element. 4. Join `JSON_TABLE` to the parent table using `CROSS JOIN` or `JOIN` with `ON TRUE`. The parent row "fans out" into multiple rows, one per array element.

Edge cases to mention: - `NESTED PATH` for arrays-inside-arrays (e.g. line items each with multiple discounts). - `ON ERROR` and `ON EMPTY` clauses control behaviour when extraction fails: `NULL ON ERROR`, `DEFAULT 'x' ON EMPTY`, `ERROR ON ERROR`. - `FOR ORDINALITY` adds an auto-incremented row number — useful when the array position is meaningful. - Performance: `JSON_TABLE` runs once per outer row. For huge JSON or huge outer tables, this can be expensive.

What NOT to say: - Do not write a stored procedure that loops through the array — JSON_TABLE is set-based and dramatically faster.

Common follow-up: *What does NESTED PATH solve that the basic form cannot?*

```
SELECT o.order_id, j.sku, j.qty, j.price
FROM orders o,
     JSON_TABLE(o.items, '$[*]'
       COLUMNS (
         sku VARCHAR(50) PATH '$.sku',
         qty INT PATH '$.qty',
         price DECIMAL(10,2) PATH '$.price'
       )) AS j
WHERE o.status = 'PAID';
```

Q119 · Generated columns indexing JSON

Tags: Difficulty: Medium · Asked at: Cred, Razorpay, Meesho, PhonePe · Engine: MySQL 8 · Topic: JSON

The question:

> You frequently query users where `profile->>'$.city' = 'Bengaluru'`. How do you index that without restructuring the table?

The 30-second answer: Create a generated column that extracts the city from the JSON, then index that generated column. The optimizer can use the index when you query either the generated column directly or via the same JSON expression.

Step-by-step thought process: 1. `ALTER TABLE users ADD COLUMN city VARCHAR(100) AS (profile->>'$.city') STORED;` — `STORED` means materialized on disk; `VIRTUAL` means computed at read time. Stored is required for some index types but eats space. 2. `CREATE INDEX idx_city ON users(city);` — now a normal B-tree index exists. 3. Query plans: `WHERE city = 'Bengaluru'` uses the index trivially. `WHERE profile->>'$.city' = 'Bengaluru'` may also use the index thanks to **functional index** support — MySQL recognises the expression matches the generated column. 4. From 8.0.13 you can skip the generated column step and create a **functional index** directly: `CREATE INDEX idx_city ON users ((CAST(profile->>'$.city' AS CHAR(100))));`

Edge cases to mention: - `STORED` vs `VIRTUAL`: `VIRTUAL` saves disk but recomputes on every read. For indexed access on a frequently-queried column, `STORED` is usually better. From 8.0.18 `VIRTUAL` columns can also be indexed cleanly. - Updating the underlying JSON automatically updates the generated column and maintains the index — no extra work in your application. - Be sure the data type and collation of the generated column match how you query, else the optimizer may not pick the index.

What NOT to say: - Do not suggest a trigger — generated columns are the correct mechanism and do not have the trigger overhead.

Common follow-up: *What is the difference between STORED and VIRTUAL generated columns?*

Q120 · Invisible indexes

Tags: Difficulty: Easy · Asked at: Flipkart, Swiggy, Razorpay, Walmart Labs · Engine: MySQL 8 · Topic: Invisible indexes

The question:

> What is an invisible index in MySQL 8 and why would you use one?

The 30-second answer: An invisible index is one the optimizer ignores during query planning, even though it is fully maintained on writes. Used to test whether dropping an index would hurt performance, without actually dropping it.

Step-by-step thought process: 1. `ALTER TABLE orders ALTER INDEX idx_status INVISIBLE;` — marks the index invisible. Writes still maintain it; reads do not use it for plan selection. 2. Run your workload. Monitor slow query log, performance schema, application latency. 3. If nothing degrades, `DROP INDEX idx_status;` — safely. 4. If something degrades, `ALTER TABLE orders ALTER INDEX idx_status VISIBLE;` — instant rollback, no rebuild needed.

Edge cases to mention: - Even invisible indexes are still updated on `INSERT` / `UPDATE` / `DELETE` — they cost write performance just like normal indexes. Goal is purely about read-time experimentation. - The primary key cannot be made invisible. - Setting `optimizer_switch='use_invisible_indexes=on'` temporarily forces the optimizer to consider them — useful in a debugging session. - Indexes default to `VISIBLE` on create.

What NOT to say: - Do not confuse invisible indexes with hidden columns or covered indexes — they are unrelated concepts.

Common follow-up: *If invisible indexes still cost writes, what is the real saving compared to just dropping the index?*

```
ALTER TABLE orders ALTER INDEX idx_status INVISIBLE;
-- monitor production for a week
ALTER TABLE orders DROP INDEX idx_status;  -- safe to drop
```

Q121 · Descending indexes

Tags: Difficulty: Medium · Asked at: Walmart Labs, Razorpay, Goldman Sachs, JP Morgan · Engine: MySQL 8 · Topic: Indexes

The question:

> What is a descending index in MySQL 8? Before 8.0, did `CREATE INDEX (col DESC)` do anything?

The 30-second answer: Before 8.0 the `DESC` keyword on index columns was accepted syntactically but ignored — all indexes were stored ascending and the optimizer faked a descending scan by walking backwards. From 8.0, true descending indexes are stored physically in descending order, eliminating the backward-scan penalty.

Step-by-step thought process: 1. The classic case: `ORDER BY created_at DESC, score ASC` with an index on `(created_at DESC, score ASC)`. Pre-8.0 the optimizer would scan an ascending composite index, but it could not efficiently combine mixed sort directions. 2. With true descending indexes, the storage order matches the query order exactly — straight forward scan, no reversal, no filesort. 3. The benefit is largest when you have **mixed direction sorts** in a composite

ordering. For single-column sorts, the backward-scan optimization was already good. 4. Use `EXPLAIN` to confirm: look for `Using index` and no `Using filesort` in the Extra column.

Edge cases to mention: - A simple `ORDER BY col DESC` on a single-column ascending index has always been fast in MySQL — the engine can read pages in reverse direction. Descending indexes mainly help mixed-direction composite sorts. - Descending indexes cost the same to maintain as ascending ones. - `EXPLAIN FORMAT=TREE` will show whether a backward scan is being used.

What NOT to say: - Do not say pre-8.0 was "impossible" to handle descending sort — it was possible, just with backward index scan that did not compose well with multi-column mixed orders.

Common follow-up: *Give an example query where a true descending index gives a noticeable improvement over the fake backward-scan approach.*

```
CREATE INDEX idx_leaderboard ON scores (game_id, score DESC, ts);
SELECT * FROM scores WHERE game_id = 5 ORDER BY score DESC, ts LIMIT 10;
```

Q122 · Histograms — what and when

Tags: Difficulty: Medium · Asked at: Walmart Labs, Razorpay, Cred, Flipkart · Engine: MySQL 8 · Topic: Statistics

The question:

> What are histograms in MySQL 8 and how do they differ from index statistics?

The 30-second answer: Histograms are per-column distribution statistics that the optimizer uses for cardinality estimation. They are independent of indexes — you can build a histogram on a non-indexed column. They help with skewed data and range predicates.

Step-by-step thought process: 1. Index statistics give the optimizer a rough idea of cardinality (number of distinct values) per index. They do not describe how values are distributed. 2. Histograms describe the actual distribution. For a `status` column with values `'PAID' (95%)`, `'REFUNDED' (1%)`, `'PENDING' (4%)`, the optimizer with a histogram knows that `WHERE status = 'REFUNDED'` returns very few rows, even if there is no index. 3. Two types: **singleton** (one bucket per value, used when distinct count is small) and **equi-height** (variable-width buckets each containing roughly equal row counts, used for larger distinct counts). 4. Build with `ANALYZE TABLE orders UPDATE HISTOGRAM ON status WITH 32 BUCKETS;`.

Edge cases to mention: - Histograms are stored in the data dictionary and persist across restarts. - Auto-update: histograms do **not** automatically refresh as data changes. You must re-run `ANALYZE TABLE` after significant data shifts. This is a big trap. - The default bucket count is 100. More buckets

means finer detail but slower estimation. - Histograms on indexed columns are also useful — they cover correlation cases that index statistics alone miss.

What NOT to say: - Do not assume histograms are a silver bullet — they only help when the optimizer cost model is being misled by data skew, which is not every query.

Common follow-up: *What does the `WITH N BUCKETS` choice imply about accuracy vs storage?*

```
ANALYZE TABLE orders UPDATE HISTOGRAM ON status WITH 16 BUCKETS;
-- Inspect:
SELECT * FROM information_schema.column_statistics WHERE table_name = 'orders';
```

Q123 · Histograms vs indexes — when to use which

Tags: Difficulty: Medium · Asked at: Walmart Labs, Cred, Razorpay, Meesho · Engine: MySQL 8 · Topic: Statistics

The question:

> Should I always create an index instead of a histogram? When is a histogram the better choice?

The 30-second answer: Histograms help the optimizer make better **plan choices** but do not speed up actual access. Indexes speed up access. Use a histogram on a low-cardinality skewed column that is queried rarely or only as part of complex predicates — where an index would be wasted overhead on writes.

Step-by-step thought process: 1. Indexes are expensive: extra disk, extra write amplification on every insert/update, plus maintenance during DDL. For a column queried once a month with a complex `WHERE`, an index may not be worth it. 2. A histogram costs nothing at write time. It is only consulted during query planning. 3. Common pattern: histogram on `status` so the optimizer knows that `status = 'REFUNDED'` returns few rows and should drive the join order, while you keep the actual index on `customer_id` (the high-selectivity filter). 4. Histograms also help when you have multiple predicates on the same table — the optimizer can combine cardinality estimates more accurately.

Edge cases to mention: - If you already have an index on the column, MySQL still finds histograms useful — they complement index statistics for range and inequality predicates. - Histograms do not help with join predicate selectivity in all cases — they are per-column, not per-correlation. - Read-mostly OLAP tables benefit more from histograms than OLTP write-heavy tables.

What NOT to say: - Do not claim histograms replace indexes — they are about plan quality, not access path speed.

Common follow-up: *What query pattern would benefit from a histogram but not from an index?*

Q124 · InnoDB gap locks and Repeatable Read

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Walmart Labs, Goldman Sachs · Engine: MySQL 8 · Topic: Locking

The question:

> Why does MySQL's default Repeatable Read isolation level prevent phantom reads, when the SQL standard says RR can allow phantoms?

The 30-second answer: Because InnoDB takes **gap locks** in addition to record locks. A gap lock prevents another transaction from inserting a new row into the gap between two existing index entries — which is exactly what would cause a phantom.

Step-by-step thought process: 1. The SQL standard defines RR as preventing **non-repeatable reads** but allowing **phantom reads** (a re-executed range query seeing new rows). 2. InnoDB's RR implementation adds gap locks to range queries with `SELECT ... FOR UPDATE` or `SELECT ... LOCK IN SHARE MODE`. A gap lock locks the empty space between two index records, blocking inserts into that range. 3. The combination of a record lock plus a gap lock is called a **next-key lock** — it locks both an existing row and the gap immediately before it. 4. The result: range reads under RR see a stable set of rows even on re-execution. Phantoms are prevented by blocking the inserts that would create them.

Edge cases to mention: - Non-locking reads (plain `SELECT` without `FOR UPDATE`) under RR use **MVCC snapshots** — they see a consistent snapshot taken at the first read of the transaction, so phantoms are also prevented for these, but via snapshot reads rather than gap locks. - Gap locks are disabled at the Read Committed isolation level — that is the main behavioural difference between RR and RC in InnoDB. - Gap locks can produce surprising deadlocks when multiple transactions insert into adjacent ranges.

What NOT to say: - Do not claim MySQL "violates the SQL standard" — it is stricter than the standard requires, which is allowed.

Common follow-up: *Give an example of a deadlock caused purely by gap locks, where no overlapping rows are touched.*

Q125 · Next-key locks and intention locks

Tags: Difficulty: Hard · Asked at: JP Morgan, Goldman, Razorpay, Walmart Labs · Engine: MySQL 8 · Topic: Locking

The question:

> Explain next-key locks and intention locks in InnoDB.

The 30-second answer: Next-key lock = record lock + gap lock immediately before the record, taken atomically. Intention locks are table-level signals (`IS` , `IX`) that a transaction will be taking row-level shared / exclusive locks — they let the engine quickly check compatibility for table-level operations like `LOCK TABLES` or DDL.

Step-by-step thought process: 1. Next-key lock notation: `(prev_record, this_record]` — open on the left, closed on the right. So a next-key lock on row with id=10 (where the previous id in the index is 5) locks the gap `(5, 10]` . 2. Intention shared lock (`IS`) is taken on a table when a transaction plans to take row-level shared locks (`LOCK IN SHARE MODE`). Intention exclusive (`IX`) for row-level X locks. 3. Intention locks are compatible with each other — many transactions can hold `IS` or `IX` simultaneously. They conflict only with table-level `S` and `X` locks. 4. The engine acquires the intention lock on the table first, then the row-level lock on individual records. This two-level scheme is what makes row-level locking efficient.

Edge cases to mention: - Next-key locks are taken on **unique secondary index lookups** as well, although the trailing gap may be released quickly if the predicate is `=` on a unique index. - Insert intention locks: a special gap lock variant taken during inserts. Compatible with other insert intentions on the same gap but conflicts with normal gap locks. - `SHOW ENGINE INNODB STATUS` displays current locks. Performance schema's `data_locks` table gives finer detail.

What NOT to say: - Do not confuse intention locks with metadata locks (MDL) — MDL is for DDL serialisation, intention locks are for row-level lock coordination.

Common follow-up: *How does an "insert intention" lock differ from a regular gap lock?*

Q126 · SELECT FOR UPDATE and SKIP LOCKED

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, PhonePe · Engine: MySQL 8 · Topic: Locking

The question:

> What does `SELECT ... FOR UPDATE` do? What is `SKIP LOCKED` (added in 8.0)?

The 30-second answer: `FOR UPDATE` takes an exclusive lock on the rows read, so no other transaction can read with `FOR UPDATE` or modify them until the current transaction commits. `SKIP LOCKED` (8.0+) tells the query to skip rows that are currently locked, returning only unlocked ones — perfect for queue-style workloads.

Step-by-step thought process: 1. Without `FOR UPDATE`, plain `SELECT` under RR sees an MVCC snapshot and takes no locks. Concurrent updates are allowed. 2. `FOR UPDATE` makes the read a locking read — equivalent in lock strength to a write. 3. `FOR SHARE` (formerly `LOCK IN SHARE MODE`) takes a shared lock — others can read but not write. 4. `SKIP LOCKED` modifier: if a row is locked by another transaction, skip it instead of blocking. The query returns whatever was unlocked at the moment. Combined with `LIMIT 1 FOR UPDATE SKIP LOCKED` you get a lock-free queue dequeuer. 5. `NOWAIT` modifier: if any row is locked, fail immediately with an error instead of waiting or skipping.

Edge cases to mention: - `SKIP LOCKED` only skips at the row level — gap locks are not the unit of skipping. - `SKIP LOCKED` can return different results on repeat executions, which is the intended behaviour for queue workers. - These modifiers were added to standardise patterns that previously required hacks like `LIMIT ... FOR UPDATE` with retry loops.

What NOT to say: - Do not use `SKIP LOCKED` if you need every row eventually — it might skip rows that never get picked up if your workers keep restarting on the wrong rows.

Common follow-up: *Show me the classic "pop next job from queue" SQL using SKIP LOCKED.*

```
START TRANSACTION;
SELECT id, payload FROM job_queue
WHERE status = 'pending'
ORDER BY priority DESC, created_at
LIMIT 1
FOR UPDATE SKIP LOCKED;
-- worker processes job
UPDATE job_queue SET status = 'done' WHERE id = ?;
COMMIT;
```

Q127 · InnoDB deadlock detection

Tags: Difficulty: Medium · Asked at: Walmart Labs, JP Morgan, Razorpay, Cred · Engine: MySQL 8 · Topic: Locking

The question:

> How does InnoDB detect deadlocks and what happens when one occurs?

The 30-second answer: InnoDB runs an explicit deadlock detector that walks the wait-for graph on every lock wait. If a cycle is found, it picks the smaller transaction (by undo log size) as the victim, rolls it back, and returns error 1213 to that connection.

Step-by-step thought process: 1. The wait-for graph has transactions as nodes, with an edge from T1 to T2 if T1 is waiting on a lock T2 holds. 2. On each lock request that has to wait, InnoDB walks the graph starting from the requester. If it reaches itself, there is a cycle — a deadlock. 3. Victim selection: typically the transaction with the smallest amount of work done (measured by undo log entries). The intuition is "kill the cheaper one to roll back". 4. The victim sees `ERROR 1213 (40001): Deadlock found when trying to get lock; try restarting transaction`. The application should retry.

Edge cases to mention: - `innodb_deadlock_detect = OFF` disables the detector. On systems with thousands of concurrent transactions, deadlock detection can become a bottleneck — it is O(graph size) on every wait. Disabling it falls back to `innodb_lock_wait_timeout`. - `SHOW ENGINE INNODB STATUS` displays the most recent deadlock with full lock detail — invaluable for debugging. - Performance schema's `events_errors_summary_by_thread_by_error` tracks deadlock counts over time. - Deadlocks are normal in any concurrent OLTP system. The application must retry, with backoff.

What NOT to say: - Do not say "deadlocks should never happen in correct code" — they do, and well-written code retries them rather than tries to prevent them entirely.

Common follow-up: *Give a two-statement scenario that always deadlocks.*

Q128 · Binlog and replication basics

Tags: Difficulty: Easy · Asked at: Flipkart, Swiggy, Razorpay, Walmart Labs · Engine: MySQL 8 · Topic: Replication

The question:

> How does MySQL replication work? What is the binlog?

The 30-second answer: The binlog (binary log) is an ordered append-only log of all data-modifying statements (or row events). Replicas pull these events from the primary and replay them locally, catching up to and then tracking the primary's state.

Step-by-step thought process: 1. On the primary, every transaction that modifies data writes to the binlog at commit time. 2. A replica runs two threads: the **IO thread** which connects to the primary and pulls binlog events into a local **relay log**, and the **SQL thread** (or worker threads for parallel replication) which reads the relay log and applies the events to local tables. 3. Three binlog formats: **STATEMENT** (logs the SQL statement; small but problematic for non-deterministic queries), **ROW** (logs the before/after image of each modified row; safe but verbose; the modern default), **MIXED** (statement where safe, row otherwise). 4. Replication lag = how far behind the replica is. Monitored via `SHOW REPLICA STATUS` — look at `Seconds_Behind_Source`.

Edge cases to mention: - ROW format with `binlog_row_image=MINIMAL` only logs changed columns plus the primary key — saves space. - Replication is **asynchronous** by default — the primary commits without waiting for the replica. - Semi-synchronous replication (an optional plugin) makes the primary wait for at least one replica to acknowledge receipt before commit returns to the client. - Parallel apply on replicas: multiple SQL workers process independent transactions concurrently. Configured via `replica_parallel_workers`.

What NOT to say: - Do not call MySQL's default replication "synchronous" — it is asynchronous unless you explicitly enable semi-sync.

Common follow-up: *What is the trade-off between ROW and STATEMENT binlog format?*

Q129 · GTIDs and why they replaced binlog coordinates

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Walmart Labs, Flipkart · Engine: MySQL 8 · Topic: Replication

The question:

> What is a GTID and why is GTID-based replication preferred over file-and-position-based?

The 30-second answer: GTID = Global Transaction Identifier. A unique tag assigned to every transaction in the binlog of the form `server_uuid:transaction_number`. Replication can resume by GTID instead of by file name and byte offset, which makes failover dramatically simpler.

Step-by-step thought process: 1. Legacy replication tracks position as `(binlog_file_name, binary_log_offset)`. On failover, computing the right starting point on a new primary involves cross-referencing logs by hand — error-prone. 2. GTID assigns each committed transaction a globally unique ID. The replica records the set of GTIDs it has applied (`gtid_executed`). 3. When connecting to a new primary, the replica asks "send me everything you have that I haven't seen", expressed as the set difference of GTIDs. No file names or offsets involved. 4. Enabled via `gtid_mode=ON` and `enforce_gtid_consistency=ON`. From MySQL 8, GTID is the recommended mode for any new deployment.

Edge cases to mention: - Once you enable GTID, every transaction must be GTID-tagged — certain operations (`CREATE TABLE ... SELECT`, non-transactional table updates inside transactions) are restricted under `enforce_gtid_consistency`. - Migrating an existing position-based replica to GTID is supported but requires a coordinated rolling change across the topology. - `gtid_executed` and `gtid_purged` are key system variables for debugging replication state.

What NOT to say: - Do not claim GTIDs eliminate replication lag — they only simplify topology and failover. Lag is determined by workload and apply parallelism.

Common follow-up: *What does enforce_gtid_consistency actually enforce?*

Q130 · Async, semi-sync, and Group Replication

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, JP Morgan, Walmart Labs · Engine: MySQL 8 · Topic: Replication

The question:

> Compare async replication, semi-sync replication, and Group Replication in MySQL.

The 30-second answer: **Async**: primary commits without waiting; data loss possible on primary failure. **Semi-sync**: primary waits for one replica to acknowledge receipt of the binlog event before commit returns. **Group Replication**: a built-in consensus-based replication group with automatic leader election and conflict detection — closer to a true multi-master cluster.

Step-by-step thought process: 1. Async is the default. Fast commits but a primary crash can lose any in-flight transactions that hadn't reached the replica. 2. Semi-sync (`rpl_semi_sync_*` plugins) makes the primary wait for a replica's acknowledgement of binlog receipt — not apply. Improves durability with modest latency cost. Falls back to async if no replicas respond within a timeout. 3. Group Replication uses a Paxos-variant protocol. Transactions are certified across the group before commit. Supports single-primary (default — only one node accepts writes) or multi-primary (all nodes accept writes, conflict detection rejects conflicting transactions). 4. InnoDB Cluster is the productised stack: Group Replication + MySQL Router + Shell. Provides HA with one or two-command provisioning.

Edge cases to mention: - Semi-sync waits for binlog receipt, not application. The replica may still lag in applying. So strictly speaking semi-sync does not guarantee read-your-writes on the replica. - Group Replication is most useful for HA, less so for read scale — it does not by itself give you many replicas to read from. Pair with traditional async replicas for read scale. - All three modes share the same binlog/relay-log substrate underneath.

What NOT to say: - Do not claim Group Replication is a sharding solution — it replicates the full dataset to every node. Use it for HA, not horizontal scaling of data volume.

Common follow-up: *If semi-sync only guarantees receipt and not application, is it really better than async for durability?*

Q131 · Performance Schema for slow queries and locks

Tags: Difficulty: Medium · Asked at: Walmart Labs, Razorpay, Cred, Flipkart · Engine: MySQL 8 · Topic: Diagnostics

The question:

> How do you find slow queries and lock contention in MySQL 8 using Performance Schema?

The 30-second answer: Query `performance_schema.events_statements_summary_by_digest` for slow statements (aggregated by query template). For locks, query `performance_schema.data_locks` and `data_lock_waits` to see who is holding what and who is waiting on whom.

Step-by-step thought process: 1. `events_statements_summary_by_digest` groups statements by their normalised template (literals stripped). Columns include `COUNT_STAR`, `SUM_TIMER_WAIT`, `AVG_TIMER_WAIT`, `SUM_ROWS_EXAMINED`. Sort by `SUM_TIMER_WAIT` to find the workload's hottest queries. 2. The traditional slow query log still works (`slow_query_log=1`, `long_query_time=1`) but Performance Schema is more flexible and queryable. 3. `data_locks`: shows current row-level locks with the index name, lock type (S/X), lock mode (record / gap / next-key), and owning thread. 4. `data_lock_waits`: rows where a session is waiting on another. Join with `data_locks` to identify both the waiter and the holder.

Edge cases to mention: - Performance Schema has measurable overhead at full instrumentation — typical sensible production setup enables statement and lock instrumentation but not the most granular wait events. - The `sys` schema (covered next) wraps these tables in user-friendly views. - Resetting digest statistics: `TRUNCATE performance_schema.events_statements_summary_by_digest;` — useful for benchmarking a specific window.

What NOT to say: - Do not rely on the slow query log alone in 2026 — it misses fast queries executed millions of times that dominate the workload.

Common follow-up: *How do you reset the digest statistics to measure performance during a specific test window?*

Q132 · The sys schema

Tags: Difficulty: Easy · Asked at: Walmart Labs, Razorpay, Flipkart, Swiggy · Engine: MySQL 8 · Topic: Diagnostics

The question:

> What is the sys schema and how does it relate to Performance Schema?

The 30-second answer: The `sys` schema is a collection of views, functions, and stored procedures that wrap Performance Schema in human-readable form. It ships built-in with MySQL 5.7+ and is the standard go-to for ad-hoc diagnostics.

Step-by-step thought process: 1. Performance Schema tables are raw and verbose — columns use timer units of picoseconds, schema and object names are scattered across multiple tables. 2. The `sys` schema views format these usefully: human-readable times, grouped layouts, sensible default sort orders. 3. Highly-used views: `sys.x$statement_analysis` (top statements), `sys.x$io_global_by_file_by_bytes` (file I/O hotspots), `sys.innodb_lock_waits` (current lock waits with all the context). 4. There are also stored procedures: `sys.diagnostics()` dumps a comprehensive snapshot, `sys.ps_setup_save()` / `ps_setup_reload_saved()` for adjusting instrumentation.

Edge cases to mention: - The double-underscore variants (`x$` prefix) return raw values without formatting — useful when piping to scripts. - The `sys` schema is sometimes not enabled by default in highly locked-down installations; check before relying on it in client environments.

What NOT to say: - Do not assume `sys` is for "system" tables like `information_schema` — it specifically wraps Performance Schema for diagnostics.

Common follow-up: *Show me a sys schema query that lists the 10 queries doing the most full table scans.*

```
SELECT * FROM sys.statements_with_full_table_scans
ORDER BY no_index_used_count DESC LIMIT 10;
```

Q133 · Roles in MySQL 8

Tags: Difficulty: Easy · Asked at: Razorpay, Cred, Walmart Labs, JP Morgan · Engine: MySQL 8 · Topic: Security

The question:

> What are roles in MySQL 8 and why are they useful compared to direct GRANT statements?

The 30-second answer: A role is a named bundle of privileges. You `CREATE ROLE`, `GRANT` privileges to the role, then `GRANT` the role to users. Adding privileges to a role propagates to all users who hold it — no more rewriting GRANTs across dozens of accounts.

Step-by-step thought process: 1. Pre-8.0 pattern: every user had their own GRANTs. Onboarding a new developer meant copy-pasting a long script of grants. Updating privileges meant updating every user. 2. With roles: `CREATE ROLE readonly_dev; GRANT SELECT ON app.* TO`

`readonly_dev; GRANT readonly_dev TO 'alice'@'%';` 3. Roles must be **activated** in a session: `SET ROLE readonly_dev;` . Or set a default with `SET DEFAULT ROLE readonly_dev TO 'alice'@'%';` . 4. Roles can be granted to other roles, forming a hierarchy. Useful for layered privilege design (e.g. `senior_dev` inherits from `dev`).

Edge cases to mention: - `mandatory_roles` system variable: a comma-separated list of roles automatically granted to every account. - `SET ROLE ALL` activates every role granted to the current user — usually what people want. - Old-style `GRANT` statements remain valid; roles are an addition, not a replacement.

What NOT to say: - Do not say roles replace user accounts — they are an additional layer above users.

Common follow-up: *How do you make a role active by default without requiring SET ROLE every connection?*

```
CREATE ROLE readwrite_app;
GRANT SELECT, INSERT, UPDATE, DELETE ON shop.* TO readwrite_app;
CREATE USER 'app_svc'@'%' IDENTIFIED BY 'secret';
GRANT readwrite_app TO 'app_svc'@'%';
SET DEFAULT ROLE readwrite_app TO 'app_svc'@'%';
```

Q134 · Hash join in MySQL 8

Tags: Difficulty: Medium · Asked at: Walmart Labs, Razorpay, Cred, Flipkart · Engine: MySQL 8 · Topic: Optimizer

The question:

> When did hash join arrive in MySQL? When does the optimizer pick it over nested-loop?

The 30-second answer: Hash join was added in MySQL 8.0.18. The optimizer picks it for joins where neither side has a usable index on the join column, or where a hash join is cheaper than the indexed nested-loop. Earlier versions of 8.0 always did nested-loop, even for unjoinable predicates.

Step-by-step thought process: 1. **Nested-loop join**: for each row in the outer table, look up matching rows in the inner table using an index. Efficient when the inner index is selective and small. 2. **Hash join**: build a hash table from one input (the "build side"), then probe it with the other (the "probe side"). Single pass over each input — `O(N+M)` regardless of indexes. 3. When inputs are large and there is no useful index on the join column, hash join is dramatically faster. 4. From 8.0.20, hash join is used for both equi-joins and non-equi-joins under certain conditions. Earlier 8.0.18-19 had it only for equi-joins without an indexed equi-join column.

Edge cases to mention: - The optimizer hint `/*+ HASH_JOIN(t1, t2) */` forces hash join; `/*+ NO_HASH_JOIN(t1, t2) */` prevents it. - Hash join uses memory up to `join_buffer_size`. If the build side does not fit, it spills to disk — much slower. - `EXPLAIN FORMAT=TREE` and `EXPLAIN ANALYZE` clearly show which join algorithm was used. `EXPLAIN` in tabular form is less explicit.

What NOT to say: - Do not say MySQL "never had hash join before" — it never had it for normal joins. There were some hash-join-like internal mechanisms in EXISTS / IN subqueries.

Common follow-up: *If hash join is so much faster for large unjoined tables, why doesn't the optimizer always pick it?*

```
EXPLAIN ANALYZE
SELECT o.*, c.name
FROM orders o JOIN customers c ON o.email = c.email;
-- Look for "Inner hash join" in the output tree.
```

Q135 · MySQL 8.4 LTS — what changed

Tags: Difficulty: Easy · Asked at: Razorpay, Cred, Walmart Labs · Engine: MySQL 8 · Topic: Versions

The question:

> What is MySQL 8.4 LTS and what are the most notable changes from 8.0?

The 30-second answer: MySQL 8.4 LTS (Long-Term Support, released April 2024) is the first LTS line after 8.0. Notable changes include removal of the deprecated `mysql_native_password` authentication plugin (now disabled by default), tighter default for `caching_sha2_password`, and various deprecations of older features.

Step-by-step thought process: 1. After 8.0, Oracle introduced an Innovation track (rolling 8.1, 8.2, 8.3 — fast-moving) and an LTS track. 8.4 is the first LTS. 2. Authentication: `mysql_native_password` is disabled by default in 8.4. You must explicitly install the plugin if legacy clients require it. Default is `caching_sha2_password`, which requires either TLS or RSA key exchange. 3. Several deprecated commands and options removed entirely. Some long-standing replication system variables renamed (e.g. `master_*` to `source_*` continues, but old names are now hard errors). 4. Replication topology compatibility: 8.4 replicas can replicate from 8.0 primaries but careful with the reverse direction.

Edge cases to mention: - Migration from 8.0 to 8.4 LTS: review the deprecation list before upgrading. Application drivers (especially older PHP / Python libs) sometimes still default to `mysql_native_password` and will fail. - LTS line gets bug fixes for ~5 years; the Innovation line is supported only until the next minor release. - The next LTS line after 8.4 is expected to be 9.x.

What NOT to say: - Do not claim there is a "MySQL 9 LTS" yet without checking — version timelines move, hedge appropriately.

Common follow-up: *If we are on 8.0 and considering an upgrade, what is the biggest compatibility risk?*

Part B — SQL Server 2022 (25 Qs)

Version assumptions: SQL Server 2022 (16.x) with Compatibility Level 160. Some features (Query Store, columnstore, Always On) have existed in earlier versions but received refinements in 2022. We call out 2022-specific additions explicitly.

These questions skew toward what you will be asked at US-headquartered GCCs in India — JP Morgan, Goldman Sachs, Wells Fargo, Morgan Stanley, Walmart Labs, Target, AmEx, BNY Mellon. The T-SQL-specific syntax matters because porting from other dialects is a common interview probe.

Q136 · OVER clause and frame variations

Tags: Difficulty: Medium · Asked at: JP Morgan, Goldman Sachs, Wells Fargo, Morgan Stanley · Engine: SQL Server 2022 · Topic: Window functions

The question:

> Explain the full structure of the OVER clause in T-SQL, including frame variations.

The 30-second answer: `OVER (PARTITION BY ... ORDER BY ... ROWS|RANGE BETWEEN <start> AND <end>)`. PARTITION resets the window, ORDER BY orders within partition, and the frame defines which rows around the current row are visible to the aggregate.

Step-by-step thought process: 1. PARTITION BY is optional; without it the whole result is one partition. 2. ORDER BY is required for ranking functions and for any frame specification. Without it, an aggregate's frame is the entire partition. 3. Frame start and end use `UNBOUNDED PRECEDING`, `N PRECEDING`, `CURRENT ROW`, `N FOLLOWING`, `UNBOUNDED FOLLOWING`. 4. Default frame when ORDER BY is specified but no explicit frame clause: `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` — and crucially this includes ties at CURRENT ROW (RANGE semantics).

Edge cases to mention: - `RANGE` vs `ROWS`: RANGE works on logical value ranges and includes peer rows (those with equal ORDER BY values). ROWS works on physical row counts and is more deterministic for running totals. - `RANGE BETWEEN INTERVAL '7' DAY PRECEDING AND CURRENT ROW`: time-based windows on date columns — supported in 2022 for date / datetime ORDER BY

columns. - Named windows: `WINDOW w AS (PARTITION BY x ORDER BY y)` — added to T-SQL in 2022 — lets multiple aggregates share a window definition.

What NOT to say: - Do not say ROWS and RANGE always behave the same — they differ exactly when ORDER BY columns have duplicates.

Common follow-up: *Show me a query where RANGE and ROWS produce different running totals.*

```
SELECT order_id, customer_id, amount,
  SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_date
                    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_rows,
  SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_date
                    RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_range
FROM orders;
```

Q137 · IGNORE NULLS in window functions (2022)

Tags: Difficulty: Medium · Asked at: JP Morgan, Goldman Sachs, Walmart Labs, AmEx · Engine: SQL Server 2022 · Topic: Window functions

The question:

> What does IGNORE NULLS do in LAG, LEAD, FIRST_VALUE, LAST_VALUE? When was it added?

The 30-second answer: `IGNORE NULLS` (added to T-SQL in SQL Server 2022) makes these window functions skip NULL values when computing the result. The standard behaviour is `RESPECT NULLS` — return NULL if the relevant row is NULL.

Step-by-step thought process: 1. Without IGNORE NULLS: `LAG(price) OVER (ORDER BY ts)` returns the actual previous row's price, which may be NULL. 2. With IGNORE NULLS: `LAG(price) IGNORE NULLS OVER (ORDER BY ts)` returns the most recent non-NULL price. 3. The most useful application is **forward-filling** a sparse time series: `LAST_VALUE(price) IGNORE NULLS OVER (ORDER BY ts ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`. 4. Without IGNORE NULLS, this pattern previously required a CTE with `MAX(CASE WHEN price IS NOT NULL THEN ts END)` and self-joins — verbose and slow.

Edge cases to mention: - IGNORE NULLS works on `LAG`, `LEAD`, `FIRST_VALUE`, `LAST_VALUE`. - Aggregates like SUM and COUNT already ignore NULLs by default (per SQL standard) — IGNORE NULLS syntax is not applicable to them. - Earlier versions (pre-2022) required clunky workarounds. If you are interviewing for a legacy SQL Server 2017 / 2019 shop, mention this as the prior pain point.

What NOT to say: - Do not say IGNORE NULLS is the default — it is not. RESPECT NULLS is default.

Common follow-up: *Show me the "forward-fill last known price" pattern using IGNORE NULLS.*

```
SELECT ts, ticker,
  LAST_VALUE(price) IGNORE NULLS OVER (
    PARTITION BY ticker ORDER BY ts
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS last_known_price
FROM ticks;
```

Q138 · RANGE vs ROWS — when does it bite

Tags: Difficulty: Hard · Asked at: Goldman Sachs, JP Morgan, Morgan Stanley, BNY Mellon · Engine: SQL Server 2022 · Topic: Window functions

The question:

> Why does my running total OVER (ORDER BY date) show all rows of the same date with the same total? How do I fix it?

The 30-second answer: Because the default frame is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, and RANGE treats all rows with the same ORDER BY value as peers — they all get the same cumulative sum. Switch to `ROWS BETWEEN ...` for a per-row running total.

Step-by-step thought process: 1. Suppose three rows have `order_date = '2026-05-26'` with amounts 100, 200, 300. With default RANGE, all three see a running total of `(prior days) + 100+200+300 = 600 + prior`. Identical for all three. 2. With ROWS, each row sees only previous rows by physical order: first row 100 + prior, second 100+200 + prior, third 100+200+300 + prior. 3. Why does the default work this way? The SQL standard defines RANGE with peer semantics to be deterministic regardless of tie-breaking — ROWS would expose the order in which ties are broken. 4. Always specify `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` explicitly when you want a per-row running total. Better still, add a tie-breaker column to ORDER BY.

Edge cases to mention: - If your ORDER BY column is unique (e.g. a primary key), RANGE and ROWS give identical results. - The bug is silent — it produces "wrong" but plausible-looking numbers. Especially common when summing daily revenue. - Performance: ROWS and RANGE typically have similar costs; the choice is semantic, not performance-driven.

What NOT to say: - Do not blame the optimizer — the behaviour is dictated by the SQL standard, not a SQL Server quirk.

Common follow-up: *Is there ever a case where RANGE is actually what you want?*

Q139 · OUTPUT clause

Tags: Difficulty: Medium · Asked at: JP Morgan, Wells Fargo, Goldman Sachs, AmEx · Engine: SQL Server 2022 · Topic: T-SQL

The question:

> What does the OUTPUT clause do? When is it useful?

The 30-second answer: OUTPUT returns inserted / updated / deleted rows from an INSERT, UPDATE, DELETE, or MERGE statement, optionally into another table. Use it to capture identity values, build audit tables, or implement returning-style queries.

Step-by-step thought process: 1. `INSERT INTO orders (customer_id, amount) OUTPUT INSERTED.id, INSERTED.created_at VALUES (...)` — returns the auto-generated id and created_at without a second query. 2. `UPDATE accounts SET balance = balance - 1000 OUTPUT DELETED.balance AS old_bal, INSERTED.balance AS new_bal WHERE id = 42` — returns both before and after values. 3. `INSERT ... OUTPUT INSERTED.* INTO audit.orders_history` — atomic write to both target and audit table in one statement. 4. `DELETE FROM jobs OUTPUT DELETED.* WHERE status = 'done' AND completed_at < ...` — capture what got deleted for downstream processing.

Edge cases to mention: - `INSERTED` and `DELETED` are the same pseudo-tables used in triggers. INSERT sees only INSERTED. DELETE sees only DELETED. UPDATE sees both. - OUTPUT INTO a target table cannot have triggers, foreign keys to other tables (some restrictions), or be the same table as the source. The restrictions have been relaxed over versions. - OUTPUT can return to the client even when used with OUTPUT INTO. You can both write to an audit table and return to the application.

What NOT to say: - Do not say "OUTPUT is like Postgres RETURNING" without qualification — it is more powerful (supports INTO a table), but has more restrictions on the target.

Common follow-up: *How would you use OUTPUT to atomically capture which rows were affected by an UPDATE for downstream processing?*

```
UPDATE accounts SET balance = balance - 100
OUTPUT DELETED.balance AS old_bal, INSERTED.balance AS new_bal, INSERTED.id
INTO audit.balance_changes (id, old_bal, new_bal)
WHERE id IN (SELECT id FROM debit_queue);
```

Q140 · MERGE and its pitfalls

Tags: Difficulty: Hard · Asked at: JP Morgan, Goldman Sachs, Wells Fargo, Walmart Labs · Engine: SQL Server 2022 · Topic: T-SQL

The question:

> What is MERGE in T-SQL and what are the known issues you should be careful of?

The 30-second answer: MERGE combines INSERT / UPDATE / DELETE in one statement based on whether rows match a join condition. It has had a history of correctness bugs (HALLOWEEN problem variants, race conditions under concurrent access). For simple upserts many seasoned engineers prefer explicit INSERT + UPDATE.

Step-by-step thought process: 1. Basic syntax: `MERGE target USING source ON match_condition WHEN MATCHED THEN UPDATE ... WHEN NOT MATCHED THEN INSERT ... WHEN NOT MATCHED BY SOURCE THEN DELETE;` 2. Known issues over the years include: concurrency anomalies under READ COMMITTED isolation where two simultaneous MERGEs could both decide a row "does not match" and both insert; trigger ordering quirks; surprises with computed columns. 3. Microsoft has long-standing official guidance to prefer separate INSERT and UPDATE statements when correctness under concurrency is paramount, or to wrap MERGE in a serializable transaction with appropriate locking hints. 4. Despite the warnings, MERGE remains useful for batch ETL-style operations where concurrency is controlled (single ETL job, no concurrent writers).

Edge cases to mention: - Unique key violations under MERGE can produce confusing error messages. Best practice: ensure the join condition uses the unique key columns. - `WHEN NOT MATCHED BY SOURCE` deletes rows in target not present in source — be very careful, this can drop a lot of data if the source query is buggy. - For OLTP upserts at scale, use `INSERT ... WITH (HOLDLOCK)` followed by `UPDATE` with `ROWCOUNT = 0` check, or use the older `INSERT ... WHERE NOT EXISTS` + `UPDATE` pattern.

What NOT to say: - Do not say MERGE is always safe — it is widely cautioned against by SQL Server MVPs for OLTP work.

Common follow-up: *Why might MERGE produce duplicate rows under concurrent execution?*

Q141 · CROSS APPLY vs LATERAL

Tags: Difficulty: Medium · Asked at: Goldman Sachs, JP Morgan, Walmart Labs, Target · Engine: SQL Server 2022 · Topic: T-SQL

The question:

> What does CROSS APPLY do? Is there a difference between APPLY and the SQL-standard LATERAL?

The 30-second answer: CROSS APPLY is T-SQL's syntax for a lateral join — for each row in the left table, evaluate a table-valued expression on the right that can reference left-side columns. Equivalent to LATERAL in standard SQL / Postgres. OUTER APPLY is the LEFT JOIN variant.

Step-by-step thought process: 1. Standard JOIN: both sides are independent queries, joined by a predicate. The right side cannot reference left-side columns. 2. CROSS APPLY: the right side is a table-valued expression that can reference left-side columns. Useful for calling table-valued functions, or for "top N per group" via a correlated subquery. 3. CROSS APPLY drops left rows if the right side returns no rows. OUTER APPLY keeps them with NULLs on the right (like LEFT JOIN). 4. T-SQL has had APPLY since SQL Server 2005 — well before LATERAL became widely adopted across other engines.

Edge cases to mention: - Common pattern: top-N per group via CROSS APPLY + TOP. `FROM customers c CROSS APPLY (SELECT TOP 3 * FROM orders o WHERE o.customer_id = c.id ORDER BY o.amount DESC) o`. Sometimes cleaner than window functions. - APPLY is essential when invoking table-valued functions that take row-specific parameters. - Performance: the optimizer can sometimes rewrite APPLY into a join + correlated lookup. Sometimes it cannot, especially when the right side is non-trivial.

What NOT to say: - Do not claim APPLY is "SQL Server only" — every modern engine has lateral joins now, just under different names. APPLY is the T-SQL name.

Common follow-up: *Give me a CROSS APPLY query that returns the latest 3 orders per customer.*

```
SELECT c.id, c.name, o.order_id, o.amount, o.order_date
FROM customers c
CROSS APPLY (
  SELECT TOP 3 order_id, amount, order_date
  FROM orders WHERE customer_id = c.id
  ORDER BY order_date DESC
) o;
```

Q142 · Recursive CTEs in T-SQL

Tags: Difficulty: Medium · Asked at: JP Morgan, Wells Fargo, Walmart Labs, AmEx · Engine: SQL Server 2022 · Topic: CTEs

The question:

> How do you write a recursive CTE in T-SQL? What is MAXRECURSION?

The 30-second answer: Same as standard SQL: anchor member UNION ALL recursive member that references the CTE name. T-SQL adds the `OPTION (MAXRECURSION N)` query hint to override the default cap of 100 iterations. Set to 0 for unlimited.

Step-by-step thought process: 1. Structure: `WITH cte AS (anchor UNION ALL recursive) SELECT ... FROM cte OPTION (MAXRECURSION 1000);` 2. The MAXRECURSION default is 100. After 100 levels, the query fails with an error to protect against runaway recursion. 3. Setting MAXRECURSION = 0 means no limit — be sure cycles in your data are impossible or detected explicitly. 4. Common patterns: hierarchy traversal (employees-managers), graph traversal with cycle detection, generating series (anchor: SELECT 1; recursive: SELECT n+1 WHERE n < 1000).

Edge cases to mention: - Recursive CTEs cannot use aggregates, TOP, or several other constructs in the recursive member. - Type widening: the data type of each column is determined from the anchor member. Recursive members are implicitly converted to those types. Mismatches cause truncation or errors. - T-SQL recursive CTEs do not support iteration "in parallel" the way some engines do — they iterate one level at a time.

What NOT to say: - Do not assume MAXRECURSION is a global setting — it is per-query via the OPTION hint.

Common follow-up: *What is the default MAXRECURSION value and why is it set there?*

```
WITH org AS (
  SELECT id, name, manager_id, 0 AS lvl FROM employees WHERE manager_id IS NULL
  UNION ALL
  SELECT e.id, e.name, e.manager_id, o.lvl + 1
  FROM employees e JOIN org o ON e.manager_id = o.id
)
SELECT * FROM org OPTION (MAXRECURSION 1000);
```

Q143 · Complex CTEs and chained references

Tags: Difficulty: Medium · Asked at: Goldman Sachs, Morgan Stanley, JP Morgan, Walmart Labs · Engine: SQL Server 2022 · Topic: CTEs

The question:

> Can a CTE reference another CTE? Can you have a recursive CTE that references a non-recursive CTE?

The 30-second answer: Yes. A later CTE in the same WITH clause can reference earlier CTEs. A recursive CTE can reference non-recursive CTEs declared above it. This composability is a major reason CTEs are preferred over deeply nested subqueries.

Step-by-step thought process: 1. `WITH cte1 AS (...), cte2 AS (SELECT ... FROM cte1 WHERE ...), cte3 AS (SELECT ... FROM cte1 JOIN cte2 ...) SELECT * FROM cte3;` — each CTE can reference earlier ones. 2. For recursive CTE referencing non-recursive: declare the non-recursive one first. The anchor and recursive members of the recursive CTE can both reference it. 3. The optimizer may inline or materialize each CTE independently. Inspect the execution plan if performance is critical. 4. CTEs cannot reference forward — `cte1` cannot reference `cte2` if `cte2` is declared after it.

Edge cases to mention: - Multi-statement CTEs: with the `WITH` clause, the immediately following statement is the only one that sees the CTEs. You cannot define a CTE and reference it in a separate batch. - Some patterns make CTE materialization implicit via window functions or aggregates — the optimizer may choose to spool the intermediate. - For very complex pipelines, temp tables (`#temp`) often perform better than CTEs because they cache statistics that the optimizer can use.

What NOT to say: - Do not claim "CTEs always materialize" — in T-SQL they are typically inlined unless complexity forces a spool.

Common follow-up: *When would you replace a complex chain of CTEs with temp tables?*

Q144 · OPENJSON

Tags: Difficulty: Medium · Asked at: JP Morgan, Goldman Sachs, Wells Fargo, Target · Engine: SQL Server 2022 · Topic: JSON

The question:

> What is OPENJSON and how does it differ from JSON_VALUE / JSON_QUERY?

The 30-second answer: OPENJSON shreds a JSON document into a relational set of rows. JSON_VALUE returns a scalar at a given path; JSON_QUERY returns a JSON sub-object at a given path. OPENJSON is for "unnest the array" workflows; the other two are for scalar / sub-object extraction.

Step-by-step thought process: 1. `JSON_VALUE(j, '$.name')` returns the scalar at `$.name` as a `NVARCHAR(4000)` . Returns NULL if the value at the path is an object or array. 2. `JSON_QUERY(j, '$.address')` returns the object or array at `$.address` as JSON text. Returns NULL if the value is scalar. 3. `OPENJSON(j, '$.items')` returns a table. Without a WITH clause, three columns: `key` , `value` , `type` . With a WITH clause, you define explicit columns and types: `OPENJSON(j, '$.items') WITH (sku NVARCHAR(50), qty INT, price DECIMAL(10,2))` . 4. Use

OPENJSON for arrays you want to join row-by-row. Use JSON_VALUE for scalar predicates in WHERE clauses.

Edge cases to mention: - OPENJSON without a WITH clause is "loose typed" — useful for inspection but every value comes back as NVARCHAR. - `WITH (... AS JSON)` extracts a sub-document as JSON for further OPENJSON processing — for nested structures. - ISJSON function validates whether a column actually contains JSON. From 2022, supports an optional second parameter for JSON type validation (VALUE, ARRAY, OBJECT, SCALAR).

What NOT to say: - Do not call OPENJSON a "JSON_TABLE equivalent" — same idea but different syntax and capabilities. Be precise.

Common follow-up: *How would you shred a nested array of order items into rows along with their parent order metadata?*

```
SELECT o.order_id, i.sku, i.qty, i.price
FROM orders o
CROSS APPLY OPENJSON(o.items_json) WITH (
  sku NVARCHAR(50) '$.sku',
  qty INT '$.qty',
  price DECIMAL(10,2) '$.price'
) i;
```

Q145 · JSON_OBJECT and JSON_ARRAY (2022)

Tags: Difficulty: Easy · Asked at: JP Morgan, Wells Fargo, Walmart Labs, Target · Engine: SQL Server 2022 · Topic: JSON

The question:

> What is JSON_OBJECT in SQL Server 2022? How did you do this before?

The 30-second answer: SQL Server 2022 added `JSON_OBJECT` and `JSON_ARRAY` for constructing JSON literals from columns. Previously you used `FOR JSON PATH` (verbose) or string concatenation (error-prone with escaping). The new functions are cleaner for ad-hoc construction.

Step-by-step thought process: 1. `SELECT JSON_OBJECT('id': id, 'name': name, 'city': city) FROM users` — produces a JSON object per row. 2. `SELECT JSON_ARRAY(tag1, tag2, tag3) FROM ...` — produces a JSON array of values. 3. Nesting: `JSON_OBJECT('user': JSON_OBJECT('id': id, 'name': name), 'roles': JSON_ARRAY(role1, role2))`. 4. NULL handling: `ABSENT ON NULL` (default) omits keys with NULL values from the object; `NULL ON NULL` includes them with JSON `null`.

Edge cases to mention: - Pre-2022 idiom: `(SELECT id, name FOR JSON PATH, WITHOUT_ARRAY_WRAPPER)` — works but awkward for single rows. - JSON_OBJECT is convenient for constructing per-row JSON in SELECT lists, especially when building API responses directly in SQL. - `FOR JSON` is still preferred for building large result sets as a single JSON document — JSON_OBJECT is row-by-row.

What NOT to say: - Do not claim FOR JSON PATH is obsolete — it remains the right tool for result-set-to-JSON conversion. JSON_OBJECT is for per-row construction.

Common follow-up: *Show me both the FOR JSON PATH approach and the JSON_OBJECT approach for the same task.*

```
SELECT id,
  JSON_OBJECT(
    'id': id,
    'name': name,
    'tags': JSON_ARRAY(tag1, tag2)
  ) AS payload
FROM users;
```

Q146 · GREATEST and LEAST (2022)

Tags: Difficulty: Easy · Asked at: JP Morgan, Goldman Sachs, Wells Fargo, Walmart Labs · Engine: SQL Server 2022 · Topic: New functions

The question:

> SQL Server 2022 added GREATEST and LEAST. What did people use before, and why is the new version better?

The 30-second answer: GREATEST returns the largest of its argument list; LEAST the smallest. Before 2022, you had to use nested `CASE WHEN`, `IIF`, or a derived table with `MAX`. The new functions are cleaner, NULL-aware, and let the optimizer reason about them properly.

Step-by-step thought process: 1. Pre-2022 idiom: `CASE WHEN a > b THEN a ELSE b END` for two arguments, and it gets worse fast for three or more. 2. With 2022: `GREATEST(a, b, c, d)` — concise, no syntactic noise. 3. NULL handling: by default, NULL arguments are ignored. `GREATEST(5, NULL, 3) = 5`. To make NULLs "infect" the result, wrap with `IIF(... IS NULL, NULL, GREATEST(...))`. 4. Type coercion: arguments are converted to the highest-precedence type using standard T-SQL rules. Watch for INT + VARCHAR mixes.

Edge cases to mention: - The new functions accept up to 254 arguments — far more than you would typically need. - For row-wise max across columns of different tables, GREATEST + a join is much

cleaner than the old `(SELECT MAX(v) FROM (VALUES (a),(b),(c)) AS t(v))` derived-table trick. - ANSI SQL has these as GREATEST and LEAST too — SQL Server is just catching up.

What NOT to say: - Do not say SQL Server "never had" these — the old workarounds existed, just clumsier.

Common follow-up: *Show the pre-2022 equivalent of GREATEST(a, b, c) using CASE.*

```
SELECT order_id,
  GREATEST(ship_date, payment_date, confirmation_date) AS latest_event,
  LEAST(ship_date, payment_date, confirmation_date) AS earliest_event
FROM orders;
```

Q147 · IS [NOT] DISTINCT FROM (2022)

Tags: Difficulty: Easy · Asked at: JP Morgan, Goldman Sachs, Morgan Stanley, AmEx · Engine: SQL Server 2022 · Topic: NULL handling

The question:

> What does IS NOT DISTINCT FROM do, and why was it added in 2022?

The 30-second answer: `IS NOT DISTINCT FROM` is NULL-safe equality. `NULL IS NOT DISTINCT FROM NULL` is TRUE, unlike `NULL = NULL` which is UNKNOWN. Before 2022, T-SQL devs wrote `(a = b OR (a IS NULL AND b IS NULL))` everywhere. The new operator is shorter and more readable.

Step-by-step thought process: 1. SQL's standard equality `=` returns UNKNOWN when either side is NULL — neither true nor false. So `WHERE a = b` excludes rows where both are NULL. 2. `IS NOT DISTINCT FROM` treats NULL as equal to NULL. Two NULLs are "not distinct". A value and NULL are "distinct". 3. Common use: comparing rows for equality including NULLable columns. Change-data-capture, diff queries, slowly-changing-dimension comparisons. 4. `IS DISTINCT FROM` is the inverse — true when values are not equal OR exactly one is NULL.

Edge cases to mention: - Equivalent pre-2022 expression: `NULLIF(a, b) IS NULL AND NULLIF(b, a) IS NULL` (clever but unreadable) or `(a = b OR (a IS NULL AND b IS NULL))` (clear but verbose). - Used heavily in WHERE clauses for "find rows where value changed" in audit queries. - Performance: behaves like equality plus a NULL check — minor overhead compared to `=`.

What NOT to say: - Do not claim NULL = NULL is true in some specific mode — not under ANSI_NULLS ON (which has been the only supported mode for years).

Common follow-up: *Write a query to find rows in table A where the value in column X differs from the value in the corresponding row of table B, considering NULL changes.*

```
SELECT a.id
FROM A a JOIN B b ON a.id = b.id
WHERE a.status IS DISTINCT FROM b.status
   OR a.amount IS DISTINCT FROM b.amount;
```

Q148 · Ledger tables (2022)

Tags: Difficulty: Medium · Asked at: JP Morgan, Wells Fargo, Goldman Sachs, AmEx · Engine: SQL Server 2022 · Topic: Audit

The question:

> What are Ledger tables in SQL Server 2022 and when would you use them?

The 30-second answer: Ledger tables are append-only tables with cryptographic chaining (blockchain-style) of row hashes. They provide tamper-evident auditing — any modification breaks the hash chain and is detectable. Use them for regulatory-grade audit logs in finance, healthcare, or compliance-heavy workflows.

Step-by-step thought process: 1. Two flavours: **append-only ledger tables** (only INSERTs allowed; UPDATE / DELETE return errors) and **updatable ledger tables** (UPDATE and DELETE allowed but history tracked in an immutable ledger history table). 2. Each row has a hash that incorporates the previous row's hash. Tampering with one row makes its hash mismatch what it should be, and the discrepancy propagates. 3. The chain itself can be periodically committed to an external store (Azure Storage with immutable blobs, or an Azure Confidential Ledger) — this provides external proof that the chain has not been rebuilt after corruption. 4. Verification: `sys.sp_verify_database_ledger` reads through the chain and confirms its consistency. Run periodically (e.g. nightly) and on demand during audits.

Edge cases to mention: - Ledger tables make DROP TABLE harder — dropping them requires explicit removal of the protected metadata. - All ledger tables use SYSTEM_VERSIONING internally — they are essentially temporal tables with cryptographic chaining bolted on. - They do not prevent tampering — they detect it. A malicious actor with admin privileges could still corrupt data; the ledger guarantees you would notice.

What NOT to say: - Do not equate ledger tables to a blockchain in the cryptocurrency sense — they are a much simpler hash-chain primitive without consensus or P2P features.

Common follow-up: *If a malicious admin can drop the database, what stops them from also faking a valid ledger?*

Q149 · Query Store basics

Tags: Difficulty: Medium · Asked at: JP Morgan, Goldman Sachs, Walmart Labs, Target · Engine: SQL Server 2022 · Topic: Diagnostics

The question:

> What is Query Store and what does it capture?

The 30-second answer: Query Store is a built-in repository that captures query plans, execution statistics, and wait stats over time. It is the standard tool for plan-regression detection and historical performance analysis. Enabled by default for new databases in SQL Server 2022.

Step-by-step thought process: 1. Captures: query text (normalised), every distinct execution plan a query has used, execution counts and durations by time window, top waits per query. 2. Data persists across server restarts (stored in the user database), unlike cache-only DMVs which clear. 3. UI: SSMS exposes Query Store reports — Top Resource Consumers, Regressed Queries, Forced Plans, etc. 4. SQL Server 2022 enables Query Store by default on new databases. Earlier versions required explicit `ALTER DATABASE ... SET QUERY_STORE = ON`.

Edge cases to mention: - Storage: Query Store data eats database space. The MAX_STORAGE_SIZE_MB option controls the cap. When hit, Query Store switches to read-only mode and stops capturing new data. - Capture mode: ALL (every query), AUTO (filters out trivial ad-hoc), CUSTOM (your thresholds). AUTO is sensible for OLTP workloads. - Per-database — must be enabled on each database you want to instrument.

What NOT to say: - Do not call Query Store "an extended event session" — it is a separate, independent feature with its own storage and tooling.

Common follow-up: *If Query Store is full and switches to read-only, how do you recover?*

```
ALTER DATABASE shop SET QUERY_STORE (
  OPERATION_MODE = READ_WRITE,
  MAX_STORAGE_SIZE_MB = 1024,
  QUERY_CAPTURE_MODE = AUTO
);
```

Q150 · Plan regression and force_plan

Tags: Difficulty: Hard · Asked at: JP Morgan, Goldman Sachs, Morgan Stanley, AmEx · Engine: SQL Server 2022 · Topic: Diagnostics

The question:

> Last week a query was fast. This week the same query is slow. How does Query Store help you fix it?

The 30-second answer: Use the **Regressed Queries** report to find queries whose performance dropped. For each, Query Store shows the plans used and their stats over time. If a known-good plan exists, you can **force** it: `sp_query_store_force_plan @query_id, @plan_id`. Subsequent executions use the forced plan, bypassing the optimizer's recent (bad) choice.

Step-by-step thought process: 1. The optimizer may pick a different plan when statistics change, parameter values shift, or plan cache evicts — even for identical SQL. 2. Query Store retains both plans. The Regressed Queries report shows side-by-side metrics: duration, CPU, logical reads, per execution count. 3. Identify the previously-good plan's plan_id. Call `sp_query_store_force_plan` to pin it. 4. The optimizer is now constrained to use the forced plan. If it cannot (e.g. an index was dropped), it falls back and Query Store records the failure.

Edge cases to mention: - Forcing a plan is a tactical fix, not a permanent solution. Investigate the root cause: stats drift, parameter sniffing issues, missing index, etc. - 2022's **automatic plan correction** can auto-force a previously-good plan when it detects a regression. Enabled via `ALTER DATABASE ... SET AUTOMATIC_TUNING (FORCE_LAST_GOOD_PLAN = ON)`. - Forced plans show up in DMVs with a `plan_forcing_type` indicator. Easy to audit.

What NOT to say: - Do not say forcing a plan "guarantees" a query stays fast — if data distribution changes drastically, the forced plan may now be wrong.

Common follow-up: *Why might a "previously good" plan suddenly become bad even without forcing it?*

Q151 · Clustered columnstore for analytics

Tags: Difficulty: Medium · Asked at: Walmart Labs, Target, JP Morgan, AmEx · Engine: SQL Server 2022 · Topic: Columnstore

The question:

> What is a clustered columnstore index and why is it good for analytics?

The 30-second answer: A clustered columnstore index stores the table column-by-column instead of row-by-row, with each column compressed independently. Massive compression ratios (often 10x), and analytical scans only read the columns they need. Designed for OLAP / data warehouse workloads on tables with hundreds of millions of rows.

Step-by-step thought process: 1. Storage model: rows are grouped into "row groups" of approximately one million rows each. Within a row group, each column is stored as a compressed segment. 2. Compression types: dictionary (for low-cardinality columns), bit-packing, run-length, value compression. The engine picks per-segment. 3. Query execution uses **batch mode**: operate on batches of ~900 rows at a time using SIMD-style processing on the column data. Dramatically faster than row-mode for aggregations. 4. Segment elimination: each segment stores min/max metadata. Scans can skip entire segments where the predicate cannot match.

Edge cases to mention: - Clustered columnstore replaces the rowstore — the table is stored column-wise as the primary structure. No separate heap or B-tree. - INSERT performance: row-by-row inserts go to a delta store (a small B-tree). Background process compresses delta into compressed row groups when delta exceeds ~100K rows. Batch inserts of ≥102,400 rows go directly to compressed row groups — much more efficient. - Updates: implemented as delete + insert internally. Heavy update workloads cause fragmentation.

What NOT to say: - Do not use clustered columnstore for OLTP single-row lookups — they are 10-100x slower than rowstore for that workload.

Common follow-up: *How does segment elimination work, and what query patterns benefit most?*

Q152 · Nonclustered columnstore and HTAP

Tags: Difficulty: Medium · Asked at: Walmart Labs, AmEx, JP Morgan, Target · Engine: SQL Server 2022 · Topic: Columnstore

The question:

> What is a nonclustered columnstore index? When would you use one alongside a regular B-tree?

The 30-second answer: A nonclustered columnstore index sits on top of a rowstore table — the table itself remains row-organised, with the columnstore as a secondary structure. Used for **HTAP** (hybrid transactional / analytical processing): OLTP queries use the rowstore, analytical queries use the columnstore, on the same table.

Step-by-step thought process: 1. Setup: `CREATE NONCLUSTERED COLUMNSTORE INDEX ix_orders_analytics ON orders (customer_id, product_id, amount, order_date)`. 2. The rowstore remains the primary structure with its own B-tree clustered index, supporting point lookups and small range scans efficiently. 3. The columnstore is maintained in parallel. Aggregations and large scans use it via batch-mode processing. 4. From SQL Server 2016+, the columnstore can be **updatable** — keep the OLTP writes going while the columnstore stays current.

Edge cases to mention: - Filtered nonclustered columnstore: `... WHERE order_date >= '2024-01-01'`. Useful for keeping only "warm" data in the columnstore while excluding ancient

history. - Maintenance overhead: every write to the rowstore is mirrored to the columnstore. Heavy write workloads pay an extra cost. - The optimizer chooses between rowstore and columnstore per query based on cost — sometimes you may force one with hints.

What NOT to say: - Do not say nonclustered columnstore "replaces" the rowstore — they coexist by design.

Common follow-up: *When does the cost of maintaining a nonclustered columnstore not pay off?*

Q153 · Read Committed and READ_COMMITTED_SNAPSHOT

Tags: Difficulty: Medium · Asked at: JP Morgan, Goldman Sachs, Wells Fargo, AmEx · Engine: SQL Server 2022 · Topic: Isolation

The question:

> What is SQL Server's default isolation level? What is READ_COMMITTED_SNAPSHOT and how does it change things?

The 30-second answer: Default isolation is Read Committed. By default it uses shared locks for reads, blocking on uncommitted writers. `READ_COMMITTED_SNAPSHOT = ON` (database-level) switches it to use row versions: readers see the last-committed version without blocking, writers block writers but not readers.

Step-by-step thought process: 1. Default Read Committed in SQL Server uses **locking** — short shared lock per row during read. Blocks if any writer holds an exclusive lock. 2. Enable RCSI: `ALTER DATABASE shop SET READ_COMMITTED_SNAPSHOT ON` — requires no active connections to the DB. 3. After enabling: readers see a snapshot of each row as of the start of the **statement**, not the transaction. So multiple statements in a transaction may see different snapshots, consistent with Read Committed semantics. 4. Side effect: tempdb grows because row versions are stored there. Watch tempdb sizing.

Edge cases to mention: - Snapshot Isolation (a separate isolation level, `SET TRANSACTION ISOLATION LEVEL SNAPSHOT`) gives a snapshot at the **transaction start**, providing read consistency across statements. This is opt-in per session, not the default. - RCSI is widely recommended for OLTP workloads in modern SQL Server — eliminates a huge class of reader-writer blocking. - New databases in Azure SQL Database have RCSI on by default. On-prem 2022 requires explicit opt-in.

What NOT to say: - Do not say RCSI is "the same as Oracle's snapshot isolation" — close, but the snapshot scope differs (statement vs transaction).

Common follow-up: *If you enable RCSI, can you ever still have reader-writer blocking?*

Q154 · NOLOCK and its pitfalls

Tags: Difficulty: Medium · Asked at: JP Morgan, Goldman Sachs, Wells Fargo, AmEx · Engine: SQL Server 2022 · Topic: Isolation

The question:

> What does WITH (NOLOCK) do? Why is it dangerous?

The 30-second answer: `WITH (NOLOCK)` is equivalent to `SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED` at the query level. It allows reading uncommitted data. Risks include **dirty reads**, **inconsistent reads** (rows seen twice or missed due to page splits), and **wrong aggregate results**. Almost always the wrong choice in 2026 — enable RCSI instead.

Step-by-step thought process: 1. Dirty read: a query reads data from an in-progress transaction that later rolls back. The query sees data that never existed. 2. Inconsistent read: while scanning a clustered index, a page split moves rows to new pages. The scan may visit those rows twice or skip them entirely. Aggregates over the scan are then wrong. 3. NOLOCK can produce errors like "could not continue scan with NOLOCK due to data movement" — happens when allocation maps change during the scan. 4. Historically devs added NOLOCK to "make queries faster" — a misconception. The real fix is RCSI, which provides non-blocking reads without dirty data risk.

Edge cases to mention: - NOLOCK does NOT make a query faster intrinsically. Any speedup is purely from avoiding the wait on a blocker — and avoiding the wait by reading garbage is rarely what you want. - Reports run with NOLOCK can give different totals on consecutive runs even with no actual changes — extremely confusing during audits. - WITH (READPAST) skips locked rows instead of reading uncommitted — a sometimes-safer alternative but with its own correctness implications.

What NOT to say: - Do not recommend NOLOCK as a default. It is a code smell in 2026.

Common follow-up: *What is the modern alternative to NOLOCK for non-blocking reads?*

Q155 · Lock escalation

Tags: Difficulty: Hard · Asked at: Goldman Sachs, JP Morgan, Wells Fargo, Morgan Stanley · Engine: SQL Server 2022 · Topic: Locking

The question:

> What is lock escalation and what triggers it?

The 30-second answer: Lock escalation is when SQL Server promotes many fine-grained locks (row or page) to a single coarse-grained lock (table or partition) to reduce lock manager overhead. Default threshold is around 5000 locks on a single object in a single statement. Once escalated, concurrent access to the table is severely restricted.

Step-by-step thought process: 1. The lock manager maintains in-memory state for every lock. Many locks per statement bloat memory and slow lock acquisition. 2. When SQL Server estimates over ~5000 locks on a single object, it tries to escalate to a table-level (or partition-level if partitioned with LOCK_ESCALATION = AUTO) lock. 3. If the escalation succeeds, concurrent transactions are blocked from any conflicting access to that whole object. 4. If another transaction holds a conflicting lock anywhere on the object, escalation fails and SQL Server retries periodically as the lock count grows.

Edge cases to mention: - `ALTER TABLE ... SET (LOCK_ESCALATION = AUTO | TABLE | DISABLE)`. AUTO escalates to partition for partitioned tables. DISABLE prevents escalation entirely — useful for hot tables. - Trace flag 1224 disables lock escalation server-wide. Sometimes used in OLTP-heavy environments where escalation causes more pain than the lock manager overhead. - Best mitigation: keep transactions small. A statement that updates 100K rows is asking for escalation. Batch in groups of 1000-2000 rows with explicit transactions per batch.

What NOT to say: - Do not say lock escalation is "always bad" — it has a real purpose. Disabling it can cause memory pressure on the lock manager.

Common follow-up: *How would you batch a large UPDATE to avoid escalation?*

```
DECLARE @rows INT = 1;
WHILE @rows > 0
BEGIN
  UPDATE TOP (2000) orders SET status = 'archived'
  WHERE archived = 0 AND order_date < '2024-01-01';
  SET @rows = @@ROWCOUNT;
END
```

Q156 · Always On Availability Groups

Tags: Difficulty: Medium · Asked at: JP Morgan, Wells Fargo, Goldman Sachs, AmEx · Engine: SQL Server 2022 · Topic: HA

The question:

> What is Always On Availability Groups? How does it differ from older mirroring?

The 30-second answer: Availability Groups (introduced in 2012, refined since) is SQL Server's modern HA technology. A group of databases is replicated to one or more secondary replicas at the log block

level. Replicas can be sync or async, automatic failover between sync replicas, listeners provide a single virtual endpoint for clients.

Step-by-step thought process: 1. Group concept: instead of one database mirrored, a *set* of databases fails over together. Critical when a single application requires multiple correlated databases. 2. Up to 8 secondary replicas in SQL Server 2016+. Each replica can be readable, allowing read-scale-out. 3. Sync replicas: log records are committed on the secondary before the primary commit returns to the client. Strong durability but transaction latency suffers if the network is slow. 4. Async replicas: log shipped asynchronously. No commit latency impact on the primary but possible data loss on failover.

Edge cases to mention: - AG Listener: a virtual DNS name + VIP that always points to the current primary. Applications connect via the listener, not directly to a server, and failover is transparent. - Distributed Availability Groups: AGs across multiple AGs, often used for cross-region disaster recovery. - Built on Windows Server Failover Clustering historically; Linux supports AGs via Pacemaker.

What NOT to say: - Do not say AGs are "the same as mirroring". Mirroring was per-database, only 1 secondary, no read access on the secondary. AGs supersede mirroring entirely (mirroring is deprecated).

Common follow-up: *What is the trade-off between sync and async commit for a given AG replica?*

Q157 · Sync vs async commit and listeners

Tags: Difficulty: Medium · Asked at: JP Morgan, Goldman Sachs, Walmart Labs, AmEx · Engine: SQL Server 2022 · Topic: HA

The question:

> Walk me through what happens when a client commits a transaction with a sync replica vs an async replica.

The 30-second answer: **Sync**: primary writes the log record, sends it to the secondary, waits for the secondary's hardened-on-disk acknowledgement, then returns commit success to the client. **Async**: primary writes the log record, queues it for shipping, returns commit success immediately. The secondary catches up in the background.

Step-by-step thought process: 1. Sync commit guarantees zero data loss for committed transactions if the primary fails — the secondary always has the log. 2. Sync commit imposes the round-trip latency of the secondary on every commit. Across data centres in different regions, this can be a deal-breaker for high-throughput OLTP. 3. Async commit imposes essentially no extra latency on commit, but during a failover transactions in flight may be lost. 4. Typical real-world setup: one sync secondary in the same data centre for HA, one async secondary in a remote DR site for disaster recovery.

Edge cases to mention: - Automatic failover only between sync secondaries — async ones require manual failover (and cause data loss). - Multi-subnet listeners: AG listener can span subnets across data centres. Connection string parameter `MultiSubnetFailover=True` for correct client-side handling. - Read-only routing: client can request `ApplicationIntent=ReadOnly` and the listener redirects to a readable secondary, reducing primary load.

What NOT to say: - Do not say async failover is "lossless" — by definition, async data may not have reached the secondary at failure time.

Common follow-up: *Why is automatic failover only allowed for sync replicas?*

Q158 · Indexed views

Tags: Difficulty: Hard · Asked at: JP Morgan, Goldman Sachs, Walmart Labs, AmEx · Engine: SQL Server 2022 · Topic: Materialized views

The question:

> What is an indexed view in SQL Server? What are the restrictions and use cases?

The 30-second answer: An indexed view is SQL Server's name for a materialized view. You create a regular view, then add a unique clustered index to it — that materializes the view's output as persistent rows. Restrictions: the view must be SCHEMABINDING, use only deterministic operators, and follow many other rules.

Step-by-step thought process: 1. `CREATE VIEW vw_sales_by_day WITH SCHEMABINDING AS SELECT order_date, COUNT_BIG(*) AS num_orders, SUM(amount) AS total FROM dbo.orders GROUP BY order_date;` then `CREATE UNIQUE CLUSTERED INDEX ix_vw_sales ON vw_sales_by_day (order_date);`. 2. SCHEMABINDING locks the view's dependent objects from change — you cannot drop / alter the underlying tables in incompatible ways. 3. The view's output is materialized: queries against the view (or against the base tables that the optimizer determines match the view) use the materialized rows directly. 4. Aggregate views must use `COUNT_BIG(*)` (not `COUNT(*)`) so that the engine can incrementally maintain row counts as base data changes.

Edge cases to mention: - Many T-SQL constructs are forbidden in indexed views: outer joins, subqueries in FROM, non-deterministic functions (like `GETDATE()`), TOP, DISTINCT, UNION. - Maintenance cost: every insert / update / delete on the base tables must update the materialized view too. Can be expensive for hot tables. - Optimizer "noexpand" — sometimes you need the `WITH (NOEXPAND)` hint on the view reference to ensure the optimizer uses the materialized version rather than expanding the view definition.

What NOT to say: - Do not call indexed views "the same as Oracle materialized views" — Oracle has incremental refresh on a schedule. Indexed views are synchronously maintained.

Common follow-up: *Why is COUNT_BIG required instead of COUNT for an indexed view with aggregates?*

Q159 · Temporal tables (system-versioned)

Tags: Difficulty: Medium · Asked at: JP Morgan, Wells Fargo, Goldman Sachs, AmEx · Engine: SQL Server 2022 · Topic: Audit

The question:

> What is a system-versioned temporal table? How do you query its history?

The 30-second answer: A system-versioned temporal table is a pair of tables: the current table plus a history table that automatically captures previous versions of every row. Every `INSERT` / `UPDATE` / `DELETE` writes the prior row to history. You query history with `FOR SYSTEM_TIME AS OF '...'` and related clauses.

Step-by-step thought process: 1. Define a table with two `PERIOD` columns (`SysStartTime` , `SysEndTime`) and `SYSTEM_VERSIONING = ON (HISTORY_TABLE = ...)` . From the database point of view, the history table is automatic. 2. On update, the row's old version is inserted into the history table with `SysEndTime = current time` . The new version sits in the main table with `SysStartTime = current time` , `SysEndTime = 9999-12-31` . 3. Query history: `SELECT * FROM orders FOR SYSTEM_TIME AS OF '2026-04-01' WHERE id = 42;` returns the state of order 42 as of April 1. 4. Other temporal predicates: `BETWEEN` , `CONTAINED IN` , `FROM ... TO ...` , `ALL` . ALL returns current + all history rows.

Edge cases to mention: - The history table is read-only from the application's perspective — you cannot directly modify it without disabling system versioning. - Storage: history can be on a separate filegroup. For analytics, history could even be a clustered columnstore index to compress old data. - Retention: by default unlimited. Use `HISTORY_RETENTION_PERIOD` to auto-clean old history.

What NOT to say: - Do not say temporal tables are the same as ledger tables — temporal is for time-travel queries, ledger adds cryptographic tamper-evidence.

Common follow-up: *How would you find every order whose status was 'cancelled' at any point in the past month?*

```
SELECT * FROM orders FOR SYSTEM_TIME AS OF '2026-05-01' WHERE id = 42;

SELECT * FROM orders FOR SYSTEM_TIME
  BETWEEN '2026-04-01' AND '2026-05-01'
  WHERE status = 'cancelled';
```

Q160 · 2022 features summary and migration

Tags: Difficulty: Easy · Asked at: JP Morgan, Goldman Sachs, Walmart Labs, AmEx · Engine: SQL Server 2022 · Topic: Versions

The question:

> If you are migrating from SQL Server 2019 to 2022, what are the headline features you would highlight?

The 30-second answer: Headline 2022 additions: **GREATEST / LEAST**, **IS [NOT] DISTINCT FROM**, **JSON_OBJECT / JSON_ARRAY**, **window function IGNORE NULLS**, **ledger tables**, **named windows**, plus enhancements to Query Store, intelligent query processing (parameter-sensitive plan optimization), and Azure-integrated services (Synapse Link, Object Storage backup).

Step-by-step thought process: 1. T-SQL language additions — the cluster mentioned above (GREATEST, LEAST, IS DISTINCT FROM, JSON_OBJECT, JSON_ARRAY, IGNORE NULLS) eliminate a lot of legacy workaround code. 2. Compliance / security: ledger tables (already covered), enhanced TDE, Microsoft Entra (formerly Azure AD) authentication for on-prem instances. 3. Performance: Intelligent Query Processing (IQP) expansions — Parameter Sensitive Plan optimization automatically caches multiple plans for a parameterised query when input cardinality varies a lot. 4. Cloud integration: backup to Azure Object Storage (S3-compatible), Azure Synapse Link for near-real-time analytics on operational data.

Edge cases to mention: - Compatibility level: even after migration, the database stays on its previous compatibility level unless you raise it. New optimizer features only kick in at compat level 160 (2022). - Some 2022 features require specific SKUs (Enterprise edition). Verify your licensing before promising features in interview answers. - Backward compatibility for clients is generally strong — driver compatibility usually carries over without changes.

What NOT to say: - Do not over-promise. If you have not personally used a feature, hedge — "I have read about it, looks useful for X".

Common follow-up: *Of these 2022 features, which one would benefit your team most and why?*

End of Sections 4 + 5. Total: 50 questions (25 MySQL 8.x + 25 SQL Server 2022). Where engine syntax differs between MySQL and SQL Server, the question made the distinction explicit so a candidate prepping for both can answer either dialect comfortably.